

Developer Specification (DEV_SPEC)

版本: 0.1 — 文档结构草案

目录

- 项目概述
- 核心特点
- 技术选型
- 测试方案
- 系统架构与模块设计
- 项目排期
- 可扩展性与未来展望

1. 项目概述

本项目基于多阶段检索增强生成 (RAG, Retrieval-Augmented Generation) 与模型上下文协议 (MCP, Model Context Protocol) 设计, 目标是搭建一个可扩展、高可观测、易迭代的智能问答与知识检索框架。

设计理念 (Design Philosophy)

核心定位: 自学与教学同步 (Learning by Teaching)

本项目是我个人技术学习、丰富简历、备战面试的实战历程, 同时也是一份同步教学的开源资源。我相信**"教是最好的学"**——在整理代码、撰写文档、录制视频的过程中, 我自己对 RAG 的理解也在不断深化。希望这份"边学边教"的成果能够帮助到更多同样在求职路上的朋友。

本项目不仅是一个功能完备的智能问答框架, 更是一个专为 **RAG 技术学习与面试求职** 设计的实战平台:

1 实战驱动学习 (Learn by Doing)

项目架构本身就是 RAG 面试题的**"活体答案"**。我们将经典面试考点直接融入代码设计, 通过动手实践来巩固理论知识:

- 分层检索 (Hierarchical Retrieval)
- Hybrid Search (BM25 + Dense Embedding)
- Rerank 重排序机制
- Embedding 策略与优化
- RAG 性能评测 (Ragas/DeepEval)

2 开箱即用与深度扩展并重 (Plug-and-Play & Extensible)

- 开箱即用**: 提供 MCP 标准接口, 可直接对接 Copilot/Claude, 拿到项目即可运行体验。
- 深度扩展**: 保留完全模块化的内部结构, 方便开发者替换组件、魔改算法, 作为具备深度的个人简历项目。
- 扩展指引**: 文档中会明确指出各模块的扩展方向与建议, 帮助你在掌握基础后继续深入迭代。

3 配套教学资源 (Comprehensive Learning Materials)

我会提供三位一体的配套学习资源，帮助你快速吃透项目：

资源类型	内容说明
📄 技术文档	架构设计文档、技术选型说明、模块详解
💻 代码示范	带详细注释的源码、关键模块的 Step-by-step 实现
🎥 视频讲解	RAG 核心知识点回顾、代码细节精讲、环境配置教程

4 学习路线与面试指南 (Study Guide & Interview Prep)

针对每个模块，我会整理：

- 📋 **知识点清单**：这块涉及哪些理论知识需要提前学习（如 BM25 原理、FAISS 索引类型、Cross-Encoder vs Bi-Encoder）
- ❓ **高频面试题**：结合项目代码讲解常见面试问题及参考答案
- 📝 **简历撰写建议**：如何将本项目的亮点写进简历，突出技术深度

5 社区交流与持续迭代 (Community & Iteration)

- 经验分享**：我自己的面试经历、大家使用本项目面试的反馈，都会汇总沉淀
- 问题讨论**：一起探讨"如何将本项目写进简历"、"针对本项目的面试题怎么答"
- 持续更新**：从代码 → 八股知识 → 面试技巧，形成完整的求职知识库，帮助大家更好地拿到 Offer 🚀

2. 核心特点

RAG 策略与设计亮点

本项目在 RAG 链路的关键环节采用了经典的工程化优化策略，平衡了检索的查准率与查全率，具体思想如下：

- 分块策略 (Chunking Strategy)**：采用智能分块与上下文增强，为高质量检索打下基础。
 - 智能分块**：摒弃机械的定长切分，采用语义感知的切分策略以保留完整语义；
 - 上下文增强**：为 Chunk 注入文档元数据（标题、页码）和图片描述（Image Caption），确保检索时不仅匹配文本，还能感知上下文。
- 粗排召回 (Coarse Recall / Hybrid Search)**：采用 **混合检索** 策略作为第一阶段召回，快速筛选候选集。
 - 结合 **稀疏检索 (Sparse Retrieval/BM25)** 利用关键词精确匹配，解决专有名词查找问题；
 - 结合 **稠密检索 (Dense Retrieval/Embedding)** 利用语义向量，解决同义词与模糊表达问题；
 - 两者互补，通过 RRF (Reciprocal Rank Fusion) 算法融合，确保查全率与查准率的平衡。
- 精排重排 (Rerank / Fine Ranking)**：在粗排召回的基础上进行深度语义排序。
 - 采用 Cross-Encoder（专用重排模型）或 LLM Rerank（可选后端）对候选集进行逐一打分，识别细微的语义差异。
 - 通过 **"粗排(低成本泛召回) -> 精排(高成本精过滤)"** 的两段式架构，在不牺牲整体响应速度的前提下大幅提升 Top-Results 的精准度。

全链路可插拔架构 (Pluggable Architecture)

鉴于 AI 技术的快速演进，本项目在架构设计上追求**极致的灵活性**，拒绝与特定模型或供应商强绑定。**整个系统**（不仅是 RAG 链路）的每一个核心环节均定义了抽象接口，支持"乐高积木式"的自由替换与组合：

- **LLM 调用层插拔 (LLM Provider Agnostic)：**
 - 核心推理 LLM 通过统一的抽象接口封装，支持**多协议无缝切换**：
 - **Azure OpenAI**：企业级 Azure 云端服务，符合合规与安全要求；
 - **OpenAI API**：直接对接 OpenAI 官方接口；
 - **本地模型**：支持 Ollama、vLLM、LM Studio 等本地私有化部署方案；
 - **其他云服务**：DeepSeek、Anthropic Claude 等第三方 API。
 - 通过配置文件一键切换后端，**零代码修改**即可完成 LLM 迁移，便于成本优化、隐私合规或 A/B 测试。
- **Embedding & Rerank 模型插拔 (Model Agnostic)：**
 - Embedding 模型与 Rerank 模型同样采用统一接口封装；
 - 支持云端服务（OpenAI Embedding, Cohere Rerank）与本地模型（Sentence-Transformers, BGE）自由切换。
- **RAG Pipeline 组件插拔：**
 - **Loader（解析器）**：支持 PDF、Markdown、Code 等多种文档解析器独立替换；
 - **Smart Splitter（切分策略）**：语义切分、定长切分、递归切分等策略可配置；
 - **Transformation（元数据/图文增强逻辑）**：OCR、Image Captioning 等增强模块可独立配置。
- **检索策略插拔 (Retrieval Strategy)：**
 - 支持动态配置纯向量、纯关键词或混合检索模式；
 - 支持灵活更换向量数据库后端（如从 Chroma 迁移至 Qdrant、Milvus）。
- **评估体系插拔 (Evaluation Framework)：**
 - 评估模块不锁定单一指标，支持挂载不同的 Evaluator（如 Ragas, DeepEval）以适应不同的业务考核维度。

这种设计确保开发者可以**零代码修改**即可进行 A/B 测试、成本优化或隐私迁移，使系统具备极强的生命力与环境适应性。

MCP 生态集成 (Copilot / ReSearch)

本项目的核心设计完全遵循 Model Context Protocol (MCP) 标准，这使得它不仅是一个独立的问答服务，更是一个即插即用的知识上下文提供者。

- **工作原理：**
 - 我们的 Server 作为一个 **MCP Server** 运行，暴露一组标准的 `tools` 和 `resources` 接口。
 - **MCP Clients**（如 GitHub Copilot, ReSearch Agent, Claude Desktop 等）可以直接连接到这个 Server。
 - **无缝接入**：当你在 GitHub Copilot 中提问时，Copilot 作为一个 MCP Host，能够自动发现并调用我们的 Server 提供的工具（如 `search_documentation`），获取我们内置的私有文档知识，然后结合这些上下文来回答你的问题。
- **优势：**

- **零前端开发**：无需为知识库开发专门的 Chat UI，直接复用开发者已有的编辑器（VS Code）和 AI 助手。
- **上下文互通**：Copilot 可以同时看到你的代码文件和我们的知识库内容，进行更深度的推理。
- **标准兼容**：任何支持 MCP 的 AI Agent（不仅是 Copilot）都可以即刻接入我们的知识库，一次开发，处处可用。

多模态图像处理 (Multimodal Image Processing)

本项目采用了经典的 "Image-to-Text" (图转文) 策略来处理文档中的图像内容，实现了低成本且高效的多模态检索：

- **图像描述生成 (Captioning)**：利用 LLM 的视觉能力，自动提取文档中插图的核心信息，并生成详细的文字描述 (Caption)。
- **统一向量空间**：将生成的图像描述文字直接嵌入到文档文本块 (Chunk) 中进行向量化。
- **优势**：
 - **架构统一**：无需引入复杂的 CLIP 等多模态向量库，复用现有的纯文本 RAG 检索链路即可实现“搜文字出图”。
 - **语义对齐**：通过 LLM 将图像的视觉特征转化为语义理解，使用户能通过自然语言精准检索到图表、流程图等视觉信息。

可观测性、可视化管理与评估体系 (Observability, Visual Management & Evaluation)

针对 RAG 系统常见的“黑盒”问题，本项目致力于让每一次生成过程都**透明可见且可量化**，并提供完整的**本地可视化管理平台**：

- **全链路白盒化 (White-box Tracing)**：
 - 记录并可视化 RAG 流水线的每一个中间状态：覆盖 Ingestion（加载→切分→增强→编码→存储）与 Query（查询预处理→Dense/Sparse 召回→融合→重排→响应构建）两条完整链路。
 - 开发者可以清晰看到“系统为什么选了这个文档”以及“Rerank 起了什么作用”，从而精准定位坏 Case。
- **可视化管理平台 (Visual Management Dashboard)**：
 - 基于 Streamlit 的本地 Web 管理面板，提供六大功能页面：
 - **系统总览**：展示当前可插拔组件配置 (LLM/Embedding/Splitter/Reranker) 与数据资产统计。
 - **数据浏览器**：查看已索引的文档列表、Chunk 详情 (原文、metadata 各字段、关联图片)，支持搜索过滤。
 - **Ingestion 管理**：通过界面选择文件触发摄取、实时展示各阶段进度、支持删除已摄入文档 (跨 4 个存储的协调删除)。
 - **Query 追踪**：查询历史列表，耗时瀑布图，Dense/Sparse 召回对比，Rerank 前后排名变化。
 - **Ingestion 追踪**：摄取历史列表，各阶段耗时与处理详情。
 - **评估面板**：运行评估任务、查看各项指标、历史趋势对比。
 - 所有页面基于 Trace 中的 `method/provider` 字段**动态渲染**，更换可插拔组件后 Dashboard 自动适配，无需修改代码。
- **自动化评估闭环 (Automated Evaluation)**：
 - 集成 Ragas 等评估框架 (可插拔)，为每一次检索和生成计算“体检报告” (如召回率 Hit Rate、准确性

Faithfulness 等指标)。

- 拒绝“凭感觉”调优，建立基于数据的迭代反馈回路，确保每一次策略调整（如修改 Chunk Size 或更换 Reranker）都有量化的分数支撑。

业务可扩展性 (Extensibility for Your Own Projects)

本项目采用**通用化架构设计**，不仅是一个开箱即用的知识问答系统，更是一个可以快速适配各类业务场景的**扩展基座**：

- **Agent 客户端扩展 (Build Your Own Agent Client):**
 - 本项目的 MCP Server 天然支持被各类 Agent 调用，你可以基于此构建属于自己的 Agent 客户端：
 - **学习 Agent 开发**：通过实现一个调用本 Server 的 Agent，深入理解 Agent 的核心概念（Tool Calling、Chain of Thought、ReAct 模式等）；
 - **定制业务 Agent**：结合你的具体业务需求，开发专属的智能助手（如代码审查 Agent、文档写作 Agent、客服问答 Agent）；
 - **多 Agent 协作**：将本 Server 作为知识检索 Agent，与其他功能 Agent（如代码生成、任务规划）组合，构建复杂的 Multi-Agent 系统。
- **业务场景快速适配 (Adapt to Your Domain):**
 - **数据层扩展**：只需替换数据源（接入你自己的文档、数据库、API），即可将本系统改造为你的私有知识库；
 - **检索逻辑定制**：基于可插拔架构，轻松调整检索策略以适配不同业务特点（如电商搜索偏重关键词、法律文档偏重语义）；
 - **Prompt 模板定制**：修改系统 Prompt 和输出格式，使其符合你的业务风格与专业术语。
- **学习与实战并重 (Learn While Building):**
 - 通过扩展本项目，你将同步掌握：
 - **Agent 架构设计**：Function Calling、Tool Use、Memory 管理等核心概念；
 - **LLM 应用工程化**：Prompt Engineering、Token 优化、流式输出等实战技能；
 - **系统集成能力**：如何将 AI 能力嵌入现有业务系统，构建端到端的智能应用。

这种设计让本项目不仅是“学完即弃”的 Demo，而是可以**持续迭代、真正落地**的工程化模板，帮助你学到的知识转化为实际项目经验。

3. 技术选型

3.1 RAG 核心流水线设计

3.1.1 数据摄取流水线

目标：构建统一、可配置且可观测的数据摄取流水线，覆盖文档加载、格式解析、语义切分、多模态增强、嵌入计算、去重与批量上载到向量存储。该能力应是可重用的库模块，便于在 `ingest.py`、Dashboard 管理面板、离线批处理和测试中调用。

- **自研 Pipeline 框架（设计灵感参考 LlamaIndex 分层思想，但不依赖 LlamaIndex 库）：**
 - 采用自定义抽象接口
(`BaseLoader` / `BaseSplitter` / `BaseTransform` / `BaseEmbedding` / `BaseVectorStore`)，实现完全可控的可插拔架构。

- 支持可组合的 Loader -> Splitter -> Transform -> Embed -> Upsert 流程，便于实现可观测的流水线。
- 与主流 embedding provider 有良好适配，架构中统一使用 Chroma 作为向量存储。

设计要点：

- **明确分层职责：**

- Loader：负责把原始文件解析为统一的 `Document` 对象（`text` + `metadata`；类型定义集中在 `src/core/types.py`）。**在当前阶段，仅实现 PDF 格式的 Loader。**
 - 统一输出格式采用规范化 Markdown 作为 `Document.text`：这样可以更好的配合后面的 Splitter（Langchain RecursiveCharacterTextSplitter）方法产出高质量切块。
 - Loader 同时抽取/补齐基础 metadata（如 `source_path`, `doc_type=pdf`, `page`, `title/heading_outline`, `images` 引用列表等），为定位、回溯与后续 Transform 提供依据。
- Splitter：基于 Markdown 结构（标题/段落/代码块等）与参数配置把 `Document` 切为若干 Chunk，保留原始位置与上下文引用。
- Transform：可插入的处理步骤（ImageCaptioning、OCR、code-block normalization、html-to-text cleanup 等），Transform 可以选择把额外信息追加到 `chunk.text` 或放入 `chunk.metadata`（推荐默认追加到 `text` 以保证检索覆盖）。
- Embed & Upsert：按批次计算 embedding，并上载到向量存储；支持向量 + metadata 上载，并提供幂等 upsert 策略（基于 id/hash）。
- Dedup & Normalize：在上载前运行向量/文本去重与哈希过滤，避免重复索引。

关键实现要素：

- Loader（统一格式与元数据）
 - **前置去重 (Early Exit / File Integrity Check)：**
 - 机制：在解析文件前，计算原始文件的 SHA256 哈希指纹。
 - 动作：检索 `ingestion_history` 表，若发现相同 Hash 且状态为 `success` 的记录，则认定该文件未发生变更，直接跳过后续所有处理（解析、切分、LLM重写），实现**零成本 (Zero-Cost)** 的增量更新。
 - **存储方案**（初期实现，可插拔）：
 - **默认选择**：SQLite，存储于 `data/db/ingestion_history.db`
 - **表结构**：

```
CREATE TABLE ingestion_history (  
    file_hash TEXT PRIMARY KEY,  
    file_path TEXT NOT NULL,  
    file_size INTEGER,  
    status TEXT NOT NULL CHECK(status IN ('success', 'failed',  
'processing')),  
    processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    error_msg TEXT,  
    chunk_count INTEGER  
);  
  
CREATE INDEX idx_status ON ingestion_history(status);  
CREATE INDEX idx_processed_at ON ingestion_history(processed_at);
```


- **查询逻辑**: `SELECT status FROM ingestion_history WHERE file_hash = ? AND status = 'success'`
- **替换路径**: 后续可升级为 Redis (分布式缓存) 或 PostgreSQL (企业级中心化存储)

🔥 持久化存储架构统一说明

本项目在多个核心模块中采用 **SQLite** 作为轻量级持久化存储方案，避免引入重量级数据库依赖，保持本地优先 (Local-First) 的设计理念：

存储模块	数据库文件	用途	表结构关键字段
文件完整性检查	<code>data/db/ingestion_history.db</code>	记录已处理文件的 SHA256 哈希，实现增量摄取	<code>file_hash</code> , <code>status</code> , <code>processed_at</code>
图片索引映射	<code>data/db/image_index.db</code>	记录 <code>image_id</code> → 文件路径映射，支持图片检索与引用	<code>image_id</code> , <code>file_path</code> , <code>collection</code>
BM25 索引元数据	<code>data/db/bm25/</code>	存储倒排索引和 IDF 统计信息（未来可扩展用 SQLite）	当前使用 pickle，可迁移至 SQLite

设计优势：

- **零依赖部署**：无需安装 MySQL/PostgreSQL 等数据库服务，`pip install` 即可运行
- **并发安全**：WAL (Write-Ahead Logging) 模式支持多进程安全读写
- **持久化保证**：摄取历史和索引映射在进程重启后自动恢复，避免重复计算
- **架构一致性**：所有 SQLite 模块遵循相同的初始化、查询与错误处理模式，便于维护与扩展

升级路径：当系统规模扩展至分布式场景时，可通过统一的抽象接口将 SQLite 替换为 PostgreSQL 或 Redis，无需修改上层业务逻辑。

- **解析与标准化**：
 - 当前范围：仅实现 **PDF -> canonical Markdown 子集** 的转换。
- 技术选型 (Python PDF -> Markdown) :
 - **首选：MarkItDown** (作为默认 PDF 解析/转换引擎)。优点是直接产出 Markdown 形态文本，便于与后续 `RecursiveCharacterTextSplitter` 的 `separators` 配合。
- 输出标准 `Document` : `id|source|text(markdown)|metadata`。metadata 至少包含 `source_path`, `doc_type`, `title/heading_outline`, `page/slide` (如适用), `images` (图片引用列表)。
- Loader 不负责切分：只做“格式统一 + 结构抽取 + 引用收集”，确保切分策略可独立迭代与度量。
- Splitter (LangChain 负责切分；独立、可控)
 - **实现方案：使用 LangChain 的 `RecursiveCharacterTextSplitter` 进行切分。**
 - 优势：该方法对 Markdown 文档的结构 (标题、段落、列表、代码块) 有天然的适配性，能够通过配置语义断点 (Separators) 实现高质量、语义完整的切块。

- Splitter 输入: Loader 产出的 Markdown Document。
- Splitter 输出: 若干 Chunk (或 Document-like chunks), 每个 chunk 必须携带稳定的定位信息与来源信息: source, chunk_index, start_offset/end_offset (或等价定位字段)。
- Transform & Enrichment (结构转换与深度增强)

本阶段是 ETL 管道的核心“智力”环节, 负责将 Splitter 产出的非结构化文本块转化为结构化、富语义的智能切片 (Smart Chunk)。

 - **结构转换 (Structure Transformation):** 将原始的 String 类型数据转化为强类型的 Record/Object, 为下游检索提供字段级支持。
 - **核心增强策略:**
 1. **智能重组 (Smart Chunking & Refinement):**
 - 策略: 利用 LLM 的语义理解能力, 对上一阶段“粗切分”的片段进行二次加工。
 - 动作: 合并逻辑上紧密相关但被物理切断的段落, 剔除无意义的页眉页脚或乱码 (去噪), 确保每个 Chunk 是自包含 (Self-contained) 的语义单元。
 2. **语义元数据注入 (Semantic Metadata Enrichment):**
 - 策略: 在基础元数据 (路径、页码) 之上, 利用 LLM 提取高维语义特征。
 - 产出: 为每个 Chunk 自动生成 Title (精准小标题)、Summary (内容摘要) 和 Tags (主题标签), 并将其注入到 Metadata 字段中, 支持后续的混合检索与精确过滤。
 3. **多模态增强 (Multimodal Enrichment / Image Captioning):**
 - 策略: 扫描文档片段中的图像引用, 调用 Vision LLM (如 GPT-4o) 进行视觉理解。
 - 动作: 生成高保真的文本描述 (Caption), 描述图表逻辑或提取截图文字。
 - 存储: 将 Caption 文本“缝合”进 Chunk 的正文或 Metadata 中, 打通模态隔阂, 实现“搜文出图”。
 - **工程特性:** Transform 步骤设计为原子化与幂等操作, 支持针对特定 Chunk 的独立重试与增量更新, 避免因 LLM 调用失败导致整个文档处理中断。
- **Embedding (双路向量化)**
 - **增量计算 (Incremental Embedding / Cost Optimization):**
 - 策略: 在调用昂贵的 Embedding API 之前, 计算 Chunk 的内容哈希 (Content Hash)。仅针对数据库中不存在的新内容哈希执行向量化计算, 对于文件名变更但内容未变的片段, 直接复用已有向量, 显著降低 API 调用成本。
 - **核心策略:** 为了支持高精度的混合检索 (Hybrid Search), 系统对每个 Chunk 并行执行双路编码计算。
 - **Dense Embeddings (语义向量):** 调用 Embedding 模型 (如 OpenAI text-embedding-3 或 BGE) 生成高维浮点向量, 捕捉文本的深层语义关联, 解决“词不同意同”的检索难题。
 - **Sparse Embeddings (稀疏向量):** 利用 BM25 编码器或 SPLADE 模型生成稀疏向量 (Keyword Weights), 捕捉精确的关键词匹配信息, 解决专有名词查找问题。
 - **批处理优化:** 所有计算均采用 batch_size 驱动的批处理模式, 最大化 CPU 利用率并减少网络 RTT。
- **Upsert & Storage (索引存储)**
 - **存储后端:** 统一使用向量数据库 (如 Chroma/Qdrant) 作为存储引擎, 同时持久化存储 Dense Vector、Sparse Vector 以及 Transform 阶段生成的富 Metadata。
 - **All-in-One 存储策略:** 执行原子化存储, 每条记录同时包含:

1. **Index Data**: 用于计算相似度的 Dense Vector 和 Sparse Vector。

2. **Payload Data**: 完整的 Chunk 原始文本 (Content) 及 Metadata。

机制优势: 确保检索命中 ID 后能立即取回对应的正文内容, 无需额外的查库操作 (Lookup), 保障了 Retrieve 阶段的毫秒级响应。

- **幂等性设计 (Idempotency)**:

- 为每个 Chunk 生成全局唯一的 `chunk_id`, 生成算法采用确定的哈希组合: `hash(source_path + section_path + content_hash)`。

- 写入时采用 "Upsert" (更新或插入) 语义, 确保同一文档即使被多次处理, 数据库中也永远只有一份最新副本, 彻底避免重复索引问题。

- **原子性保证**: 以 Batch 为单位进行事务性写入, 确保索引状态的一致性。

- **文档生命周期管理 (Document Lifecycle Management)**

为支持 Dashboard 管理面板中的文档浏览与删除功能, Ingestion 层需要提供完整的文档生命周期管理能力:

- **DocumentManager (文档管理器)**: 独立于 Pipeline 的文档管理模块

- (`src/ingestion/document_manager.py`), 负责跨存储的协调操作:

- `list_documents(collection?) -> List[DocumentInfo]`: 列出已摄入文档及其统计信息 (chunk 数、图片数、摄入时间)。
 - `get_document_detail(doc_id) -> DocumentDetail`: 获取单个文档的详细信息 (所有 chunk 内容、metadata、关联图片)。
 - `delete_document(source_path, collection) -> DeleteResult`: 协调删除跨 4 个存储的关联数据:

- 1. **Chroma** — 按 `metadata.source` 删除所有 chunk 向量

- 2. **BM25 Indexer** — 移除对应文档的倒排索引条目

- 3. **ImageStorage** — 删除该文档关联的所有图片文件

- 4. **FileIntegrity** — 移除处理记录, 使文件可重新摄入

- `get_collection_stats(collection?) -> CollectionStats`: 返回集合级统计 (文档数、chunk 数、存储大小等)。

- **Pipeline 进度回调 (Progress Callback)**: 在 `IngestionPipeline.run()` 方法中新增可选

- `on_progress` 参数:

```
def run(self, source_path: str, collection: str = "default",
        on_progress: Callable[[str, int, int], None] | None = None) ->
IngestionResult:
```

- 回调签名: `on_progress(stage_name: str, current: int, total: int)`

- 各阶段 (load / split / transform / embed / upsert) 在处理每个 batch 时调用回调, Dashboard 据此展示实时进度条。

- `on_progress` 为 None 时行为与当前完全一致, 不影响 CLI 和测试场景。

- **存储层接口扩展**: 为支持 DocumentManager 的删除操作, 需扩展以下存储接口:

- **BaseVectorStore** 新增 `delete_by_metadata(filter: dict) -> int` — 按 metadata 条件批量删除

- **BM25Indexer** 新增 `remove_document(source: str) -> None` — 移除指定文档的索引条目

- `FileIntegrityChecker` 新增 `remove_record(file_hash: str) -> None` 和 `list_processed() -> List[dict]`

3.1.2 检索流水线

本模块实现核心的 RAG 检索引擎，采用“**多阶段过滤 (Multi-stage Filtering)**”架构，负责接收已消歧的独立查询 (Standalone Query)，并精准召回 Top-K 最相关片段。

- **Query Processing (查询预处理)**
 - **核心假设**：输入 Query 已由上游 (Client/MCP Host) 完成会话上下文补全 (De-referencing)，不仅如此，还进行了指代消歧。
 - **查询转换 (Transformation) 与扩张策略 (Expansion Strategy)**:
 - **Keyword Extraction**：利用 NLP 工具提取 Query 中的关键实体与动词（去停用词），生成用于稀疏检索的 Token 列表。
 - **Query Expansion** :
 - 系统可做 Synonym/Alias Expansion（同义词/别名/缩写扩展），默认策略采用“**扩展融入稀疏检索、稠密检索保持单次**”以控制成本与复杂度。
 - **Sparse Route (BM25)**：将“关键词 + 同义词/别名”合并为一个查询表达式（逻辑上按 OR 扩展），**只执行一次稀疏检索**。原始关键词可赋予更高权重以抑制语义漂移。
 - **Dense Route (Embedding)**：使用原始 query（或轻度改写后的语义 query）生成 embedding，**只执行一次稠密检索**；默认不为每个同义词单独触发额外的向量检索请求。
- **Hybrid Search Execution (双路混合检索)**
 - **并行召回 (Parallel Execution)**:
 - **Dense Route**：计算 Query Embedding -> 检索向量库 (Cosine Similarity) -> 返回 Top-N 语义候选。
 - **Sparse Route**：使用 BM25 算法 -> 检索倒排索引 -> 返回 Top-N 关键词候选。
 - **结果融合 (Fusion)**:
 - 采用 **RRF (Reciprocal Rank Fusion)** 算法，不依赖各路分数的绝对值，而是基于排名的倒数进行加权融合。
 - 公式策略： $Score = 1 / (k + Rank_Dense) + 1 / (k + Rank_Sparse)$ ，平滑因单一模式缺陷导致的漏召回。
- **Filtering & Reranking (精确过滤与重排)**
 - **Metadata Filtering Strategy (通用过滤策略)**:
 - **原则**：先解析、能前置则前置、无法前置则后置兜底。
 - Query Processing 阶段应将结构化约束解析为通用 `filters`（例如 `collection/doc_type/language/time_range/access_level` 等）。
 - 若底层索引支持且属于硬约束 (Hard Filter)，则在 Dense/Sparse 检索阶段做 Pre-filter 以缩小候选集、降低成本。
 - 无法前置的过滤（索引不支持或字段缺失/质量不稳）在 Rerank 前统一做 Post-filter 作为 safety net；对缺失字段默认采取“宽松包含”(missing->include) 以避免误杀召回。
 - 软偏好 (Soft Preference，例如“更近期更好”) 不应硬过滤，而应作为排序信号在融合/重排阶段加权。

- **Rerank Backend (可插拔精排后端):**

- **目标:** 在 Top-M 候选上进行高精度排序/过滤; 该模块必须可关闭, 并提供稳定回退策略。
- **后端选项:**
 1. **None (关闭精排):** 直接返回融合后的 Top-K (RRF 排名作为最终结果)。
 2. **Cross-Encoder Rerank (本地/托管模型):** 输入为 [Query, Chunk] 对, 输出相关性分数并排序; 适合稳定、结构化输出。CPU 环境下建议默认仅对较小的 Top-M 执行 (例如 M=10~30), 并提供超时回退。
 3. **LLM Rerank (可选):** 使用 LLM 对候选集排序/选择; 适合需要更强指令理解或无本地模型环境时。为控制成本与稳定性, 候选数应更小 (例如 M≤20), 并要求输出严格结构化格式 (如 JSON 的 ranked ids)。
- **默认与回退 (Fallback):**
 - 默认策略面向通用框架与 CPU 环境: 优先保证“可用与可控”, Cross-Encoder/LLM 均为可选增强。
 - 当精排不可用/超时/失败时, 必须回退到融合阶段的排序 (RRF Top-K), 确保系统可用性与结果稳定性。

3.2 MCP 服务设计 (MCP Service Design)

目标: 设计并实现一个符合 Model Context Protocol (MCP) 规范的 Server, 使其能够作为知识上下文提供者, 无缝对接主流 MCP Clients (如 GitHub Copilot、Claude Desktop 等), 让用户通过现有 AI 助手即可查询私有知识库。

3.2.1 核心设计理念

- **协议优先 (Protocol-First):** 严格遵循 MCP 官方规范 (JSON-RPC 2.0), 确保与任何合规 Client 的互操作性。
- **开箱即用 (Zero-Config for Clients):** Client 端无需任何特殊配置, 只需在配置文件中添加 Server 连接信息即可使用全部功能。
- **引用透明 (Citation Transparency):** 所有检索结果必须携带完整的来源信息, 支持 Client 端展示“回答依据”, 增强用户对 AI 输出的信任。
- **多模态友好 (Multimodal-Ready):** 返回格式应支持文本与图像等多种内容类型, 为未来的富媒体展示预留扩展空间。

3.2.2 传输协议: Stdio 本地通信

本项目采用 **Stdio Transport** 作为唯一通信模式。

- **工作方式:** Client (VS Code Copilot、Claude Desktop) 以子进程方式启动我们的 Server, 双方通过标准输入/输出交换 JSON-RPC 消息。
- **选型理由:**
 - **零配置:** 无需网络端口、无需鉴权, 用户只需在 Client 配置文件中指定启动命令即可使用。
 - **隐私安全:** 数据不经过网络, 天然适合处理私有知识库与敏感业务数据。
 - **契合定位:** Stdio 完美适配开发者本地工作流, 满足私有知识管理与快速原型验证需求。
- **实现约束:**
 - `stdout` 仅输出合法 MCP 消息, 禁止混入任何日志或调试信息。

- 日志统一输出至 `stderr`，避免污染通信通道。

3.2.3 SDK 与实现库选型

- **首选：Python 官方 MCP SDK (`mcp`)**
 - **优势：**
 - 官方维护，与协议规范同步更新，保证最新特性支持（如 `outputSchema`、`annotations` 等）。
 - 提供 `@server.tool()` 等装饰器，声明式定义 Tools/Resources/Prompts，代码简洁。
 - 内置 Stdio 与 HTTP Transport 支持，无需手动处理 JSON-RPC 序列化与生命周期管理。
 - **适用：**本项目的默认实现方案。
- **备选：FastAPI + 自定义协议层**
 - **场景：**需要深度定制 HTTP 行为（如自定义中间件、复杂鉴权流程）或希望学习 MCP 协议底层细节时可考虑。
 - **权衡：**开发成本更高，需自行实现能力协商 (Capability Negotiation)、错误码映射等，且需持续跟进协议版本更新。
- **协议版本：**跟踪 MCP 最新稳定版本（如 `2025-06-18`），在 `initialize` 阶段进行版本协商，确保 Client/Server 兼容性。

3.2.4 对外暴露的工具函数设计 (Tools Design)

Server 通过 `tools/list` 向 Client 注册可调用的工具函数。工具设计应遵循"单一职责、参数明确、输出丰富"原则。

- **核心工具集：**

工具名称	功能描述	典型输入参数	输出特点
<code>query_knowledge_hub</code>	主检索入口，执行混合检索 + Rerank，返回最相关片段	<code>query: string</code> , <code>top_k?: int</code> , <code>collection?: string</code>	返回带引用的结构化结果
<code>list_collections</code>	列举知识库中可用的文档集合	无	集合名称、描述、文档数量
<code>get_document_summary</code>	获取指定文档的摘要与元信息	<code>doc_id: string</code>	标题、摘要、创建时间、标签

- **扩展工具 (Agentic 演进方向)：**
 - `search_by_keyword` / `search_by_semantic`：拆分独立的检索策略，供 Agent 自主选择。
 - `verify_answer`：事实核查工具，检测生成内容是否有依据支撑。
 - `list_document_sections`：浏览文档目录结构，支持多步导航式检索。

3.2.5 返回内容与引用透明设计 (Response & Citation Design)

MCP 协议的 Tool 返回格式支持多种内容类型（`content` 数组），本项目将充分利用这一特性实现"可溯源"的回答：

- **结构化引用设计：**
 - 每个检索结果片段应包含完整的定位信息：`source_file`（文件名/路径）、`page`（页码，如适用）、`chunk_id`（片段标识）、`score`（相关性分数）。
 - 推荐在返回的 `structuredContent` 中采用统一的 Citation 格式：

```
{
  "answer": "...",
  "citations": [
    { "id": 1, "source": "xxx.pdf", "page": 5, "text": "原文片段...", "score": 0.92 },
    ...
  ]
}
```

- 同时在 `content` 数组中以 Markdown 格式呈现人类可读的带引用回答（`[1]` 标注），保证 Client 无论是否解析结构化内容都能展示引用。
- **多模态内容返回：**
 - **文本内容 (TextContent)：**默认返回类型，Markdown 格式，支持代码块、列表等富文本。
 - **图像内容 (ImageContent)：**当检索结果关联图像时，Server 读取本地图片文件并编码为 Base64 返回。
 - **格式：** { "type": "image", "data": "<base64>", "mimeType": "image/png" }
 - **工作流程：**数据摄取阶段存储图片本地路径 → 检索命中后 Server 动态读取 → 编码为 Base64 → 嵌入返回消息。
 - **Client 兼容性：**图像展示能力取决于 Client 实现，GitHub Copilot 可能降级处理，Claude Desktop 支持完整渲染。Server 端统一返回 Base64 格式，由 Client 决定如何渲染。
- **Client 适配策略：**
 - **GitHub Copilot (VS Code)：**当前对 MCP 的支持集中在 Tools 调用，返回的 `content` 中的文本会展示给用户。建议以清晰的 Markdown 文本（含引用标注）为主，图像作为补充。
 - **Claude Desktop：**对 MCP Tools/Resources 有完整支持，图像与资源链接可直接渲染。可更激进地使用多模态返回。
 - **通用兼容原则：**始终在 `content` 数组第一项提供纯文本/Markdown 版本的答案，确保最低兼容性；将结构化数据、图像等放在后续项或 `structuredContent` 中，供高级 Client 解析。

3.3 可插拔架构设计 (Pluggable Architecture Design)

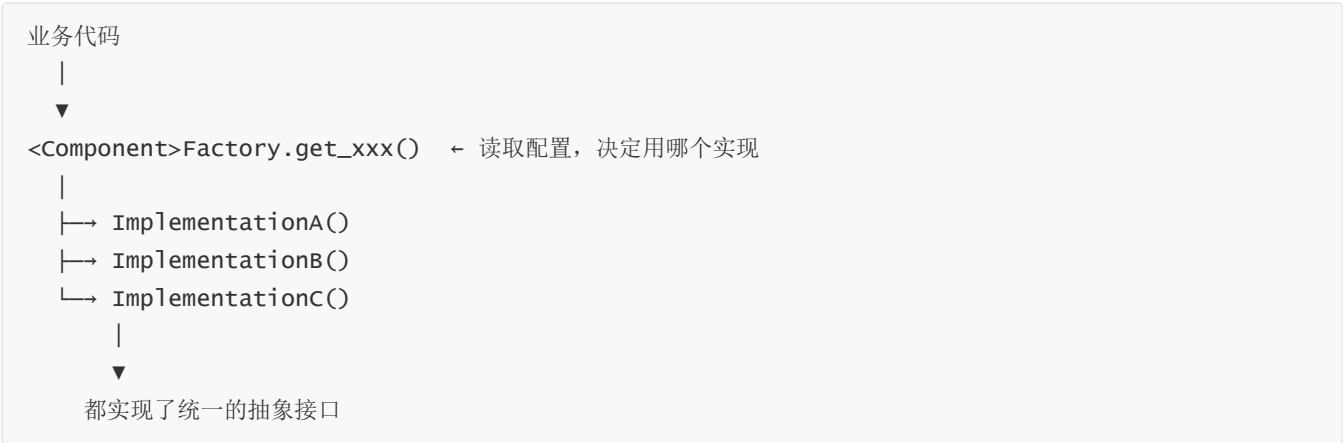
目标： 定义清晰的抽象层与接口契约，使 RAG 链路的每个核心组件都能够独立替换与升级，避免技术锁定，支持低成本的 A/B 测试与环境迁移。

术语说明： 本节中的"提供者 (Provider)"、"实现 (Implementation)"指的是完成某项功能的**具体技术方案**，而非传统 Web 架构中的"后端服务器"。例如，LLM 提供者可以是远程的 Azure OpenAI API，也可以是本地运行的 Ollama；向量存储可以是本地嵌入式的 Chroma，也可以是云端托管的 Pinecone。本项目作为本地 MCP Server，通过统一接口对接这些不同的提供者，实现灵活切换。

3.3.1 设计原则

- **接口隔离 (Interface Segregation):** 为每类组件定义最小化的抽象接口，上层业务逻辑仅依赖接口而非具体实现。
- **配置驱动 (Configuration-Driven):** 通过统一配置文件（如 `settings.yaml`）指定各组件的具体后端，代码无需修改即可切换实现。
- **工厂模式 (Factory Pattern):** 使用工厂函数根据配置动态实例化对应的实现类，实现"一处配置，处处生效"。
- **优雅降级 (Graceful Fallback):** 当首选后端不可用时，系统应自动回退到备选方案或安全默认值，保障可用性。

通用结构示意（适用于 3.3.2 / 3.3.3 / 3.3.4 等可插拔组件）：



3.3.2 LLM 与 Embedding 提供者抽象

这是可插拔设计的核心环节，因为模型提供者的选择直接影响成本、性能与隐私合规。

- **统一接口层 (Unified API Abstraction):**
 - **设计思路:** 无论底层使用 Azure OpenAI、OpenAI 原生 API、DeepSeek 还是本地 Ollama，上层调用代码应保持一致。
 - **关键抽象:**
 - `LLMClient`: 暴露 `chat(messages) -> response` 方法，屏蔽不同 Provider 的认证方式与请求格式差异。
 - `EmbeddingClient`: 暴露 `embed(texts) -> vectors` 方法，统一处理批量请求与维度归一化。
- **提供者选项与切换场景:**

提供者类型	典型场景	配置切换点
Azure OpenAI	企业合规、私有云部署、区域数据驻留	<code>provider: azure, endpoint, api_key, deployment_name</code>
OpenAI 原生	通用开发、最新模型尝鲜	<code>provider: openai, api_key, model</code>
DeepSeek / 其他云端	成本优化、特定语言优化	<code>provider: deepseek, api_key, model</code>

提供者类型	典型场景	配置切换点
Ollama / vLLM (本地)	完全离线、隐私敏感、无 API 成本	<code>provider: ollama, base_url, model</code>

- **技术选型建议：**
 - 本项目采用自研的 `BaseLLM` / `BaseEmbedding` 抽象基类，配合工厂模式 (`llm_factory.py` / `embedding_factory.py`) 实现统一调用接口。已内置 Azure OpenAI、OpenAI、Ollama、DeepSeek 四种 Provider 适配。
 - 对于其他 Provider，可通过 **OpenAI-Compatible 模式** 接入（设置自定义 `api_base`），或实现 `BaseLLM` 接口并在工厂中注册。
 - 对于企业级需求，可在其基础上增加统一的 **重试、限流、日志** 中间层，提升生产可靠性，但本项目暂不实现，这里仅提供思路。
 - **Vision LLM 扩展：**针对图像描述生成 (Image Captioning) 需求，系统扩展了 `BaseVisionLLM` 接口，支持文本+图片的多模态输入。当前实现：
 - **Azure OpenAI Vision** (GPT-4o/GPT-4-Vision)：企业级合规部署，支持复杂图表解析，与 Azure 生态深度集成。

3.3.3 检索策略抽象

检索层的可插拔性决定了系统在不同数据规模与查询模式下的适应能力。

设计模式：抽象工厂模式

与 3.3.2 节的 LLM 抽象类似，检索层各组件的可插拔性同样依赖两层设计：

1. **自研的统一抽象接口：**本项目为向量数据库 (`BaseVectorStore`)、Embedding (`BaseEmbedding`)、分块 (`BaseSplitter`) 等核心组件定义了统一的抽象基类，不同实现只需遵循相同接口即可无缝替换。
2. **工厂函数路由：**每个抽象层配套工厂函数（如 `embedding_factory.py`、`splitter_factory.py`），根据 `settings.yaml` 中的配置字段自动实例化对应实现，实现"改配置不改代码"的切换体验。

通用的“配置驱动 + 工厂路由”结构示意见 3.3.1 节。

下面分别说明各组件如何应用这一模式：

1. 分块策略 (Chunking Strategy)

分块是 Ingestion Pipeline 的核心环节之一，决定了文档如何被切分为适合检索的语义单元。本项目的 Splitter 层采用可插拔设计 (`BaseSplitter` 抽象接口 + `SplitterFactory` 工厂)，不同分块实现只需遵循相同接口即可无缝替换。

常见的分块策略包括：

- **固定长度切分：**按字符数或 Token 数切分，简单但可能破坏语义完整性。
- **递归字符切分：**按层级分隔符（段落→句子→字符）递归切分，在长度限制内尽量保持语义边界。
- **语义切分：**利用 Embedding 相似度检测语义断点，确保每个 Chunk 是自包含的语义单元。
- **结构感知切分：**根据文档结构（Markdown 标题、代码块、列表等）进行切分。

本项目当前采用 **LangChain** 的 `RecursiveCharacterTextSplitter` 进行切分，该方法对 Markdown 文档的结构（标题、段落、列表、代码块）有天然的适配性，能够通过配置语义断点（Separators）实现高质量、语义完整的切块。

当前实现说明：目前系统使用 LangChain `RecursiveCharacterTextSplitter`。架构设计上预留了切换能力，如需切换为 `SentenceSplitter`、`SemanticSplitter` 或自定义切分器，只需实现 `BaseSplitter` 接口并在配置中指定即可。

2. 向量数据库 (Vector Store)

本项目自定义了统一的 `BaseVectorStore` 抽象接口，暴露 `.add()`、`.query()`、`.delete()` 等方法。所有向量数据库后端（Chroma、Qdrant、Pinecone 等）只需实现该接口即可插拔替换，通过 `VectorStoreFactory` 根据配置自动选择具体实现。

本项目选用 **Chroma** 作为向量数据库。相比 Qdrant、Milvus、Weaviate 等需要 Docker 容器或分布式架构支撑的方案，Chroma 采用嵌入式设计，`pip install chromadb` 即可使用，无需额外部署数据库服务，非常适合本地开发与快速原型验证。同时 `ChromaStore` 适配器（`src/libs/vector_store/chroma_store.py`），与 Pipeline 无缝集成。

当前实现说明：目前系统仅实现了 Chroma 后端。虽然架构设计上预留了工厂模式以支持未来扩展，但当前版本尚未实现其他向量数据库的适配器。

3. 向量编码策略 (Embedding Strategy)

向量编码是 Ingestion Pipeline 的关键环节，决定了 Chunk 如何被转换为可检索的向量表示。本项目自定义了 `BaseEmbedding` 抽象接口（`src/libs/embedding/base.py`），支持不同 Embedding 模型的可插拔替换。

常见的编码策略包括：

- **纯稠密编码 (Dense Only)**：仅生成语义向量，适合通用场景。
- **纯稀疏编码 (Sparse Only)**：仅生成关键词权重向量，适合精确匹配场景。
- **双路编码 (Dense + Sparse)**：同时生成稠密向量和稀疏向量，为混合检索提供数据基础。

本项目当前采用 **双路编码 (Dense + Sparse)** 策略：

- **Dense Embeddings (语义向量)**：调用 Embedding 模型（如 OpenAI text-embedding-3）生成高维浮点向量，捕捉文本的深层语义关联。
- **Sparse Embeddings (稀疏向量)**：利用 BM25 编码器生成稀疏向量（Keyword Weights），捕捉精确的关键词匹配信息。

存储时，Dense Vector 和 Sparse Vector 与 Chunk 原文、Metadata 一起原子化写入向量数据库，确保检索时可同时利用两种向量。

当前实现说明：目前系统实现了 Dense + Sparse 双路编码。架构设计上预留了切换能力，如需使用其他 Embedding 模型（如 BGE、Ollama 本地模型）或调整编码策略，可在 Pipeline 中替换相应组件。

4. 召回策略 (Retrieval Strategy)

召回策略决定了查询阶段如何从知识库中检索相关内容。基于 Ingestion 阶段存储的向量类型，可采用不同的召回方案：

- **纯稠密召回 (Dense Only)**：仅使用语义向量进行相似度匹配。

- **纯稀疏召回 (Sparse Only)**：仅使用 BM25 进行关键词匹配。
- **混合召回 (Hybrid)**：并行执行稠密和稀疏两路召回，再通过融合算法合并结果。
- **混合召回 + 精排 (Hybrid + Rerank)**：在混合召回基础上，增加精排步骤进一步提升相关性。

本项目当前采用 **混合召回 + 精排 (Hybrid + Rerank)** 策略：

- **稠密召回 (Dense Route)**：计算 Query Embedding，在向量库中进行 Cosine Similarity 检索，返回 Top-N 语义候选。
- **稀疏召回 (Sparse Route)**：使用 BM25 算法检索倒排索引，返回 Top-N 关键词候选。
- **融合 (Fusion)**：使用 RRF (Reciprocal Rank Fusion) 算法将两路结果合并排序。
- **精排 (Rerank)**：对融合后的候选集进行重排序，支持 None / Cross-Encoder / LLM Rerank 三种模式。

当前实现说明：目前系统实现了 Hybrid + Rerank 策略。架构设计上预留了策略切换能力，如需使用纯稠密或纯稀疏召回，可通过配置切换；融合算法和 Reranker 同样支持替换。

3.3.4 评估框架抽象

评估体系的可插拔性确保团队可以根据业务目标灵活选择或组合不同的质量度量维度。

- **设计思路：**
 - 定义统一的 `Evaluator` 接口，暴露 `evaluate(query, retrieved_chunks, generated_answer, ground_truth) -> metrics` 方法。
 - 各评估框架实现该接口，输出标准化的指标字典。
- **可选评估框架：**

框架	特点	适用场景
Ragas	RAG 专用、指标丰富 (Faithfulness, Answer Relevancy, Context Precision 等)	全面评估 RAG 质量、学术对比
DeepEval	LLM-as-Judge 模式、支持自定义评估标准	需要主观质量判断、复杂业务规则
自定义指标	Hit Rate, MRR, Latency P99 等基础工程指标	快速回归测试、上线前 Sanity Check

- **组合与扩展：**
 - 评估模块设计为**组合模式**，可同时挂载多个 Evaluator，生成综合报告。
 - 配置示例：`evaluation.backends: [ragas, custom_metrics]`，系统并行执行并汇总结果。

3.3.5 配置管理与切换流程

- **配置文件结构示例 (config/settings.yaml)：**

```
llm:
  provider: azure # azure | openai | ollama | deepseek
  model: gpt-4o
  # provider-specific configs...
```

```

embedding:
  provider: openai
  model: text-embedding-3-small

vector_store:
  backend: chroma # chroma | qdrant | pinecone

retrieval:
  sparse_backend: bm25 # bm25 | elasticsearch
  fusion_algorithm: rrf # rrf | weighted_sum
  rerank_backend: cross_encoder # none | cross_encoder | llm

evaluation:
  backends: [ragas, custom_metrics]

dashboard:
  enabled: true
  port: 8501
  traces_dir: ./logs

```

- **切换流程:**

1. 修改 `settings.yaml` 中对应组件的 `backend` / `provider` 字段。
2. 确保新后端的依赖已安装、凭据已配置。
3. 重启服务，工厂函数自动加载新实现，无需修改业务代码。

3.4 可观测性与可视化管理平台设计 (Observability & Visual Management Platform Design)

目标: 针对 RAG 系统常见的"黑盒"问题，设计全链路可观测的追踪体系与完整的可视化管理平台。覆盖 **Ingestion (摄取链路)** 与 **Query (查询链路)** 两条完整流水线的追踪记录，同时提供数据浏览、文档管理、组件概览等管理功能，使整个系统**透明可见、可管理且可量化**。

3.4.1 设计理念

- **双链路全覆盖追踪 (Dual-Pipeline Tracing):**
 - **Ingestion Trace:** 以 `trace_id` 为核心，记录一次摄取从文件加载到存储完成的全过程 (load → split → transform → embed → upsert)，包含各阶段耗时、处理的 chunk 数量、跳过/失败详情。
 - **Query Trace:** 以 `trace_id` 为核心，记录一次查询从 Query 输入到 Response 输出的全过程 (query_processing → dense → sparse → fusion → rerank)，包含各阶段候选数量、分数分布与耗时。
- **透明可回溯 (Transparent & Traceable):** 每个阶段的中间状态都被记录，开发者可以清晰看到"系统为什么召回了这些文档"、"Rerank 前后排名如何变化"，从而精准定位问题。
- **低侵入性 (Low Intrusiveness):** 追踪逻辑与业务逻辑解耦，通过 `TraceContext` 显式调用模式注入，避免污染核心代码。
- **轻量本地化 (Lightweight & Local):** 采用结构化日志 + 本地 Dashboard 的方案，零外部依赖，开箱即用。
- **动态组件感知 (Dynamic Component Awareness):** Dashboard 基于 Trace 中的 `method` / `provider` / `details` 字段动态渲染，更换可插拔组件后自动适配展示内容，无需修改 Dashboard 代码。

3.4.2 追踪数据结构

系统定义两类 Trace 记录，分别覆盖查询与摄取两条链路：

A. Query Trace（查询追踪）

每次查询请求生成唯一的 `trace_id`，记录从 Query 输入到 Response 输出的全过程：

基础信息：

- `trace_id`：请求唯一标识
- `trace_type`："query"
- `timestamp`：请求时间戳
- `user_query`：用户原始查询
- `collection`：检索的知识库集合

各阶段详情 (Stages)：

阶段	记录内容
Query Processing	原始 Query、改写后 Query（若有）、提取的关键词、method、耗时
Dense Retrieval	返回的 Top-N 候选及相似度分数、provider、耗时
Sparse Retrieval	返回的 Top-N 候选及 BM25 分数、method、耗时
Fusion	融合后的统一排名、algorithm、耗时
Rerank	重排后的最终排名及分数、backend、是否触发 Fallback、耗时

汇总指标：

- `total_latency`：端到端总耗时
- `top_k_results`：最终返回的 Top-K 文档 ID
- `error`：异常信息（若有）

评估指标 (Evaluation Metrics)：

- `context_relevance`：召回文档与 Query 的相关性分数
- `answer_faithfulness`：生成答案与召回文档的一致性分数（若有生成环节）

B. Ingestion Trace（摄取追踪）

每次文档摄取生成唯一的 `trace_id`，记录从文件加载到存储完成的全过程：

基础信息：

- `trace_id`：摄取唯一标识
- `trace_type`："ingestion"
- `timestamp`：摄取开始时间
- `source_path`：源文件路径
- `collection`：目标集合名称

各阶段详情 (Stages):

阶段	记录内容
Load	文件大小、解析器 (method: markdown)、提取的图片数、耗时
Split	splitter 类型 (method)、产出 chunk 数、平均 chunk 长度、耗时
Transform	各 transform 名称与处理详情 (refined/enriched/captioned 数量)、LLM provider、耗时
Embed	embedding provider、batch 数、向量维度、dense + sparse 编码耗时
Upsert	存储后端 (method: chroma)、upsert 数量、BM25 索引更新、图片存储、耗时

汇总指标:

- `total_latency`: 端到端总耗时
- `total_chunks`: 最终存储的 chunk 数量
- `total_images`: 处理的图片数量
- `skipped`: 跳过的文件/chunk 数 (已存在、未变更等)
- `error`: 异常信息 (若有)

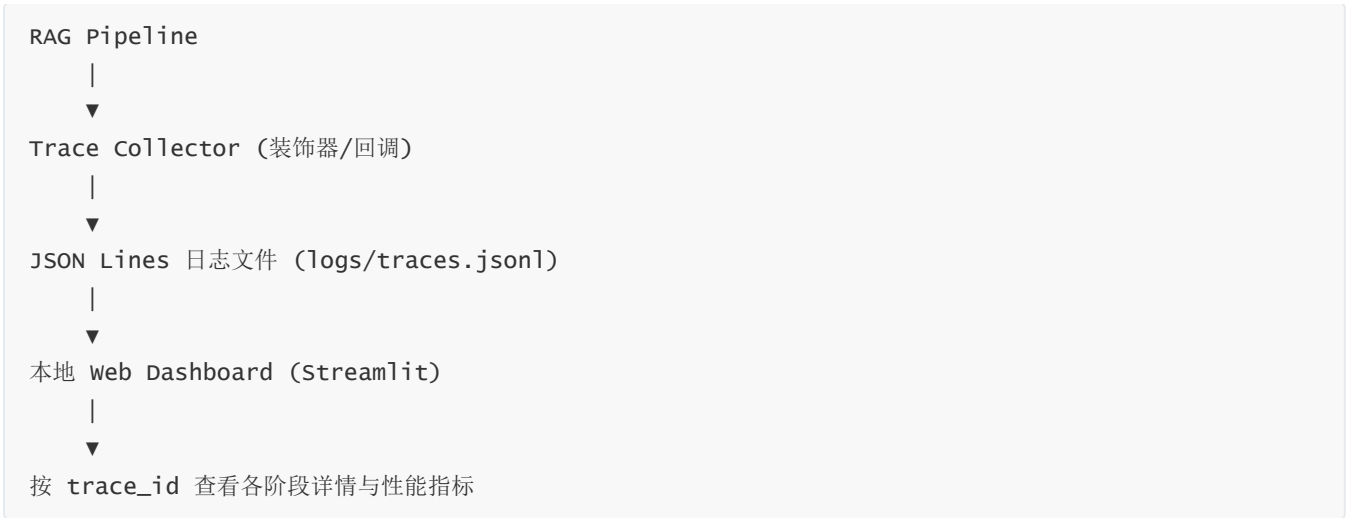
3.4.3 技术方案: 结构化日志 + 本地 Web Dashboard

本项目采用 "结构化日志 + 本地 Web Dashboard" 作为可观测性的实现方案。

选型理由:

- 零外部依赖**: 不依赖 LangSmith、LangFuse 等第三方平台, 无需网络连接与账号注册, 完全本地化运行。
- 轻量易部署**: 仅需 Python 标准库 + 一个轻量 Web 框架 (如 Streamlit), `pip install` 即可使用, 无需 Docker 或数据库服务。
- 学习成本低**: 结构化日志是通用技能, 调试时可直接用 `jq`、`grep` 等命令行工具查询; Dashboard 代码简单直观, 便于理解与二次开发。
- 契合项目定位**: 本项目面向本地 MCP Server 场景, 单用户、单机运行, 无需分布式追踪或多租户隔离等企业级能力。

实现架构:



核心组件：

- **结构化日志层：**基于 Python `logging` + JSON Formatter，将每次请求的 Trace 数据以 JSON Lines 格式追加写入本地文件。每行一条完整的请求记录，包含 `trace_id`、各阶段详情与耗时。
- **本地 Web Dashboard：**基于 Streamlit 构建的轻量级 Web UI，读取日志文件并提供交互式可视化。核心功能是按 `trace_id` 检索并展示单次请求的完整追踪链路。

3.4.4 追踪机制实现

为确保各 RAG 阶段（可替换、可自定义）都能输出统一格式的追踪日志，系统采用 **TraceContext（追踪上下文）** 作为核心机制。

工作原理：

1. **请求开始：**Pipeline 入口创建一个 `TraceContext` 实例，生成唯一 `trace_id`，记录请求基础信息（Query、Collection 等）。
2. **阶段记录：**`TraceContext` 提供 `record_stage()` 方法，各阶段执行完毕后调用该方法，传入阶段名称、耗时、输入输出等数据。
3. **请求结束：**调用 `trace.finish()`，`TraceContext` 将收集的完整数据序列化为 JSON，追加写入日志文件。

与可插拔组件的配合：

- 各阶段组件（Retriever、Reranker 等）的接口约定中包含 `TraceContext` 参数。
- 组件实现者在执行核心逻辑后，调用 `trace.record_stage()` 记录本阶段的关键信息。
- 这是**显式调用**模式：不强制、不会因未调用而报错，但依赖开发者主动记录。好处是代码透明，开发者清楚知道哪些数据被记录；代价是需要开发者自觉遵守约定。

阶段划分原则：

- **Stage 是固定的通用大类：**`retrieval`（检索）、`rerank`（重排）、`generation`（生成）等，不随具体实现方案变化。
- **具体实现是阶段内部的细节：**在 `record_stage()` 中通过 `method` 字段记录采用的具体方法（如 `bm25`、`hybrid`），通过 `details` 字段记录方法相关的细节数据。
- 这样无论底层方案怎么替换，阶段结构保持稳定，Dashboard 展示逻辑无需调整。

3.4.5 Dashboard 功能设计（六页面架构）

Dashboard 基于 Streamlit 构建多页面应用（`st.navigation`），提供六大功能页面：

页面 1：系统总览 (Overview)

- **组件配置卡片**：读取 `Settings`，展示当前可插拔组件的配置状态：
 - LLM：provider + model（如 `azure / gpt-4o`）
 - Embedding：provider + model + 维度
 - Splitter：类型 + chunk_size + overlap
 - Reranker：backend + model（或 None）
 - Evaluator：已启用的 backends 列表
- **数据资产统计**：调用 `DocumentManager.get_collection_stats()` 展示各集合的文档数、chunk 数、图片数。
- **系统健康指标**：最近一次 Ingestion/Query trace 的时间与耗时。

页面 2：数据浏览器 (Data Browser)

- **文档列表视图**：展示已摄入的文档（source_path、集合、chunk 数、摄入时间），支持按集合筛选与关键词搜索。
- **Chunk 详情视图**：点击文档展开其所有 chunk，每个 chunk 显示：
 - 原文内容（可折叠长文本）
 - Metadata 各字段（title、summary、tags、page、image_refs 等）
 - 关联图片预览（从 ImageStorage 读取并展示缩略图）
- **数据来源**：通过 `ChromaStore.get_all()` 或 `get_by_metadata()` 读取 chunk 数据。

页面 3：Ingestion 管理 (Ingestion Manager)

- **文件选择与摄取触发**：
 - 文件上传组件（`st.file_uploader`）或目录路径输入
 - 选择目标集合（下拉选择或新建）
 - 点击"开始摄取"按钮触发 `IngestionPipeline.run()`
 - 利用 `on_progress` 回调驱动 Streamlit 进度条（`st.progress`），实时显示当前阶段与处理进度
- **文档删除**：
 - 在文档列表中提供"删除"按钮
 - 调用 `DocumentManager.delete_document()` 协调跨存储删除
 - 删除完成后刷新列表
- **注意**：Pipeline 执行为同步阻塞操作，Streamlit 的 rerun 机制天然支持（进度条在同一 request 中更新）。

页面 4：Ingestion 追踪 (Ingestion Traces)

- **摄取历史列表**：按时间倒序展示 `trace_type == "ingestion"` 的历史记录，显示文件名、集合、总耗时、状态（成功/失败）。
- **单次摄取详情**：

- **阶段耗时瀑布图**: 横向条形图展示 load/split/transform/embed/upsert 各阶段时间分布。
- **处理统计**: chunk 数、图片数、跳过数、失败数。
- **各阶段详情展开**: 点击查看 method/provider、输入输出样本。

页面 5: Query 追踪 (Query Traces)

- **查询历史列表**: 按时间倒序展示 `trace_type == "query"` 的历史记录, 支持按 Query 关键词筛选。
- **单次查询详情**:
 - **耗时瀑布图**: 展示 query_processing/dense/sparse/fusion/rerank 各阶段时间分布。
 - **Dense vs Sparse 对比**: 并列展示两路召回结果的 Top-N 文档 ID 与分数。
 - **Rerank 前后对比**: 展示融合排名与精排后排名的变化 (排名跃升/下降标记)。
 - **最终结果表**: 展示 Top-K 候选文档的标题、分数、来源。

页面 6: 评估面板 (Evaluation Panel)

- **评估运行**: 选择评估后端 (Ragas / Custom / All) 与 golden test set, 点击运行。
- **指标展示**: 以表格和图表展示 hit_rate、mrr、faithfulness 等指标。
- **历史趋势**: 对比不同时间的评估结果, 观察策略调整的效果。
- **注意**: 评估面板在 Phase H 实现, Phase G 完成后该页面显示"评估模块尚未启用"的占位提示。

Dashboard 技术架构:

```
src/observability/dashboard/
├─ app.py                # Streamlit 入口, 页面导航注册
├─ pages/
│  ├─ overview.py        # 页面 1: 系统总览
│  ├─ data_browser.py     # 页面 2: 数据浏览器
│  ├─ ingestion_manager.py # 页面 3: Ingestion 管理
│  ├─ ingestion_traces.py # 页面 4: Ingestion 追踪
│  ├─ query_traces.py     # 页面 5: Query 追踪
│  └─ evaluation_panel.py # 页面 6: 评估面板
└─ services/
   ├─ trace_service.py    # Trace 数据读取服务 (解析 traces.jsonl)
   ├─ data_service.py     # 数据浏览服务 (封装 ChromaStore/ImageStorage 读取)
   └─ config_service.py   # 配置读取服务 (封装 Settings 读取与展示)
```

Dashboard 与 Trace 的数据关系:

- Dashboard 页面 4/5 读取 `logs/traces.jsonl` (通过 `TraceService`) , 按 `trace_type` 分类展示。
- Dashboard 页面 1/2/3 直接读取存储层 (通过 `DataService` 封装 `ChromaStore/ImageStorage/FileIntegrity`) , 不依赖 Trace。
- 所有页面基于 Trace 中 `method/provider` 字段动态渲染标签, 更换组件后自动适配。

3.4.6 配置示例

```
observability:
  enabled: true

# 日志配置
logging:
  log_file: logs/traces.jsonl # JSON Lines 格式日志文件
  log_level: INFO # DEBUG | INFO | WARNING

# 追踪粒度控制
detail_level: standard # minimal | standard | verbose

# Dashboard 管理平台配置
dashboard:
  enabled: true
  port: 8501 # Streamlit 服务端口
  traces_dir: ./logs # Trace 日志文件目录
  auto_refresh: true # 是否自动刷新（轮询新 trace）
  refresh_interval: 5 # 自动刷新闻隔（秒）
```

3.5 多模态图片处理设计 (Multimodal Image Processing Design)

目标：设计一套完整的图片处理方案，使 RAG 系统能够理解、索引并检索文档中的图片内容，实现"用自然语言搜索图片"的能力，同时保持架构的简洁性与可扩展性。

3.5.1 设计理念与策略选型

多模态 RAG 的核心挑战在于：**如何让纯文本的检索系统"看懂"图片**。业界主要有两种技术路线：

策略	核心思路	优势	劣势
Image-to-Text (图转文)	利用 Vision LLM 将图片转化为文本描述，复用纯文本 RAG 链路	架构统一、实现简单、成本可控	描述质量依赖 LLM 能力，可能丢失视觉细节
Multi-Embedding (多模态向量)	使用 CLIP 等模型将图文统一映射到同一向量空间	保留原始视觉特征，支持图搜图	需引入额外向量库，架构复杂度高

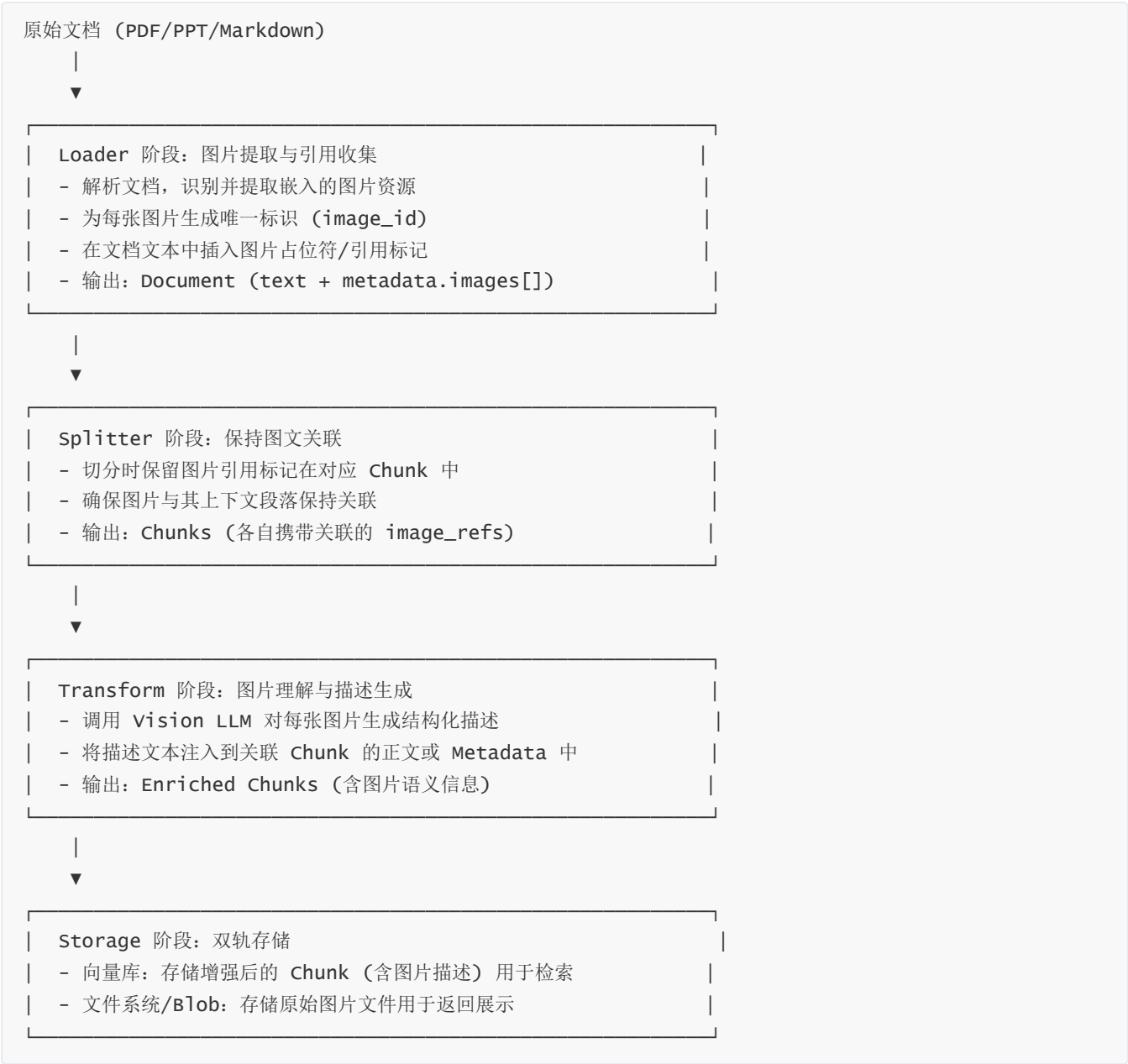
本项目选型：Image-to-Text（图转文）策略

选型理由：

- **架构统一：**无需引入 CLIP 等多模态 Embedding 模型，无需维护独立的图像向量库，完全复用现有的文本 RAG 链路（Ingestion → Hybrid Search → Rerank）。
- **语义对齐：**通过 LLM 将图片的视觉信息转化为自然语言描述，天然与用户的文本查询在同一语义空间，检索效果可预期。
- **成本可控：**仅在数据摄取阶段一次性调用 Vision LLM，检索阶段无额外成本。
- **渐进增强：**未来如需支持"图搜图"等高级能力，可在此基础上叠加 CLIP Embedding，无需重构核心链路。

3.5.2 图片处理全流程设计

图片处理贯穿 Ingestion Pipeline 的多个阶段，整体流程如下：



3.5.3 各阶段技术要点

1. Loader 阶段：图片提取与引用收集

- **提取策略：**
 - 解析文档时识别嵌入的图片资源（PDF 中的 `XObject`、PPT 中的媒体文件、Markdown 中的 `![]()` 引用）。
 - 为每张图片生成全局唯一的 `image_id`（建议格式：`{doc_hash}_{page}_{seq}`）。
 - 将图片二进制数据提取并暂存，记录其在原文档中的位置信息。
- **引用标记：**
 - 在转换后的 Markdown 文本中，于图片原始位置插入占位符（如 `[IMAGE: {image_id}]`）。
 - 在 Document 的 Metadata 中维护 `images` 列表，记录每张图片的 `image_id`、原始路径、页码、尺

- 寸等基础信息。
- **存储原始图片：**
 - 将提取的图片保存至本地文件系统的约定目录（如 `data/images/{collection}/{image_id}.png`）。
 - 仅保存需要的图片格式（推荐统一转换为 PNG/JPEG），控制存储体积。

2. Splitter 阶段：保持图文关联

- **关联保持原则：**
 - 图片引用标记应与其说明性文字（Caption、前后段落）尽量保持在同一 Chunk 中。
 - 若图片出现在章节开头或结尾，切分时应将其归入语义上最相关的 Chunk。
- **Chunk Metadata 扩展：**
 - 每个 Chunk 的 Metadata 中增加 `image_refs: List[image_id]` 字段，记录该 Chunk 关联的图片列表。
 - 此字段用于后续 Transform 阶段定位需要处理的图片，以及检索命中后定位需要返回的图片。

3. Transform 阶段：图片理解与描述生成

这是多模态处理的核心环节，负责将视觉信息转化为可检索的文本语义。

- **Vision LLM 选型：**

模型	提供商	特点	适用场景	推荐指数
GPT-4o	OpenAI / Azure	理解能力强，支持复杂图表解读，英文文档表现优异	高质量需求、复杂业务文档、国际化场景	★★★★★
Qwen-VL-Max	阿里云 (DashScope)	中文理解能力出色，性价比高，对中文图表/文档支持好	中文文档、国内部署、成本敏感场景	★★★★★
Qwen-VL-Plus	阿里云 (DashScope)	速度更快，成本更低，适合大批量处理	大批量中文文档、快速迭代场景	★★★★
Claude 3.5 Sonnet	Anthropic	多模态原生支持，长上下文	需要结合大段文字理解图片	★★★★
Gemini Pro Vision	Google	成本较低，速度较快	大批量处理、成本敏感场景	★★★
GLM-4V	智谱 AI (ZhipuAI)	国内老牌，稳定性好，中文支持佳	国内部署备选、企业级应用	★★★★

双模型选型策略（推荐）：

本项目采用**国内 + 国外双模型**方案，通过配置切换，兼顾不同部署环境和文档类型：

部署环境	主选模型	备选模型	说明
国际化 / Azure 环境	GPT-4o (Azure)	Qwen-VL-Max	英文文档优先用 GPT-4o，中文文档可切换 Qwen-VL
国内部署 / 纯中文场景	Qwen-VL-Max	GPT-4o	中文图表理解用 Qwen-VL，特殊需求可切换 GPT-4o
成本敏感 / 大批量	Qwen-VL-Plus	Gemini Pro Vision	牺牲部分质量换取速度和成本

选型理由：

1. GPT-4o (国外首选)：

- 视觉理解能力业界领先，复杂图表解读准确率高
- Azure 部署可满足企业合规要求
- 英文技术文档理解效果最佳

2. Qwen-VL-Max (国内首选)：

- 中文场景下表现与 GPT-4o 接近，部分中文图表任务甚至更优
- 通过阿里云 DashScope API 调用，国内访问稳定、延迟低
- 价格约为 GPT-4o 的 1/3 ~ 1/5，性价比极高
- 原生支持中文 OCR，对中文截图、表格识别更准确

• 描述生成策略：

- 结构化 Prompt**：设计专用的图片理解 Prompt，引导 LLM 输出结构化描述，而非自由发挥。
- 上下文感知**：将图片的前后文本段落一并传入 Vision LLM，帮助其理解图片在文档中的语境与作用。
- 分类型处理**：针对不同类型的图片采用差异化的理解策略：

图片类型	理解重点	Prompt 引导方向
流程图/架构图	节点、连接关系、流程逻辑	"描述这张图的结构和流程步骤"
数据图表	数据趋势、关键数值、对比关系	"提取图表中的关键数据和结论"
截图/UI	界面元素、操作指引、状态信息	"描述截图中的界面内容和关键信息"
照片/插图	主体对象、场景、视觉特征	"描述图片中的主要内容"

• 描述注入方式：

- 推荐：注入正文**：将生成的描述直接替换或追加到 Chunk 正文中的图片占位符位置，格式如 `[图片描述: {caption}]`。这样描述会被 Embedding 覆盖，可被直接检索。
- 备选：注入 Metadata**：将描述存入 `chunk.metadata.image_captions` 字段。需确保检索时该字段也被索引。

• 幂等与增量处理：

- 为每张图片的描述计算内容哈希，存入 `processing_cache` 表。
- 重复处理时，若图片内容未变且 Prompt 版本一致，直接复用缓存的描述，避免重复调用 Vision LLM。

4. Storage 阶段：双轨存储

- 向量库存储（用于检索）：
 - 存储增强后的 Chunk，其正文已包含图片描述，Metadata 包含 image_refs 列表。
 - 检索时通过文本相似度即可命中包含相关图片描述的 Chunk。
- 原始图片存储（用于返回）：
 - 图片文件存储于本地文件系统，路径记录在独立的 images 索引表中。
 - 索引表字段：image_id, file_path, source_doc, page, width, height, mime_type。
 - 检索命中后，根据 Chunk 的 image_refs 查询索引表，获取图片文件路径用于返回。

3.5.4 检索与返回流程

当用户查询命中包含图片的 Chunk 时，系统需要将图片与文本一并返回：



MCP 响应格式：

```
{
  "content": [
    {
      "type": "text",
      "text": "根据文档，系统架构如下: ...\\n\\n[1] 来源: architecture.pdf，第5页"
    },
    {
      "type": "image",
      "data": "<base64-encoded-image>",
      "mimeType": "image/png"
    }
  ]
}
```

3.5.5 质量保障与边界处理

- 描述质量检测：
 - 对生成的描述进行基础质量检查（长度、是否包含关键信息）。
 - 若描述过短或 LLM 返回"无法识别"，标记该图片为 `low_quality`，可选择人工复核或跳过索引。
- 大尺寸/特殊图片处理：
 - 超大图片在传入 Vision LLM 前进行压缩（保持宽高比，限制最大边长）。
 - 对于纯装饰性图片（如分隔线、背景图），可通过尺寸或位置规则过滤，不进入描述生成流程。
- 批量处理优化：
 - 图片描述生成支持批量异步调用，提高吞吐量。
 - 单个文档处理失败时，记录失败的图片 ID，不影响其他图片的处理进度。
- 降级策略：
 - 当 Vision LLM 不可用时，系统回退到"仅保留图片占位符"模式，图片不参与检索但不阻塞 Ingestion 流程。
 - 在 Chunk 中标记 `has_unprocessed_images: true`，后续可增量补充描述。

4. 测试方案

4.1 设计理念：测试驱动开发 (TDD)

本项目采用**测试驱动开发 (Test-Driven Development)** 作为核心开发范式，确保每个组件在实现前就已明确其预期行为，通过自动化测试持续验证系统质量。

核心原则：

- **早测试、常测试**：每个功能模块实现的同时就编写对应的单元测试，而非事后补测。
- **测试即文档**：测试用例本身就是最准确的行为规范，新加入的开发者可通过阅读测试快速理解各模块功能。
- **快速反馈循环**：单元测试应在秒级完成，支持开发者高频执行，立即发现引入的问题。
- **分层测试金字塔**：大量快速的单元测试作为基座，少量关键路径的集成测试作为保障，极少数端到端测试验证完整流程。



4.2 测试分层策略

4.2.1 单元测试 (Unit Tests)

目标：验证每个独立组件的内部逻辑正确性，隔离外部依赖。

覆盖范围：

模块	测试重点	典型测试用例
Loader (文档解析器)	格式解析、元数据提取、图片引用收集	- 测试解析单页/多页 PDF - 验证 Markdown 标题层级提取 - 检查图片占位符插入位置
Splitter (切分器)	切分边界、上下文保留、元数据传递	- 验证按标题切分不破坏段落 - 测试超长文本的递归切分 - 检查 Chunk 的 <code>source</code> 字段正确性
Transform (增强器)	图片描述生成、元数据注入	- Mock Vision LLM，验证描述注入逻辑 - 测试无图片时的降级行为 - 验证幂等性（重复处理相同输入）
Embedding (向量化)	批处理、差量计算、向量维度	- 验证相同文本生成相同向量 - 测试批量请求的拆分与合并 - 检查缓存命中逻辑
BM25 (稀疏编码)	关键词提取、权重计算	- 验证停用词过滤 - 测试 IDF 计算准确性 - 检查稀疏向量格式
Retrieval (检索器)	召回精度、融合算法	- 测试纯 Dense/Sparse/Hybrid 三种模式 - 验证 RRF 融合分数计算 - 检查 Top-K 结果排序
Reranker (重排器)	分数归一化、降级回退	- Mock Cross-Encoder，验证分数重排 - 测试超时后的 Fallback 逻辑 - 验证空候选集处理

技术选型：

- 测试框架： `pytest` (Python 标准选择，支持参数化测试、Fixture 机制)
- Mock 工具： `unittest.mock` / `pytest-mock` (隔离外部依赖，如 LLM API)
- 断言增强： `pytest-check` (支持多断言不中断执行)

4.2.2 集成测试 (Integration Tests)

目标：验证多个组件协作时的数据流转与接口兼容性。

覆盖范围：

测试场景	验证要点	测试策略
Ingestion Pipeline	Loader → Splitter → Transform → Storage 的完整流程	- 使用真实的测试 PDF 文件 - 验证最终存入向量库的数据完整性 - 检查中间产物（如临时图片文件）是否正确清理
Hybrid Search	Dense + Sparse 召回的融合结果	- 准备已知答案的查询-文档对 - 验证融合后的 Top-1 是否命中正确文档 - 测试极端情况（某一路无结果）
Rerank Pipeline	召回 → 过滤 → 重排的组合	- 验证 Metadata 过滤后的候选集正确性 - 检查 Reranker 是否改变了 Top-1 结果 - 测试 Reranker 失败时的回退
MCP Server	工具调用的端到端流程	- 模拟 MCP Client 发送 JSON-RPC 请求 - 验证返回的 content 格式符合协议 - 测试错误处理（如查询语法错误）

技术选型：

- 数据隔离：每个测试使用独立的临时数据库/向量库（`pytest-tempdir`）
- 异步测试： `pytest-asyncio`（若 MCP Server 采用异步实现）
- 契约测试：定义各模块间的 Schema，确保接口不漂移

4.2.3 端到端测试 (End-to-End Tests)

目标：模拟真实用户操作，验证完整业务流程的可用性。

核心场景：

场景 1：数据准备（离线摄取）

- 测试目标：验证文档摄取流程的完整性与正确性
- 测试步骤：
 - 准备测试文档（PDF 文件，包含文本、图片、表格等多种元素）
 - 执行离线摄取脚本，将文档导入知识库
 - 验证摄取结果：检查生成的 Chunk 数量、元数据完整性、图片描述生成
 - 验证存储状态：确认向量库和 BM25 索引正确创建
 - 验证幂等性：重复摄取同一文档，确保不产生重复数据
- 验证要点：
 - Chunk 的切分质量（语义完整性、上下文保留）
 - 元数据字段完整性（source、page、title、tags 等）
 - 图片处理结果（Caption 生成、Base64 编码存储）

- 向量与稀疏索引的正确性

场景 2：召回测试

- **测试目标：**验证检索系统的召回精度与排序质量
- **测试步骤：**
 - 基于已摄取的知识库，准备一组测试查询（包含不同难度与类型）
 - 执行混合检索（Dense + Sparse + Rerank）
 - 验证召回结果：检查 Top-K 文档是否包含预期来源
 - 对比不同检索策略的效果（纯 Dense、纯 Sparse、Hybrid）
 - 验证 Rerank 的影响：对比重排前后的结果变化
- **验证要点：**
 - Hit Rate@K：Top-K 结果命中率是否达标
 - 排序质量：正确答案是否排在前列（MRR、NDCG）
 - 边界情况处理：空查询、无结果查询、超长查询
 - 多模态召回：包含图片的文档是否能通过文本查询召回

场景 3：MCP Client 功能测试

- **测试目标：**验证 MCP Server 与 Client（如 GitHub Copilot）的协议兼容性与功能完整性
- **测试步骤：**
 - 启动 MCP Server（Stdio Transport 模式）
 - 模拟 MCP Client 发送各类 JSON-RPC 请求
 - 测试工具调用：`query_knowledge_hub`、`list_collections` 等
 - 验证返回格式：符合 MCP 协议规范（content 数组、structuredContent）
 - 测试引用透明性：返回结果包含完整的 Citation 信息
 - 测试多模态返回：包含图片的响应正确编码为 Base64
- **验证要点：**
 - 协议合规性：JSON-RPC 2.0 格式、错误码映射
 - 工具注册：`tools/list` 返回所有可用工具及其 Schema
 - 响应格式：TextContent 与 ImageContent 的正确组合
 - 错误处理：无效参数、超时、服务不可用等异常场景
 - 性能指标：单次请求的端到端延迟（含检索、重排、格式化）

测试工具：

- **BDD 框架：**`behave` 或 `pytest-bdd`（以 Gherkin 语法描述场景）
- **环境准备：**
 - 临时测试向量库（独立于生产数据）
 - 预置的标准测试文档集
 - 本地 MCP Server 进程（Stdio Transport）

4.3 RAG 质量评估测试

目标：验证已设计的评估体系（见 3.3.4 评估框架抽象）是否正确实现，并能有效评估 RAG 系统的召回与生成质量。

测试要点：

- 1. 黄金测试集准备
 - 构建标准的"问题-答案-来源文档"测试集 (JSON 格式)
 - 初期人工标注核心场景，后期持续积累坏 Case
- 2. 评估框架实现验证
 - 验证 Ragas/DeepEval 等评估框架的正确集成
 - 确认评估接口能输出标准化的指标字典
 - 测试多评估器并行执行与结果汇总
- 3. 关键指标达标验证
 - 检索指标：Hit Rate@K ≥ 90%、MRR ≥ 0.8、NDCG@K ≥ 0.85
 - 生成指标：Faithfulness ≥ 0.9、Answer Relevancy ≥ 0.85
 - 定期运行评估，监控指标是否回归

说明：本节重点是验证评估体系的工程实现，而非重新设计评估方法（评估方法的设计见第 3 章技术选型）。

4.4 性能与压力测试（可选）

说明：本项目定位为本地 MCP Server，单用户开发环境，采用 Stdio Transport 通信方式。性能与压力测试在当前阶段不是必需的，此处列出主要用于：

- 1. 架构完整性：展示完整的工程化测试体系，体现系统设计的专业性
- 2. 未来扩展性：若后续需要云端部署或多用户支持，可直接参考此方案
- 3. 性能基准建立：通过基础性能测试了解系统瓶颈，为优化提供数据支撑

可选测试场景：

测试类型	验证点	工具	优先级
延迟测试	单次查询的 P50/P95/P99 延迟	pytest-benchmark	中（可帮助识别慢查询）
吞吐量测试	并发查询时的 QPS 上限	locust	低（本地单用户无需求）
内存泄漏检测	长时间运行后的内存占用	memory_profiler	低（短期运行无影响）
向量库性能	不同数据规模下的查询速度	自定义 Benchmark	中（验证扩展性）

4.5 测试工具链与 CI/CD 集成

本地开发工作流：

- 快速验证：仅运行单元测试，秒级反馈
- 完整验证：单元测试 + 集成测试，生成覆盖率报告
- 质量评估：定期执行 RAG 质量测试，监控指标变化

CI/CD Pipeline 设计（可选）：

说明：本地项目不强制要求 CI/CD，但配置自动化测试流程有助于代码质量保障与持续集成实践。

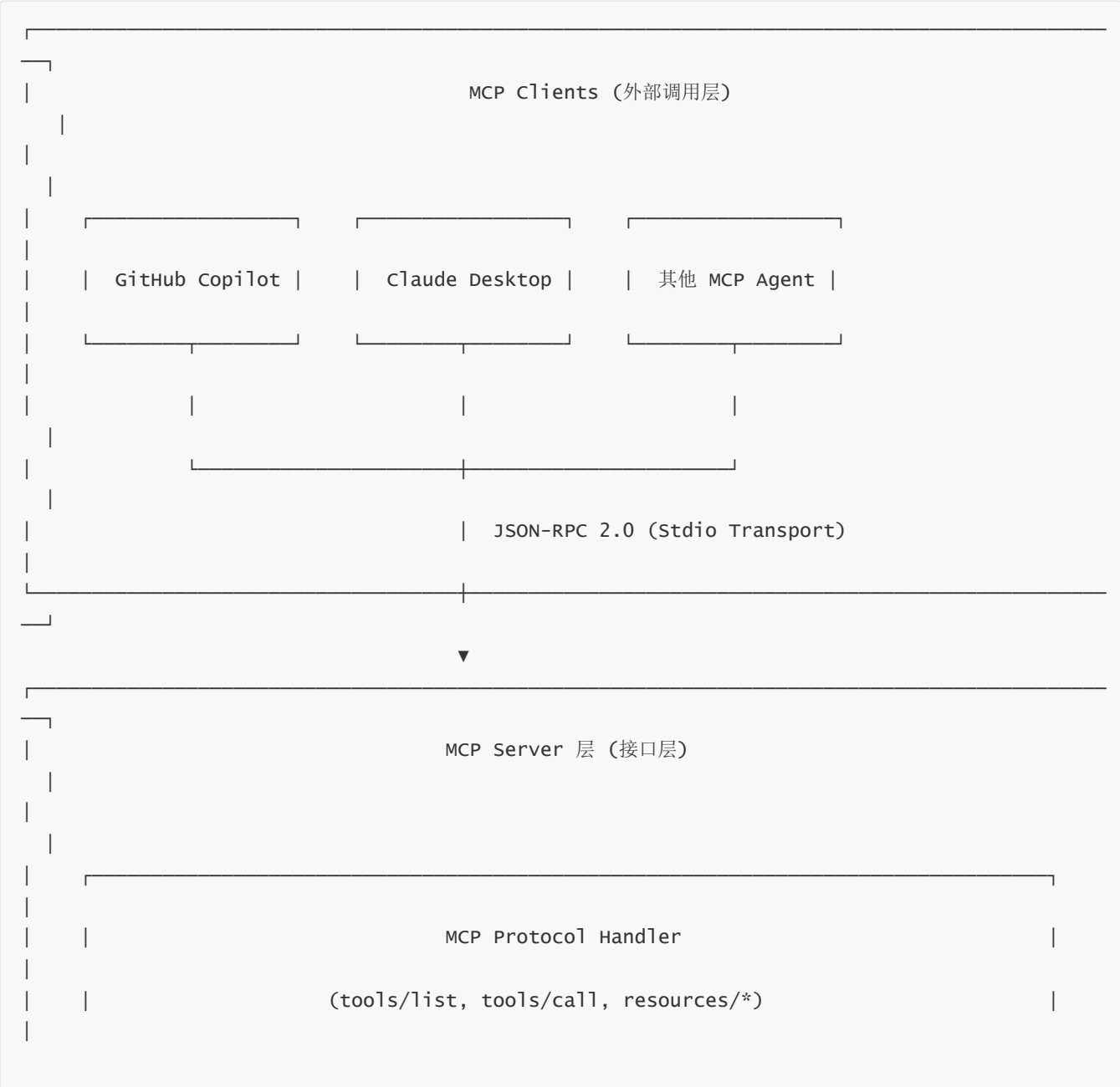
- **单元测试阶段：**每次提交自动触发，验证基础功能，生成覆盖率报告
- **集成测试阶段：**单元测试通过后执行，验证模块协作
- **质量评估阶段：**PR 触发，运行完整的 RAG 质量测试，发布评估报告

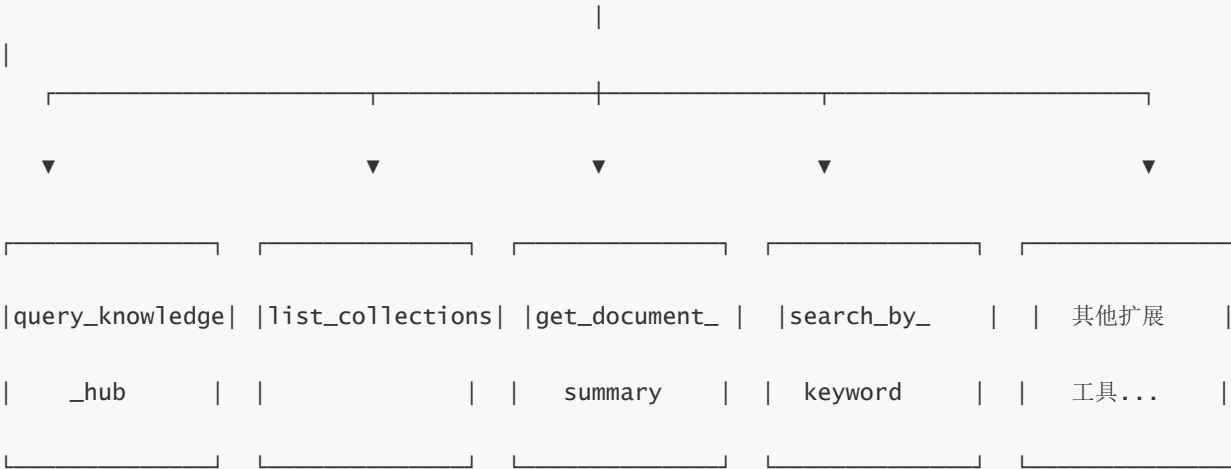
测试覆盖率目标：

- **单元测试：**核心逻辑覆盖率 $\geq 80\%$
- **集成测试：**关键路径覆盖率 100%（如 Ingestion、Hybrid Search）
- **E2E 测试：**核心用户场景覆盖率 100%（至少 3 个关键流程）

5. 系统架构与模块设计

5.1 整体架构图





Core 层 (核心业务逻辑)

Query Engine (查询引擎)

Query Processor (查询预处理)

关键词提取 | 查询扩展 (同义词/别名) | Metadata 解析

Hybrid Search Engine (混合检索引擎)

Dense Route

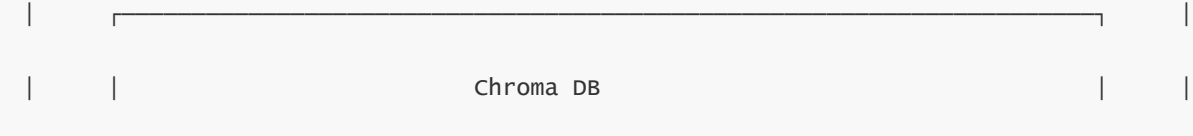
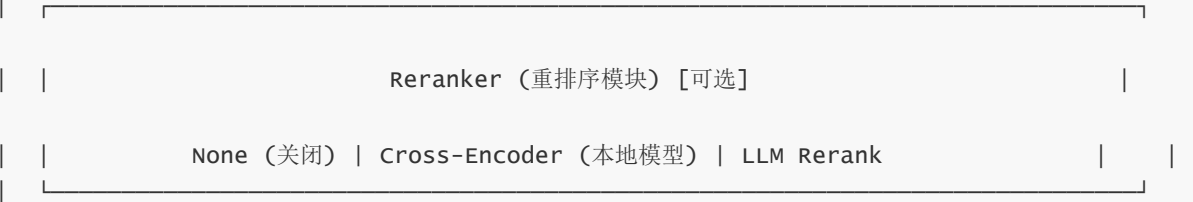
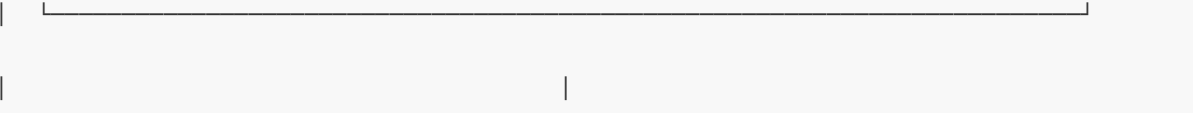
Fusion

Sparse Route

(Embedding 语义)

(RRF)

(BM25 关键词)



Libs 层（可插拔抽象层）

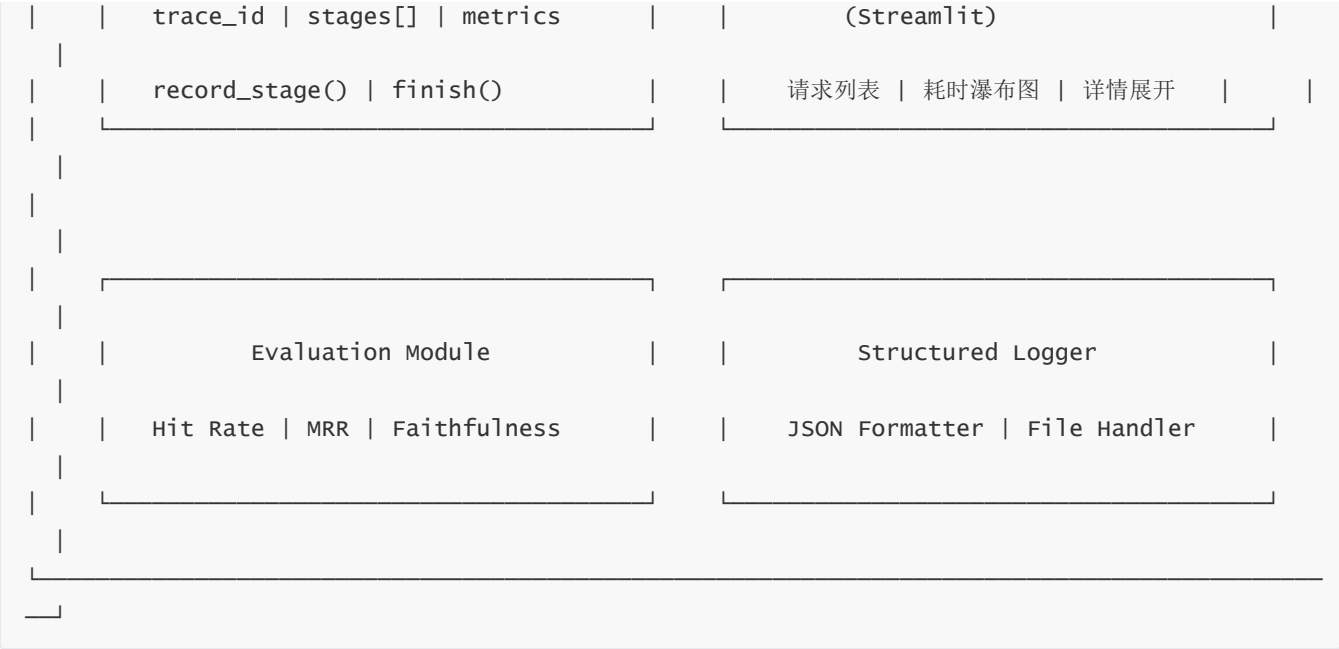
Factory Pattern（工厂模式）

LLM Client	Embedding	Splitter	VectorStore	Reranker	Evaluator
Factory	Factory	Factory	Factory	Factory	Factory
· Azure	· OpenAI	· Recursive	· Chroma	· None	· Ragas
· OpenAI	· BGE	· Semantic	· Qdrant	· CrossEnc	· DeepEval
· Ollama	· Ollama	· FixedLen	· Pinecone	· LLM	· Custom
· DeepSeek	· ...	· ...	· ...		
· Vision🌟					

observability 层（可观测性）

Trace Context

web Dashboard



5.2 目录结构

```
smart-knowledge-hub/
├── config/                                # 配置文件目录
│   ├── settings.yaml                    # 主配置文件 (LLM/Embedding/VectorStore 配置)
│   └── prompts/                         # Prompt 模板目录
│       ├── image_captioning.txt        # 图片描述生成 Prompt
│       ├── chunk_refinement.txt        # Chunk 重写 Prompt
│       └── rerank.txt                  # LLM Rerank Prompt
├── src/                                  # 源代码主目录
│   ├── mcp_server/                     # MCP Server 层 (接口层)
│   │   ├── __init__.py                # MCP Server 入口 (Stdio Transport)
│   │   ├── server.py                  # JSON-RPC 协议处理
│   │   ├── protocol_handler.py        # MCP Tools 定义
│   │   └── tools/
│   │       ├── __init__.py            # 主检索工具
│   │       ├── query_knowledge_hub.py # 列出集合工具
│   │       ├── list_collections.py    # 文档摘要工具
│   │       └── get_document_summary.py
│   ├── core/                           # Core 层 (核心业务逻辑)
│   │   ├── __init__.py                # 配置加载与校验 (Settings:
│   │   ├── settings.py                # load_settings/validate_settings())
│   │   ├── types.py                   # 核心数据类型/契约 (Document/Chunk/ChunkRecord), 供
│   │   └── ingestion/retrieval/mcp 复用
│   │       ├── query_engine/          # 查询引擎模块
│   │       │   ├── __init__.py
│   │       │   ├── query_processor.py # 查询预处理 (关键词提取/查询扩展)
│   │       │   └── hybrid_search.py   # 混合检索引擎 (Dense + Sparse + RRF)
```



```

| | | | └─ dense_retriever.py      # 稠密向量检索
| | | | └─ sparse_retriever.py    # 稀疏检索 (BM25)
| | | | └─ fusion.py              # 结果融合 (RRF 算法)
| | | | └─ reranker.py            # 重排序模块 (None/CrossEncoder/LLM)
| | | |
| | | └─ response/                # 响应构建模块
| | | | └─ __init__.py
| | | | └─ response_builder.py    # 响应构建器
| | | | └─ citation_generator.py  # 引用生成器
| | | | └─ multimodal_assembler.py # 多模态内容组装 (Text + Image)
| | | |
| | | └─ trace/                  # 追踪模块
| | | | └─ __init__.py
| | | | └─ trace_context.py       # 追踪上下文 (trace_id/stages)
| | | | └─ trace_collector.py     # 追踪收集器
| | |
| └─ ingestion/                  # Ingestion Pipeline (离线数据摄取)
| | └─ __init__.py
| | └─ pipeline.py               # Pipeline 主流程编排 (支持 on_progress 回调)
| | └─ document_manager.py       # 文档生命周期管理 (list/delete/stats)
| |
| | └─ chunking/                 # Chunking 模块 (文档切分)
| | | └─ __init__.py
| | | └─ document_chunker.py     # Document → Chunks 转换 (调用 libs.splitter)
| | |
| | └─ transform/                # Transform 模块 (增强处理)
| | | └─ __init__.py
| | | └─ base_transform.py       # Transform 抽象基类
| | | └─ chunk_refiner.py        # Chunk 智能重组/去噪
| | | └─ metadata_enricher.py    # 语义元数据注入 (Title/Summary/Tags)
| | | └─ image_captioner.py      # 图片描述生成 (Vision LLM)
| | |
| | └─ embedding/                # Embedding 模块 (向量化)
| | | └─ __init__.py
| | | └─ dense_encoder.py        # 稠密向量编码
| | | └─ sparse_encoder.py       # 稀疏向量编码 (BM25)
| | | └─ batch_processor.py      # 批处理优化
| | |
| | └─ storage/                  # Storage 模块 (存储)
| | | └─ __init__.py
| | | └─ vector_upserter.py      # 向量库 Upsert
| | | └─ bm25_indexer.py         # BM25 索引构建
| | | └─ image_storage.py        # 图片文件存储
| | |
| └─ libs/                       # Libs 层 (可插拔抽象层)
| | └─ __init__.py
| |
| | └─ loader/                   # Loader 抽象 (文档加载)
| | | └─ __init__.py
| | | └─ base_loader.py          # Loader 抽象基类
| | | └─ pdf_loader.py           # PDF Loader (MarkItDown)
| | | └─ file_integrity.py       # 文件完整性检查 (SHA256 哈希)
| | |

```

```

| | | └─ llm/                                # LLM 抽象
| | |   └─ __init__.py
| | |   └─ base_llm.py                        # LLM 抽象基类
| | |   └─ llm_factory.py                    # LLM 工厂
| | |   └─ azure_llm.py                      # Azure OpenAI 实现
| | |   └─ openai_llm.py                     # OpenAI 实现
| | |   └─ ollama_llm.py                     # ollama 本地模型实现
| | |   └─ deepseek_llm.py                   # DeepSeek 实现
| | |   └─ base_vision_llm.py                 # Vision LLM 抽象基类（支持图像输入）
| | |   └─ azure_vision_llm.py               # Azure Vision 实现（GPT-4o/GPT-4-Vision）
| | |
| | | └─ embedding/                          # Embedding 抽象
| | |   └─ __init__.py
| | |   └─ base_embedding.py                 # Embedding 抽象基类
| | |   └─ embedding_factory.py              # Embedding 工厂
| | |   └─ openai_embedding.py               # OpenAI Embedding 实现
| | |   └─ azure_embedding.py                # Azure Embedding 实现
| | |   └─ ollama_embedding.py               # ollama 本地模型实现
| | |
| | | └─ splitter/                           # Splitter 抽象（切分策略）
| | |   └─ __init__.py
| | |   └─ base_splitter.py                  # Splitter 抽象基类
| | |   └─ splitter_factory.py                # Splitter 工厂
| | |   └─ recursive_splitter.py              # RecursiveCharacterTextSplitter 实现
| | |   └─ semantic_splitter.py              # 语义切分实现
| | |   └─ fixed_length_splitter.py           # 定长切分实现
| | |
| | | └─ vector_store/                       # VectorStore 抽象
| | |   └─ __init__.py
| | |   └─ base_vector_store.py              # VectorStore 抽象基类
| | |   └─ vector_store_factory.py           # VectorStore 工厂
| | |   └─ chroma_store.py                   # Chroma 实现
| | |
| | | └─ reranker/                           # Reranker 抽象
| | |   └─ __init__.py
| | |   └─ base_reranker.py                  # Reranker 抽象基类
| | |   └─ reranker_factory.py                # Reranker 工厂
| | |   └─ cross_encoder_reranker.py          # CrossEncoder 实现
| | |   └─ llm_reranker.py                   # LLM Rerank 实现
| | |
| | | └─ evaluator/                          # Evaluator 抽象
| | |   └─ __init__.py
| | |   └─ base_evaluator.py                 # Evaluator 抽象基类
| | |   └─ evaluator_factory.py               # Evaluator 工厂
| | |   └─ ragas_evaluator.py                # Ragas 实现
| | |   └─ custom_evaluator.py               # 自定义指标实现
| | |
| └─ observability/                          # Observability 层（可观测性）
|   └─ __init__.py
|   └─ logger.py                             # 结构化日志（JSON Formatter）
|   └─ dashboard/                            # Web Dashboard（可视化管理平台）
|     └─ __init__.py
|     └─ app.py                              # Streamlit 入口（页面导航注册）

```

```

├── pages/ # 六大功能页面
│   ├── overview.py # 系统总览（组件配置 + 数据统计）
│   ├── data_browser.py # 数据浏览器（文档/Chunk/图片查看）
│   ├── ingestion_manager.py # Ingestion 管理（触发摄取/删除文档）
│   ├── ingestion_traces.py # Ingestion 追踪（摄取历史与详情）
│   ├── query_traces.py # Query 追踪（查询历史与详情）
│   └── evaluation_panel.py # 评估面板（运行评估/查看指标）
├── services/ # Dashboard 数据服务层
│   ├── trace_service.py # Trace 读取服务（解析 traces.jsonl）
│   ├── data_service.py # 数据浏览服务（ChromaStore/ImageStorage）
│   └── config_service.py # 配置读取服务（Settings 展示）
├── evaluation/ # 评估模块
│   ├── __init__.py
│   ├── eval_runner.py # 评估执行器
│   ├── ragas_evaluator.py # Ragas 评估实现
│   └── composite_evaluator.py # 组合评估器（多后端并行）
├── data/ # 数据目录
│   ├── documents/ # 原始文档存放
│   │   └── {collection}/ # 按集合分类
│   ├── images/ # 提取的图片存放
│   │   └── {collection}/ # 按集合分类（实际存储在 {doc_hash}/ 子目录下）
│   └── db/ # 数据库与索引文件目录
│       ├── ingestion_history.db # 文件完整性历史记录（SQLite）
│       └── # 表结构: file_hash, file_path, status,
processed_at, error_msg # 用途: 增量摄取, 避免重复处理未变更文件
│       └── image_index.db # 图片索引映射（SQLite）
│           # 表结构: image_id, file_path, collection, doc_hash,
page_num # 用途: 快速查询 image_id → 本地文件路径, 支持图片检索与引
用
├── chroma/ # Chroma 向量库目录
│   └── bm25/ # 存储 Dense Vector、Sparse Vector 与 Chunk Metadata
│       # BM25 索引目录
│       # 存储倒排索引与 IDF 统计信息（当前使用 pickle）
├── cache/ # 缓存目录
│   ├── embeddings/ # Embedding 缓存（按内容哈希）
│   ├── captions/ # 图片描述缓存
│   └── processing/ # 处理状态缓存（文件哈希/Chunk 哈希）
├── logs/ # 日志目录
│   ├── traces.jsonl # 追踪日志（JSON Lines）
│   └── app.log # 应用日志
├── tests/ # 测试目录
│   ├── unit/ # 单元测试
│   │   ├── test_dense_retriever.py # D2: 稠密检索器测试
│   │   ├── test_sparse_retriever.py # D3: 稀疏检索器测试
│   │   ├── test_fusion_rrf.py # D4: RRF 融合测试
│   │   └── test_reranker_fallback.py # D6: Reranker 回退测试

```

			test_protocol_handler.py	# E2: 协议处理器测试
			test_response_builder.py	# E3: 响应构建器测试
			test_list_collections.py	# E4: 集合列表工具测试
			test_get_document_summary.py	# E5: 文档摘要工具测试
			test_trace_context.py	# F1: 追踪上下文测试
			test_jsonl_logger.py	# F2: JSON Lines 日志测试
			...	# 其他已有单元测试
			integration/	# 集成测试
			test_ingestion_pipeline.py	
			test_hybrid_search.py	# D5: 混合检索集成测试
			test_mcp_server.py	# E1-E6: MCP 服务器集成测试
			e2e/	# 端到端测试
			test_data_ingestion.py	
			test_recall.py	# G2: 召回回归测试
			test_mcp_client.py	# G1: MCP Client 模拟测试
			fixtures/	# 测试数据
			sample_documents/	
			golden_test_set.json	# F5/G2: 黄金测试集
			scripts/	# 脚本目录
			ingest.py	# 数据摄取脚本（离线摄取入口）
			query.py	# 查询测试脚本（在线查询入口）
			evaluate.py	# 评估运行脚本
			start_dashboard.py	# Dashboard 启动脚本
			main.py	# MCP Server 启动入口
			pyproject.toml	# Python 项目配置
			requirements.txt	# 依赖列表
			README.md	# 项目说明

5.3 模块说明

5.3.1 MCP Server 层

模块	职责	关键技术点
server.py	MCP Server 主入口，处理 Stdio Transport 通信	Python MCP SDK, JSON-RPC 2.0
protocol_handler.py	协议解析与能力协商	initialize、tools/list、tools/call
tools/*	对外暴露的工具函数实现	装饰器定义，参数校验，响应格式化

5.3.2 Core 层

模块	职责	关键技术点
settings.py	配置加载与校验	读取 config/settings.yaml，解析为 Settings，必填字段校验 (fail-fast)

模块	职责	关键技术点
<code>types.py</code>	核心数据类型/契约（全链路复用）	定义 <code>Document/Chunk/ChunkRecord/ProcessedQuery/RetrievalResult</code> ；序列化稳定；作为 <code>ingestion/retrieval/mcp</code> 的数据契约中心
<code>query_processor.py</code>	查询预处理	关键词提取、同义词扩展、Metadata 解析
<code>hybrid_search.py</code>	混合检索编排	并行 <code>Dense/Sparse</code> 召回，结果融合，Metadata 过滤
<code>dense_retriever.py</code>	语义向量检索	<code>Query Embedding + VectorStore</code> 检索， <code>Cosine Similarity</code>
<code>sparse_retriever.py</code>	BM25 关键词检索	倒排索引查询， <code>TF-IDF</code> 打分
<code>fusion.py</code>	结果融合	<code>RRF</code> 算法，排名倒数加权
<code>reranker.py</code>	精排重排	<code>CrossEncoder / LLM Rerank / Fallback</code> 回退
<code>response_builder.py</code>	响应构建	<code>MCP</code> 响应格式化， <code>Markdown</code> 生成
<code>citation_generator.py</code>	引用生成	从检索结果生成结构化引用列表
<code>multimodal_assembler.py</code>	多模态组装	<code>Text + Image Base64</code> 编码， <code>MCP</code> 多内容类型
<code>trace_context.py</code>	追踪上下文	<code>trace_id</code> 生成，阶段记录， <code>finish</code> 汇总
<code>trace_collector.py</code>	追踪收集器	收集 <code>trace</code> 并触发持久化到 <code>JSON Lines</code>

5.3.3 Scripts 层（命令行入口）

脚本	职责	关键技术点
<code>ingest.py</code>	离线数据摄取入口	<code>CLI</code> 参数解析，调用 <code>Ingestion Pipeline</code> ，支持 <code>--collection / --path / --force</code>
<code>query.py</code>	在线查询测试入口	<code>CLI</code> 参数解析，调用 <code>HybridSearch + Reranker</code> ，支持 <code>--query / --top-k / --verbose</code>
<code>evaluate.py</code>	评估运行入口	加载 <code>golden_test_set</code> ，运行评估，输出 <code>metrics</code>
<code>start_dashboard.py</code>	Dashboard 启动入口	<code>Streamlit</code> 应用启动

5.3.4 Ingestion Pipeline 层

模块	职责	关键技术点
<code>pipeline.py</code>	Pipeline 流程编排	串行执行（或分阶段可观测），异常处理，增量更新；支持 <code>on_progress</code> 回调；统一使用 <code>core/types.py</code> 的数据契约
<code>document_manager.py</code>	文档生命周期管理	<code>list/delete/stats</code> 操作；跨 4 个存储（ <code>Chroma/BM25/ImageStorage/FileIntegrity</code> ）的协调删除；供 Dashboard 与 <code>CLI</code> 调用

| chunking/document_chunker.py | Document→Chunks 转换 | 调用 libs.splitter 进行文本切分；生成稳定 Chunk ID（格式：{doc_id}_{index:04d}_{hash}）；继承 metadata；建立 source_ref 溯源链接 |

| transform/base_transform.py | Transform 抽象 | 原子化、幂等；可独立重试；失败降级不阻塞 |

| transform/chunk_refiner.py | Chunk 智能重组 | 规则去噪 + 可选 LLM 二次加工；可回退 |

| transform/metadata_enricher.py | 元数据增强 | Title/Summary/Tags 规则生成 + 可选 LLM 增强 |

| transform/image_captioner.py | 图片描述生成 | Vision LLM；写回 metadata/text；禁用/失败降级 |

| embedding/dense_encoder.py | 稠密向量编码 | 通过 libs.embedding 调用具体 provider；批处理 |

| embedding/sparse_encoder.py | 稀疏向量编码 | BM25 编码/统计（或替换实现）；批处理 |

| storage/vector_upserter.py | 向量存储写入 | 通过 libs.vector_store Upsert；幂等；metadata 完整 |

5.3.5 Libs 层 (可插拔抽象)

抽象接口	当前默认实现	可替换选项
LLMClient	Azure OpenAI	OpenAI / Ollama / DeepSeek
VisionLLMClient	Azure OpenAI Vision (GPT-4o)	OpenAI Vision / Ollama Vision (LLaVA)
EmbeddingClient	OpenAI text-embedding-3	BGE / Ollama 本地模型
Loader	PDF Loader (MarkItDown)	Markdown/HTML/Code Loader 等
FileIntegrity	SQLite (data/db/ingestion_history.db)	Redis (分布式) / PostgreSQL (企业级) / JSON文件 (测试)
Splitter	RecursiveCharacterTextSplitter	Semantic / FixedLen
VectorStore	Chroma	Qdrant / Pinecone / Milvus
Reranker	CrossEncoder	LLM Rerank / None (关闭)
Evaluator	Ragas	DeepEval / 自定义指标

5.3.6 Observability 层

模块	职责	关键技术点
logger.py	结构化日志	JSON Formatter, JSON Lines 输出
trace_context.py	请求级追踪	trace_id, trace_type (query/ingestion), 阶段耗时记录, finish() + to_dict() 序列化
trace_collector.py	追踪收集器	收集 trace 并触发持久化到 JSON Lines
dashboard/app.py	Dashboard 入口	Streamlit 多页面应用, st.navigation 页面注册
dashboard/pages/overview.py	系统总览	组件配置卡片, 数据资产统计
dashboard/pages/data_browser.py	数据浏览器	文档列表, Chunk 详情, 图片预览
dashboard/pages/ingestion_manager.py	Ingestion 管理	文件上传, 摄取触发 (进度条), 文档删除

模块	职责	关键技术点
<code>dashboard/pages/ingestion_traces.py</code>	Ingestion 追踪	摄取历史, 阶段耗时瀑布图
<code>dashboard/pages/query_traces.py</code>	Query 追踪	查询历史, Dense/Sparse 对比, Rerank 变化
<code>dashboard/pages/evaluation_panel.py</code>	评估面板	运行评估, 指标展示, 历史趋势 (Phase H 实现)
<code>dashboard/services/trace_service.py</code>	Trace 数据服务	解析 traces.jsonl, 按 trace_type 分类
<code>dashboard/services/data_service.py</code>	数据浏览服务	封装 ChromaStore/ImageStorage 读取
<code>dashboard/services/config_service.py</code>	配置读取服务	封装 Settings 展示
<code>evaluation/eval_runner.py</code>	评估执行	黄金测试集, 指标计算, 报告生成
<code>evaluation/ragas_evaluator.py</code>	Ragas 评估	Faithfulness, Answer Relevancy, Context Precision
<code>evaluation/composite_evaluator.py</code>	组合评估器	多后端并行执行, 结果汇总

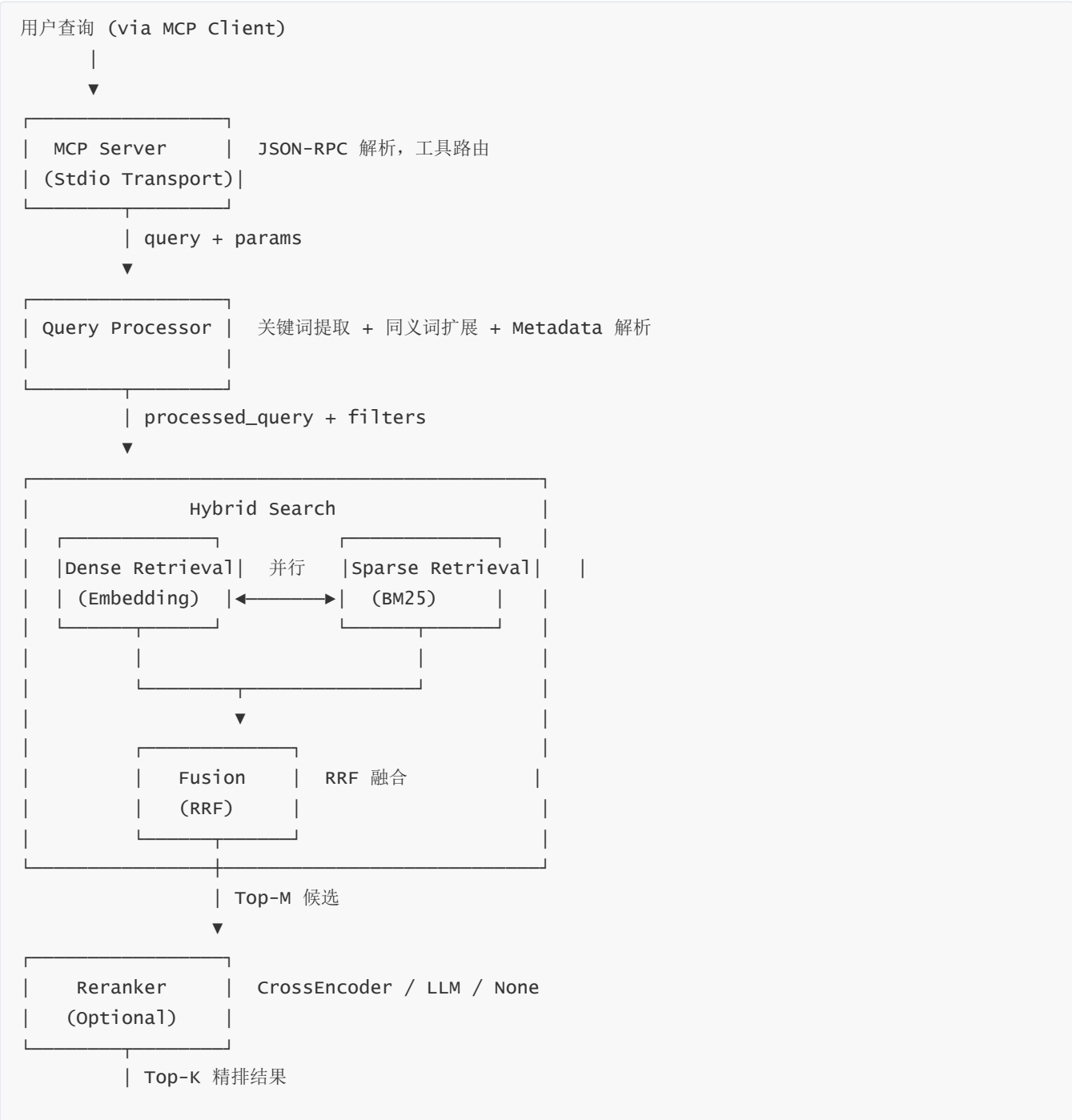
5.4 数据流说明

5.4.1 离线数据摄取流 (Ingestion Flow)





5.4.2 在线查询流 (Query Flow)





5.4.3 管理操作流 (Management Flow)



5.5 配置驱动设计

系统通过 config/settings.yaml 统一配置各组件实现，支持零代码切换：

```
# config/settings.yaml 示例

# LLM 配置
llm:
  provider: azure           # azure | openai | ollama | deepseek
  model: gpt-4o
```

```
azure_endpoint: "..."  
api_key: "${AZURE_API_KEY}"  
  
# Embedding 配置  
embedding:  
  provider: openai          # openai | azure | ollama (本地)  
  model: text-embedding-3-small  
  
# Vision LLM 配置 (图片描述)  
vision_llm:  
  provider: azure          # azure | dashscope (Qwen-VL)  
  model: gpt-4o  
  
# 向量存储配置  
vector_store:  
  backend: chroma          # chroma | qdrant | pinecone  
  persist_path: ./data/db/chroma  
  
# 检索配置  
retrieval:  
  sparse_backend: bm25      # bm25 | elasticsearch  
  fusion_algorithm: rrf     # rrf | weighted_sum  
  top_k_dense: 20  
  top_k_sparse: 20  
  top_k_final: 10  
  
# 重排配置  
rerank:  
  backend: cross_encoder   # none | cross_encoder | llm  
  model: cross-encoder/ms-marco-MiniLM-L-6-v2  
  top_m: 30  
  
# 评估配置  
evaluation:  
  backends: [ragas, custom]  
  golden_test_set: ./tests/fixtures/golden_test_set.json  
  
# 可观测性配置  
observability:  
  enabled: true  
  log_file: ./logs/traces.jsonl  
  
# Dashboard 管理平台配置  
dashboard:  
  enabled: true  
  port: 8501                # Streamlit 服务端口  
  traces_dir: ./logs        # Trace 日志文件目录  
  auto_refresh: true        # 是否自动刷新 (轮询新 trace)  
  refresh_interval: 5       # 自动刷新闻隔 (秒)
```

5.6 扩展性设计要点

1. **新增 LLM Provider**: 实现 `BaseLLM` 接口, 在 `llm_factory.py` 注册, 配置文件指定 `provider` 即可
2. **新增文档格式**: 实现 `BaseLoader` 接口, 在 Pipeline 中注册对应文件扩展名的处理器
3. **新增检索策略**: 实现检索接口, 在 `hybrid_search.py` 中组合调用
4. **新增评估指标**: 实现 `BaseEvaluator` 接口, 在配置中添加到 `backends` 列表

6. 项目排期

排期原则 (严格对齐本 DEV_SPEC 的架构分层与目录结构)

- **只按本文档设计落地**: 以第 5.2 节目录树为“交付清单”, 每一步都要在文件系统上产生可见变化。
- **1 小时一个可验收增量**: 每个小阶段 (≈1h) 都必须同时给出“验收标准 + 测试方法”, 尽量做到 TDD。
- **先打通主闭环, 再补齐默认实现**: 优先做“可跑通的端到端路径 (Ingestion → Retrieval → MCP Tool)”, 并在 Libs 层补齐可运行的默认后端实现, 避免出现“只有接口没有实现”的空转。
- **外部依赖可替换/可 Mock**: LLM/Embedding/Vision/VectorStore 的真实调用在单元测试中一律用 Fake/Mock, 集成测试再开真实后端 (可选)。

阶段总览 (大阶段 → 目的)

1. 阶段 A: 工程骨架与测试基座

- 目的: 建立可运行、可配置、可测试的工程骨架; 后续所有模块都能以 TDD 方式落地。

2. 阶段 B: Libs 可插拔层 (Factory + Base 接口 + 默认可运行实现)

- 目的: 把“可替换”变成代码事实; 并补齐可运行的默认后端实现, 确保 Core / Ingestion 不仅“可编译”, 还可在真实环境跑通。

3. 阶段 C: Ingestion Pipeline (PDF→MD→Chunk→Embedding→Upsert)

- 目的: 离线摄取链路跑通, 能把样例文档写入向量库/BM25 索引并支持增量。

4. 阶段 D: Retrieval (Dense + Sparse + RRF + 可选 Rerank)

- 目的: 在线查询链路跑通, 得到 Top-K chunks (含引用信息), 并具备稳定回退策略。

5. 阶段 E: MCP Server 层与 Tools 落地

- 目的: 按 MCP 标准暴露 tools, 让 Copilot/Claude 可直接调用查询能力。

6. 阶段 F: Trace 基础设施与打点

- 目的: 增强 TraceContext, 实现结构化日志持久化, 在 Ingestion + Query 双链路打点, 添加 Pipeline 进度回调。

7. 阶段 G: 可视化管理平台 Dashboard

- 目的: 搭建 Streamlit 六页面管理平台 (系统总览 / 数据浏览 / Ingestion 管理 / Ingestion 追踪 / Query 追踪 / 评估占位), 实现 DocumentManager 跨存储协调。

8. 阶段 H: 评估体系

- 目的: 实现 RagasEvaluator + CompositeEvaluator + EvalRunner, 启用评估面板页面, 建立 golden test set 回归基线。

9. 阶段 I: 端到端验收与文档收口

- 目的：补齐 E2E 测试（MCP Client 模拟 + Dashboard 冒烟），完善 README，全链路验收，确保“开箱即用 + 可复现”。

进度跟踪表 (Progress Tracking)

状态说明： ☐ 未开始 | ☒ 进行中 | ☒ 已完成

更新时间： 每完成一个子任务后更新对应状态

阶段 A：工程骨架与测试基座

任务编号	任务名称	状态	完成日期	备注
A1	初始化目录树与最小可运行入口	[x]	2026-01-26	目录结构、配置文件、main.py 已创建
A2	引入 pytest 并建立测试目录约定	[x]	2026-01-26	pytest 配置、tests/ 目录结构、22 个冒烟测试
A3	配置加载与校验 (Settings)	[x]	2026-01-26	配置加载、校验与单元测试

阶段 B：Libs 可插拔层

任务编号	任务名称	状态	完成日期	备注
B1	LLM 抽象接口与工厂	[x]	2026-01-27	BaseLLM + LLMFactory + 16个单元测试
B2	Embedding 抽象接口与工厂	[x]	2026-01-27	BaseEmbedding + EmbeddingFactory + 22个单元测试
B3	Splitter 抽象接口与工厂	[x]	2026-01-27	BaseSplitter + SplitterFactory + 20个单元测试
B4	VectorStore 抽象接口与工厂	[x]	2026-01-27	BaseVectorStore + VectorStoreFactory + 34个单元测试
B5	Reranker 抽象接口与工厂 (含 None 回退)	[x]	2026-01-27	BaseReranker + RerankerFactory + NoneReranker + 单元测试
B6	Evaluator 抽象接口与工厂	[x]	2026-01-27	BaseEvaluator + EvaluatorFactory + CustomEvaluator + 单元测试
B7.1	OpenAI-Compatible LLM 实现	[x]	2026-01-28	OpenAILLM + AzureLLM + DeepSeekLLM + 33个单元测试
B7.2	Ollama LLM 实现	[x]	2026-01-28	OllamaLLM + 32个单元测试

任务编号	任务名称	状态	完成日期	备注
B7.3	OpenAI & Azure Embedding 实现	[x]	2026-01-28	OpenAIEmbedding + AzureEmbedding + 27个单元测试
B7.4	Ollama Embedding 实现	[x]	2026-01-28	OllamaEmbedding + 20个单元测试
B7.5	Recursive Splitter 默认实现	[x]	2026-01-28	RecursiveSplitter + 24个单元测试 + langchain集成
B7.6	ChromaStore 默认实现	[x]	2026-01-30	ChromaStore + 20个集成测试 + roundtrip 验证
B7.7	LLM Reranker 实现	[x]	2026-01-30	LLMReranker + 20个单元测试 + prompt模板支持
B7.8	Cross-Encoder Reranker 实现	[x]	2026-01-30	CrossEncoderReranker + 26个单元测试 + 工厂集成
B8	Vision LLM 抽象接口与工厂集成	[x]	2026-01-31	BaseVisionLLM + ImageInput + LLMFactory 扩展 + 35个单元测试
B9	Azure Vision LLM 实现	[x]	2026-01-31	AzureVisionLLM + 22个单元测试 + mock测试 + 图片压缩

阶段 C: Ingestion Pipeline MVP

任务编号	任务名称	状态	完成日期	备注
C1	定义核心数据类型/契约 (Document/Chunk/ChunkRecord)	[x]	2026-01-30	Document/Chunk/ChunkRecord + 18个单元测试
C2	文件完整性检查 (SHA256)	[x]	2026-01-30	FileIntegrityChecker + SQLiteIntegrityChecker + 25个单元测试
C3	Loader 抽象基类与 PDF Loader	[x]	2026-01-30	BaseLoader + PdfLoader + PyMuPDF图片提取 + 21单元测试 + 9集成测试
C4	Splitter 集成 (调用 Libs)	[x]	2026-01-31	DocumentChunker + 19个单元测试 + 5个核心增值功能
C5	Transform 基类 + ChunkRefiner	[x]	2026-01-31	BaseTransform + ChunkRefiner (Rule + LLM) + TraceContext + 25单元测试 + 5集成测试
C6	MetadataEnricher	[x]	2026-01-31	MetadataEnricher (Rule + LLM) + 26单元测试 + 真实LLM集成测试
C7	ImageCaptioner	[x]	2026-02-01	ImageCaptioner + Azure Vision LLM 实现 + 集成测试

任务编号	任务名称	状态	完成日期	备注
C8	DenseEncoder	[x]	2026-02-01	批量编码+Azure集成测试
C9	SparseEncoder	[x]	2026-02-01	词频统计+语料库统计+26单元测试
C10	BatchProcessor	[x]	2026-02-01	BatchProcessor + BatchResult + 20个单元测试
C11	BM25Indexer (倒排索引+IDF计算)	[x]	2026-02-01	BM25索引器+IDF计算+持久化+26单元测试
C12	VectorUpsserter (幂等upsert)	[x]	2026-02-01	稳定chunk_id生成+幂等upsert+21单元测试
C13	ImageStorage (图片存储+SQLite索引)	[x]	2026-02-01	ImageStorage + SQLite索引 + 37个单元测试 + WAL并发支持
C14	Pipeline 编排 (MVP 串起来)	[x]	2026-02-02	完整流程编排+Azure LLM/Embedding集成测试通过
C15	脚本入口 ingest.py	[x]	2026-02-02	CLI脚本+E2E测试+文件发现+skip功能

阶段 D: Retrieval MVP

任务编号	任务名称	状态	完成日期	备注
D1	QueryProcessor (关键词提取 + filters)	[x]	2026-02-03	ProcessedQuery类型+关键词提取+停用词过滤+filter语法+38单元测试
D2	DenseRetriever (调用 VectorStore.query)	[x]	2026-02-03	RetrievalResult类型+依赖注入+ChromaStore.query修复+30单元测试
D3	SparseRetriever (BM25 查询)	[x]	2026-02-04	BaseVectorStore.get_by_ids+ChromaStore实现+SparseRetriever+26单元测试
D4	RRF Fusion	[x]	2026-02-04	RRFFusion类+k参数可配置+加权融合+确定性输出+34单元测试
D5	HybridSearch 编排	[x]	2026-02-04	HybridSearch类+并行检索+优雅降级+元数据过滤+29集成测试
D6	Reranker (Core 层编排 + Fallback)	[x]	2026-02-04	CoreReranker+LLM Reranker集成+Fallback机制+27单元测试+7集成测试
D7	脚本入口 query.py (查询可用)	[x]	2026-02-04	CLI 查询入口 + verbose 输出

阶段 E：MCP Server 层与 Tools

任务编号	任务名称	状态	完成日期	备注
E1	MCP Server 入口与 Stdio 约束	[x]	2026-02-04	server.py 使用官方 MCP SDK + stdio + 2集成测试
E2	Protocol Handler 协议解析与能力协商	[x]	2026-02-04	ProtocolHandler类+tool注册+错误处理+20单元测试
E3	query_knowledge_hub Tool	[x]	2026-02-04	ResponseBuilder+CitationGenerator+Tool注册+24单元测试+2集成测试
E4	list_collections Tool	[x]	2026-02-04	ListCollectionsTool+CollectionInfo+ChromaDB集成+41单元测试+2集成测试
E5	get_document_summary Tool	[x]	2026-02-04	GetDocumentSummaryTool+DocumentSummary+错误处理+71单元测试
E6	多模态返回组装 (Text + Image)	[x]	2026-02-04	MultimodalAssembler+base64编码+MIME检测+ResponseBuilder集成+54单元测试+4集成测试

阶段 F：Trace 基础设施与打点

任务编号	任务名称	状态	完成日期	备注
F1	TraceContext 增强 (finish + 耗时统计 + trace_type)	[x]	2026-02-08	TraceContext增强 (trace_type/finish/elapsed_ms/to_dict)+TraceCollector+28单元测试
F2	结构化日志 logger (JSON Lines)	[x]	2026-02-08	JSONFormatter+get_trace_logger+write_trace+16单元测试
F3	在 Query 链路打点	[x]	2026-02-08	HybridSearch+CoreReranker trace注入(5阶段)+14集成测试
F4	在 Ingestion 链路打点	[x]	2026-02-08	Pipeline五阶段trace注入 (load/split/transform/embed/upsert)+11集成测试
F5	Pipeline 进度回调 (on_progress)	[x]	2026-02-08	on_progress回调(6阶段通知)+6单元测试

阶段 G：可视化管理平台 Dashboard

任务编号	任务名称	状态	完成日期	备注
G1	Dashboard 基础架构与系统总览页	[x]	2026-02-09	app.py多页面导航+overview页+ConfigService+start_dashboard.py+11单元测试

任务编号	任务名称	状态	完成日期	备注
G2	DocumentManager 实现	[x]	2026-02-09	DocumentManager跨存储协调 (ChromaStore+BM25+ImageStorage+IntegrityChecker)+ 文档删除+21单元测试
G3	数据浏览器页面	[x]	2026-02-09	DataService只读门面+文档列表+chunk内容展示+元数据JSON展开+collection切换
G4	Ingestion 管理页面	[x]	2026-02-09	文件上传+IngestionPipeline集成+实时进度条+TraceContext自动记录
G5	Ingestion 追踪页面	[x]	2026-02-09	TraceService读取traces.jsonl+阶段时间线+耗时柱状图+stage详情展开
G6	Query 追踪页面	[x]	2026-02-09	Query trace过滤+检索结果展示+rerank对比+耗时分析

阶段 H：评估体系

任务编号	任务名称	状态	完成日期	备注
H1	RagasEvaluator 实现	[x]	2026-02-09	19/19 tests passed
H2	CompositeEvaluator 实现	[x]	2026-02-09	11/11 tests passed
H3	EvalRunner + Golden Test Set	[x]	2026-02-09	15/15 tests passed
H4	评估面板页面	[x]	2026-02-09	6/6 tests passed, dashboard page with history tracking
H5	Recall 回归测试 (E2E)	[x]	2026-02-09	3 unit+4 e2e(skip without data), hit@k+MRR threshold gating

阶段 I：端到端验收与文档收口

任务编号	任务名称	状态	完成日期	备注
I1	E2E：MCP Client 侧调用模拟	[x]	2026-02-23	7个E2E测试+import死锁修复+非阻塞readline
I2	E2E：Dashboard 冒烟测试	[x]	2026-02-24	6个页面冒烟测试+AppTest框架+mock服务
I3	完善 README (运行说明 + MCP + Dashboard)	[x]	2026-02-24	快速开始+配置说明+MCP配置+Dashboard指南+测试+FAQ

任务编号	任务名称	状态	完成日期	备注
I4	清理接口一致性（契约测试补齐）	[x]	2026-02-24	VectorStore+Reranker+Evaluator边界测试+83测试全绿
I5	全链路 E2E 验收	[x]	2026-02-24	1198单元+30e2e通过,ingest/query/evaluate脚本验证

 总体进度

阶段	总任务数	已完成	进度
阶段 A	3	3	100%
阶段 B	16	16	100%
阶段 C	15	15	100%
阶段 D	7	7	100%
阶段 E	6	6	100%
阶段 F	5	5	100%
阶段 G	6	6	100%
阶段 H	5	5	100%
阶段 I	5	5	100%
总计	68	68	100%

阶段 A：工程骨架与测试基座（目标：先可导入，再可测试）

A1：初始化目录树与最小可运行入口

- 目标：在 repo 根目录创建第 5.2 节所述目录骨架与空模块文件（可 import）。
- 修改文件：
 - `main.py`
 - `pyproject.toml`
 - `README.md`
 - `.gitignore`（Python 项目标准忽略规则：`__pycache__`、`.venv`、`.env`、`*.pyc`、IDE 配置等）
 - `src/**/__init__.py`（按目录树补齐）
 - `config/settings.yaml`（最小可解析配置）
 - `config/prompts/image_captioning.txt`（可先放占位内容，后续阶段补充 Prompt）

- `config/prompts/chunk_refinement.txt` (可先放占位内容, 后续阶段补充 Prompt)
- `config/prompts/rerank.txt` (可先放占位内容, 后续阶段补充 Prompt)
- **实现类/函数:** 无 (仅骨架)。
- **实现类/函数:** 无 (仅骨架, 不实现业务逻辑)。
- **实现类/函数:** 为当前项目创建一个虚拟环境模块。
- **验收标准:**
- 目录结构与 DEV_SPEC 5.2 一致 (至少把对应目录创建出来)。
- `config/prompts/` 目录存在, 且三个 prompt 文件可被读取 (即使只是占位文本)。
- 能导入关键顶层包 (与目录结构一一对应) :
 - `python -c "import mcp_server; import core; import ingestion; import libs; import observability"`
- 可以启动虚拟环境模块
- **测试方法:** 运行 `python -m compileall src` (仅做语法/可导入性检查; pytest 基座在 A2 建立)。

A2: 引入 pytest 并建立测试目录约定

- **目标:** 建立 `tests/unit|integration|e2e|fixtures` 目录与 pytest 运行基座。
- **修改文件:**
 - `pyproject.toml` (添加 pytest 配置: testpaths、markers 等)
 - `tests/unit/test_smoke_imports.py`
 - `tests/fixtures/sample_documents/` (放 1 个最小样例文档占位)
- **实现类/函数:** 无。
- **实现类/函数:** 无 (新增的是测试文件与 pytest 配置)。
- **验收标准:**
 - `pytest -q` 可运行并通过。
 - 至少 1 个冒烟测试 (例如 `tests/unit/test_smoke_imports.py` 只做关键包 import 校验)。
- **测试方法:** `pytest -q tests/unit/test_smoke_imports.py`。

A3: 配置加载与校验 (Settings)

- **目标:** 实现读取 `config/settings.yaml` 的配置加载器, 并在启动时校验关键字段存在。
- **修改文件:**
 - `main.py` (启动时调用 `load_settings()`, 缺字段直接 fail-fast 退出)
 - `src/observability/logger.py` (先占位: 提供 `get_logger`, `stderr` 输出)
 - `src/core/settings.py` (新增: 集中放 Settings 数据结构与加载/校验逻辑)
 - `config/settings.yaml` (补齐字段: `llm/embedding/vector_store/retrieval/rerank/evaluation/observability`)
 - `tests/unit/test_config_loading.py`
- **实现类/函数:**

- `Settings` (dataclass: 只做结构与最小校验; 不在此处做任何网络/IO 的“业务初始化”)
 - `load_settings(path: str) -> Settings` (读取 YAML -> 解析为 `Settings` -> 校验必填字段)
 - `validate_settings(settings: Settings) -> None` (把“必填字段检查”集中化, 错误信息包含字段路径, 例如 `embedding.provider`)
 - 验收标准:
 - `main.py` 启动时能成功加载 `config/settings.yaml` 并拿到 `Settings` 对象。
 - 删除/缺失关键字段时 (例如 `embedding.provider`), 启动或 `load_settings()` 抛出“可读错误” (明确指出缺的是哪个字段)。
 - 测试方法: `pytest -q tests/unit/test_config_loading.py`。
-

阶段 B: Libs 可插拔层 (目标: Factory 可工作, 且至少有“默认后端”可跑通端到端)

B1: LLM 抽象接口与工厂

- 目标: 定义 `BaseLLM` 与 `LLMFactory`, 支持按配置选择 provider。
- 修改文件:
 - `src/libs/llm/base_llm.py`
 - `src/libs/llm/llm_factory.py`
 - `tests/unit/test_llm_factory.py`
- 实现类/函数:
 - `BaseLLM.chat(messages) -> str` (或统一 response 对象)
 - `LLMFactory.create(settings) -> BaseLLM`
- 验收标准: 在测试里用 Fake provider (测试内 stub) 验证工厂路由逻辑。
- 测试方法: `pytest -q tests/unit/test_llm_factory.py`。

B2: Embedding 抽象接口与工厂

- 目标: 定义 `BaseEmbedding` 与 `EmbeddingFactory`, 支持批量 embed。
- 修改文件:
 - `src/libs/embedding/base_embedding.py`
 - `src/libs/embedding/embedding_factory.py`
 - `tests/unit/test_embedding_factory.py`
- 实现类/函数:
 - `BaseEmbedding.embed(texts: list[str], trace: TraceContext | None = None) -> list[list[float]]`
 - `EmbeddingFactory.create(settings) -> BaseEmbedding`
- 验收标准: Fake embedding 返回稳定向量, 工厂按 provider 分流。
- 测试方法: `pytest -q tests/unit/test_embedding_factory.py`。

B3: Splitter 抽象接口与工厂

- **目标:** 定义 `BaseSplitter` 与 `SplitterFactory`, 支持不同切分策略 (Recursive/Semantic/Fixed)。
- **修改文件:**
 - `src/libs/splitter/base_splitter.py`
 - `src/libs/splitter/splitter_factory.py`
 - `tests/unit/test_splitter_factory.py`
- **实现类/函数:**
 - `BaseSplitter.split_text(text: str, trace: TraceContext | None = None) -> List[str]`
 - `SplitterFactory.create(settings) -> BaseSplitter`
- **验收标准:** Factory 能根据配置返回不同类型的 Splitter 实例 (测试中可用 Fake 实现)。
- **测试方法:** `pytest -q tests/unit/test_splitter_factory.py`。

B4: VectorStore 抽象接口与工厂 (先定义契约)

- **目标:** 定义 `BaseVectorStore` 与 `VectorStoreFactory`, 先不接真实 DB。
- **修改文件:**
 - `src/libs/vector_store/base_vector_store.py`
 - `src/libs/vector_store/vector_store_factory.py`
 - `tests/unit/test_vector_store_contract.py`
- **实现类/函数:**
 - `BaseVectorStore.upsert(records, trace: TraceContext | None = None)`
 - `BaseVectorStore.query(vector, top_k, filters, trace: TraceContext | None = None)`
- **验收标准:** 契约测试 (contract test) 约束输入输出 shape。
- **测试方法:** `pytest -q tests/unit/test_vector_store_contract.py`。

B5: Reranker 抽象接口与工厂 (含 None 回退)

- **目标:** 实现 `BaseReranker`、`RerankerFactory`, 提供 `NoneReranker` 作为默认回退。
- **修改文件:**
 - `src/libs/reranker/base_reranker.py`
 - `src/libs/reranker/reranker_factory.py`
 - `tests/unit/test_reranker_factory.py`
- **实现类/函数:**
 - `BaseReranker.rerank(query, candidates, trace: TraceContext | None = None) -> ranked_candidates`
 - `NoneReranker` (保持原顺序)
- **验收标准:** backend=None 时不会改变排序; 未知 backend 明确报错。
- **测试方法:** `pytest -q tests/unit/test_reranker_factory.py`。

B6: Evaluator 抽象接口与工厂（先做自定义轻量指标）

- **目标:** 定义 `BaseEvaluator`、`EvaluatorFactory`，实现最小 `CustomEvaluator`（例如 `hit_rate/mrr`）。
- **修改文件:**
 - `src/libs/evaluator/base_evaluator.py`
 - `src/libs/evaluator/evaluator_factory.py`
 - `src/libs/evaluator/custom_evaluator.py`
 - `tests/unit/test_custom_evaluator.py`
- **验收标准:** 输入 `query + retrieved_ids + golden_ids` 能输出稳定 metrics。
- **测试方法:** `pytest -q tests/unit/test_custom_evaluator.py`。

B7: 补齐 Libs 默认实现（拆分为≈1h可验收增量）

说明: B7 只补齐与端到端主链路强相关的默认实现 (LLM/Embedding/Splitter/VectorStore/Reranker)。其余可选扩展 (例如额外 splitter 策略、更多 vector store 后端、更多 evaluator 后端等) 保持原排期不提前。

B7.1: OpenAI-Compatible LLM (OpenAI/Azure/DeepSeek)

- **目标:** 补齐 OpenAI-compatible 的 LLM 实现，确保通过 `LLMFactory` 可创建并可被 mock 测试。
- **修改文件:**
 - `src/libs/llm/openai_llm.py`
 - `src/libs/llm/azure_llm.py`
 - `src/libs/llm/deepseek_llm.py`
 - `tests/unit/test_llm_providers_smoke.py` (mock HTTP, 不走真实网络)
- **验收标准:**
 - 配置不同 `provider` 时工厂路由正确。
 - `chat(messages)` 对输入 shape 校验清晰，异常信息可读 (包含 `provider` 与错误类型)。
- **测试方法:** `pytest -q tests/unit/test_llm_providers_smoke.py`。

B7.2: Ollama LLM (本地后端)

- **目标:** 补齐 `ollama_llm.py`，支持本地 HTTP endpoint (默认 `base_url + model`)，并可被 mock 测试。
- **修改文件:**
 - `src/libs/llm/ollama_llm.py`
 - `tests/unit/test_ollama_llm.py` (mock HTTP)
- **验收标准:**
 - `provider=ollama` 时可由 `LLMFactory` 创建。
 - 在连接失败/超时等场景下，抛出可读错误且不泄露敏感配置。
- **测试方法:** `pytest -q tests/unit/test_ollama_llm.py`。

B7.3: OpenAI & Azure Embedding 实现

- **目标:** 补齐 `openai_embedding.py` 和 `azure_embedding.py`, 支持 OpenAI 官方 API 和 Azure OpenAI 服务的 Embedding 调用, 支持批量 `embed(texts)`, 并可被 mock 测试。
- **修改文件:**
 - `src/libs/embedding/openai_embedding.py`
 - `src/libs/embedding/azure_embedding.py`
 - `tests/unit/test_embedding_providers_smoke.py` (mock HTTP, 包含 OpenAI 和 Azure 测试用例)
- **验收标准:**
 - `provider=openai` 时 `EmbeddingFactory` 可创建, 支持 OpenAI 官方 API 的 `text-embedding-3-small/large` 等模型。
 - `provider=azure` 时 `EmbeddingFactory` 可创建, 正确处理 Azure 特有的 `endpoint`、`api-version`、`api-key` 配置, 支持 Azure 部署的 `text-embedding-ada-002` 等模型。
 - 空输入、超长输入有明确行为 (报错或截断策略由配置决定)。
 - Azure 实现复用 OpenAI Embedding 的核心逻辑, 保持行为一致性。
- **测试方法:** `pytest -q tests/unit/test_embedding_providers_smoke.py`。

B7.4: Ollama Embedding 实现

- **目标:** 补齐 `ollama_embedding.py`, 支持通过 Ollama HTTP API 调用本地部署的 Embedding 模型 (如 `nomi-embed-text`、`mxbai-embed-large` 等), 实现 `embed(texts)` 批量向量化功能。
- **修改文件:**
 - `src/libs/embedding/ollama_embedding.py`
 - `tests/unit/test_ollama_embedding.py` (包含 mock HTTP 测试)
- **验收标准:**
 - `provider=ollama` 时 `EmbeddingFactory` 可创建。
 - 支持配置 Ollama 服务地址 (默认 <http://localhost:11434>) 和模型名称。
 - 输出向量维度由模型决定 (如 `nomi-embed-text` 为 768 维), 满足 `ingestion/retrieval` 的接口契约。
 - 支持批量 `embed(texts)` 调用, 内部处理单条/批量请求逻辑。
 - 空输入、超长输入有明确行为 (报错或截断策略)。
 - mock 测试覆盖正常响应、连接失败、超时等场景。
- **测试方法:** `pytest -q tests/unit/test_ollama_embedding.py`。

B7.5: Recursive Splitter 默认实现

- **目标:** 补齐 `recursive_splitter.py`, 封装 LangChain 的切分逻辑, 作为默认切分器。
- **修改文件:**
 - `src/libs/splitter/recursive_splitter.py`
 - `tests/unit/test_recursive_splitter_lib.py`

- **验收标准:**
 - `provider=recursive` 时 `SplitterFactory` 可创建。
 - `split_text` 能正确处理 Markdown 结构（标题/代码块不被打断）。
- **测试方法:** `pytest -q tests/unit/test_recursive_splitter_lib.py`。

B7.6: ChromaStore (VectorStore 默认后端)

- **目标:** 补齐 `chroma_store.py`，支持最小 `upsert(records)` 与 `query(vector, top_k, filters)`，并支持本地持久化目录（例如 `data/db/chroma/`）。
- **修改文件:**
 - `src/libs/vector_store/chroma_store.py`
 - `tests/integration/test_chroma_store_roundtrip.py`
- **验收标准:**
 - `provider=chroma` 时 `VectorStoreFactory` 可创建。
 - **必须完成完整的 upsert→query roundtrip 测试:** 使用 mock 数据完成真实的存储和检索流程，验证返回结果的确定性和正确性。
 - 测试应覆盖: 基本 upsert、向量查询、top_k 参数、metadata filters（如支持）。
 - 使用临时目录进行持久化测试，测试结束后清理。
- **测试方法:** `pytest -q tests/integration/test_chroma_store_roundtrip.py`

B7.7: LLM Reranker (读取 rerank prompt)

- **目标:** 补齐 `llm_reranker.py`，读取 `config/prompts/rerank.txt` 构造 prompt（测试中可注入替代文本），并可在失败时返回可回退信号。
- **修改文件:**
 - `src/libs/reranker/llm_reranker.py`
 - `tests/unit/test_llm_reranker.py` (mock LLM)
- **验收标准:**
 - `backend=llm` 时 `RerankerFactory` 可创建。
 - 输出严格结构化（例如 ranked ids），不满足 schema 时抛出可读错误。
- **测试方法:** `pytest -q tests/unit/test_llm_reranker.py`。

B7.8: Cross-Encoder Reranker (本地/托管模型，占位可跑)

- **目标:** 补齐 `cross_encoder_reranker.py`，支持对 Top-M candidates 打分排序；测试中用 mock scorer 保证 deterministic。
- **修改文件:**
 - `src/libs/reranker/cross_encoder_reranker.py`
 - `tests/unit/test_cross_encoder_reranker.py` (mock scorer)
- **验收标准:**
 - `backend=cross_encoder` 时 `RerankerFactory` 可创建。

- 提供超时/失败回退信号（供 Core 层 D6 fallback 使用）。
- **测试方法：** `pytest -q tests/unit/test_cross_encoder_reranker.py`。

B8: Vision LLM 抽象接口与工厂集成

- **目标：** 定义 `BaseVisionLLM` 抽象接口，扩展 `LLMFactory` 支持 Vision LLM 创建，为 C7 的 ImageCaptioner 提供底层抽象。
- **修改文件：**
 - `src/libs/llm/base_vision_llm.py`
 - `src/libs/llm/llm_factory.py`（扩展 `create_vision_llm` 方法）
 - `tests/unit/test_vision_llm_factory.py`
- **实现类/函数：**
 - `BaseVisionLLM.chat_with_image(text: str, image_path: str | bytes, trace: TraceContext | None = None) -> ChatResponse`
 - `LLMFactory.create_vision_llm(settings) -> BaseVisionLLM`
- **验收标准：**
 - 抽象接口清晰定义多模态输入（文本+图片路径/base64）。
 - 工厂方法 `create_vision_llm` 能根据配置路由到不同 provider（测试中用 Fake Vision LLM 验证）。
 - 接口设计支持图片预处理（压缩、格式转换）的扩展点。
- **测试方法：** `pytest -q tests/unit/test_vision_llm_factory.py`。

B9: Azure Vision LLM 实现

- **目标：** 实现 `AzureVisionLLM`，支持通过 Azure OpenAI 调用 GPT-4o/GPT-4-Vision-Preview 进行图像理解。
 - **修改文件：**
 - `src/libs/llm/azure_vision_llm.py`
 - `tests/unit/test_azure_vision_llm.py`（mock HTTP，不走真实 API）
 - **实现类/函数：**
 - `AzureVisionLLM(BaseVisionLLM)`：实现 `chat_with_image` 方法
 - 支持 Azure 特有配置：`azure_endpoint`, `api_version`, `deployment_name`, `api_key`
 - **验收标准：**
 - `provider=azure` 且配置 `vision_llm` 时，`LLMFactory.create_vision_llm()` 可创建 Azure Vision LLM 实例。
 - 支持图片路径和 base64 两种输入方式。
 - 图片过大时自动压缩至 `max_image_size` 配置的尺寸（默认2048px）。
 - API 调用失败时抛出清晰错误，包含 Azure 特有错误码。
 - mock 测试覆盖：正常调用、图片压缩、超时、认证失败等场景。
 - **测试方法：** `pytest -q tests/unit/test_azure_vision_llm.py`。
-

阶段 C: Ingestion Pipeline MVP (目标: 能把 PDF 样例摄取到本地存储)

注: 本阶段严格按 5.4.1 的离线数据流落地, 并优先实现“增量跳过 (SHA256) ”。

C1: 定义核心数据类型/契约 (Document/Chunk/ChunkRecord)

- **目标:** 定义全链路 (ingestion → retrieval → mcp tools) 共用的数据结构/契约, 避免散落在各子模块内导致的耦合与重复。
- **修改文件:**
 - `src/core/types.py`
 - `src/core/__init__.py` (可选: 统一 re-export 以简化导入路径)
 - `tests/unit/test_core_types.py`
- **实现类/函数 (建议):**
 - `Document(id, text, metadata)`
 - `Chunk(id, text, metadata, start_offset, end_offset, source_ref?)`
 - `ChunkRecord(id, text, metadata, dense_vector?, sparse_vector?)` (用于存储/检索载体; 字段按后续 C8~C12 演进)
- **验收标准:**
 - 类型可序列化 (dict/json) 且字段稳定 (单元测试断言)。
 - `metadata` 约定最少包含 `source_path`, 其余字段允许增量扩展但不得破坏兼容。
 - `metadata.images` **字段规范** (用于多模态支持):
 - 结构: `List[{"id": str, "path": str, "page": int, "text_offset": int, "text_length": int, "position": dict}]`
 - `id`: 全局唯一图片标识符 (建议格式: `{doc_hash}_{page}_{seq}`)
 - `path`: 图片文件存储路径 (约定: `data/images/{collection}/{image_id}.png`)
 - `page`: 图片在原文档中的页码 (可选, 适用于PDF等分页文档)
 - `text_offset`: 占位符在 `Document.text` 中的起始字符位置 (从0开始计数)
 - `text_length`: 占位符的字符长度 (通常为 `len("[IMAGE: {image_id}"])`)
 - `position`: 图片在原文档中的物理位置信息 (可选, 如PDF坐标、像素位置、尺寸等)
 - 说明: 通过 `text_offset` 和 `text_length` 可精确定位图片在文本中的位置, 支持同一图片多次出现的场景
 - **文本中图片占位符规范:** 在 `Document.text` 中, 图片位置使用 `[IMAGE: {image_id}]` 格式标记。
- **测试方法:** `pytest -q tests/unit/test_core_types.py`。

C2: 文件完整性检查 (SHA256)

- **目标:** 在Libs中实现 `file_integrity.py`: 计算文件 hash, 并提供“是否跳过”的判定接口 (使用 SQLite 作为默认存储, 支持后续替换为 Redis/PostgreSQL)。
- **修改文件:**
 - `src/libs/loader/file_integrity.py`
 - `tests/unit/test_file_integrity.py`
 - 数据库文件: `data/db/ingestion_history.db` (自动创建)
- **实现类/函数:**
 - `FileIntegrityChecker` 类 (抽象接口)
 - `SQLiteIntegrityChecker(FileIntegrityChecker)` 类 (默认实现)
 - `compute_sha256(path: str) -> str`
 - `should_skip(file_hash: str) -> bool`
 - `mark_success(file_hash: str, file_path: str, ...)`
 - `mark_failed(file_hash: str, error_msg: str)`
- **验收标准:**
 - 同一文件多次计算hash结果一致
 - 标记 success 后, `should_skip` 返回 `True`
 - 数据库文件正确创建在 `data/db/ingestion_history.db`
 - 支持并发写入 (SQLite WAL模式)
- **测试方法:** `pytest -q tests/unit/test_file_integrity.py`。

C3: Loader 抽象基类与 PDF Loader 壳子

- **目标:** 在Libs中定义 `BaseLoader`, 并实现 `PdfLoader` 的最小行为。
- **修改文件:**
 - `src/libs/loader/base_loader.py`
 - `src/libs/loader/pdf_loader.py`
 - `tests/unit/test_loader_pdf_contract.py`
- **实现类/函数:**
 - `BaseLoader.load(path) -> Document`
 - `PdfLoader.load(path)`
- **验收标准:**
 - **基础要求:** 对 sample PDF (fixtures) 能产出 Document, metadata 至少含 `source_path`。
 - **图片处理要求** (遵循 C1 定义的契约):
 - 若 PDF 包含图片, 应提取图片并保存到 `data/images/{doc_hash}/` 目录
 - 在 `Document.text` 中, 图片位置插入占位符: `[IMAGE: {image_id}]`
 - 在 `metadata.images` 中记录图片信息 (格式见 C1 规范)

- 若 PDF 无图片, `metadata.images` 可为空列表或省略该字段
 - **降级行为**: 图片提取失败不应阻塞文本解析, 可在日志中记录警告。
- **测试方法**: `pytest -q tests/unit/test_loader_pdf_contract.py`。
- **测试建议**:
 - 准备两个测试文件: `simple.pdf` (纯文本) 和 `with_images.pdf` (包含图片)
 - 验证纯文本PDF能正常解析
 - 验证带图片PDF能提取图片并正确插入占位符

C4: Splitter 集成 (调用 Libs)

- **目标**: 实现 Chunking 模块作为 `libs.splitter` 和 Ingestion Pipeline 之间的**适配器层**, 完成 Document→Chunks 的业务对象转换。
- **核心职责 (DocumentChunker 相比 `libs.splitter` 的增值)** :
 - **职责边界说明**:
 - `libs.splitter`: 纯文本切分工具 (`str → List[str]`), 不涉及业务对象
 - `DocumentChunker`: 业务适配器 (`Document对象 → List[Chunk对象]`), 添加业务逻辑
 - **6 个增值功能**:
 1. **Chunk ID 生成**: 为每个文本片段生成唯一且确定性的 ID (格式: `{doc_id}_{index:04d}_{hash_8chars}`)
 2. **元数据继承**: 将 `Document.metadata` 复制到每个 `Chunk.metadata` (`source_path`, `doc_type`, `title` 等)
 3. **添加 chunk_index**: 记录 chunk 在文档中的序号 (从 0 开始), 用于排序和定位
 4. **建立 source_ref**: 记录 `Chunk.source_ref` 指向父 `Document.id`, 支持溯源
 5. **图片引用按需分发**: 扫描每个 chunk 文本中的 `[IMAGE: {id}]` 占位符, 从 `Document.metadata["images"]` 中提取该 chunk 实际引用的 `ImageRef`, 写入 `chunk.metadata["images"]` (仅含该 chunk 引用的子集) 和 `chunk.metadata["image_refs"]` (`image_id` 列表)。无占位符的 chunk 不含 `images` 字段。⚠ 不可简单整体继承或丢弃文档级 `images`, 否则下游 C7 ImageCaptioner 将无法定位图片路径。
 6. **类型转换**: 将 `libs.splitter` 的 `List[str]` 转换为符合 `core.types` 契约的 `List[Chunk]` 对象
- **修改文件**:
 - `src/ingestion/chunking/document_chunker.py`
 - `src/ingestion/chunking/__init__.py`
 - `tests/unit/test_document_chunker.py`
- **实现类/函数**:
 - `DocumentChunker` 类
 - `__init__(settings: Settings)`: 通过 `SplitterFactory` 获取配置的 splitter 实例
 - `split_document(document: Document) -> List[Chunk]`: 完整的转换流程
 - `_generate_chunk_id(doc_id: str, index: int, text: str) -> str`: 生成稳定 Chunk ID

- `_inherit_metadata(document: Document, chunk_index: int, chunk_text: str) -> dict:`
元数据继承 + 图片引用按需分发逻辑 (需要 `chunk_text` 来扫描 `[IMAGE: id]` 占位符)
- **验收标准:**
 - **配置驱动:** 通过修改 `settings.yaml` 中的 `splitter` 配置 (如 `chunk_size`) , 产出的 `chunk` 数量和长度发生相应变化
 - **ID 唯一性:** 每个 `Chunk` 的 `ID` 在整个文档中唯一
 - **ID 确定性:** 同一 `Document` 对象重复切分产生相同的 `Chunk ID` 序列
 - **元数据完整性:** `Chunk.metadata` 包含所有 `Document.metadata` 字段 + `chunk_index` 字段
 - **图片分发正确性:** 含 `[IMAGE: id]` 占位符的 `chunk` 其 `metadata["images"]` 仅包含该 `chunk` 引用的图片子集; 不含占位符的 `chunk` 无 `images` 字段; `metadata["image_refs"]` 列表与占位符一致
 - **溯源链接:** 所有 `Chunk.source_ref` 正确指向父 `Document.id`
 - **类型契约:** 输出的 `Chunk` 对象符合 `core/types.py` 中的 `Chunk` 定义 (可序列化、字段完整)
- **测试方法:** `pytest -q tests/unit/test_document_chunker.py` (使用 `FakeSplitter` 隔离测试, 无需真实 LLM/外部依赖) 。

C5: Transform 抽象基类 + ChunkRefiner (规则去噪 + LLM 增强)

- **目标:** 定义 `BaseTransform`; 实现 `ChunkRefiner`: 先做规则去噪, 再通过 LLM 进行智能增强, 并提供失败降级机制 (LLM 异常时回退到规则结果, 不阻塞 ingestion) 。
- **前置条件** (必须准备) :
 - **必须配置 LLM:** 在 `config/settings.yaml` 中配置可用的 LLM (`provider/model/api_key`)
 - **环境变量:** 设置对应的 API key 环境变量 (`OPENAI_API_KEY` / `OLLAMA_BASE_URL` 等)
 - **验证目的:** 通过真实 LLM 测试验证配置正确性和 refinement 效果
- **修改文件:**
 - `src/ingestion/transform/base_transform.py` (新增)
 - `src/ingestion/transform/chunk_refiner.py` (新增)
 - `src/core/trace/trace_context.py` (新增: 最小实现, Phase F 完善)
 - `config/prompts/chunk_refinement.txt` (已存在, 需验证内容并补充 `{text}` 占位符)
 - `tests/fixtures/noisy_chunks.json` (新增: 8 个典型噪声场景)
 - `tests/unit/test_chunk_refiner.py` (新增: 27 个单元测试)
 - `tests/integration/test_chunk_refiner_llm.py` (新增: 真实 LLM 集成测试)
- **实现类/函数:**
 - `BaseTransform.transform(chunks, trace) -> List[Chunk]`
 - `ChunkRefiner.__init__(settings, llm?, prompt_path?)`
 - `ChunkRefiner.transform(chunks, trace) -> List[Chunk]`
 - `ChunkRefiner._rule_based_refine(text) -> str` (去空白/页眉页脚/格式标记/HTML 注释)
 - `ChunkRefiner._llm_refine(text, trace) -> str | None` (可选 LLM 重写, 失败返回 None)
 - `ChunkRefiner._load_prompt(prompt_path?)` (从文件加载 prompt 模板, 支持默认 fallback)

- **实现流程建议：**

1. 先创建 `tests/fixtures/noisy_chunks.json`，包含8个典型噪声场景：
 - `typical_noise_scenario`: 综合噪声（页眉/页脚/空白）
 - `ocr_errors`: OCR错误文本
 - `page_header_footer`: 页眉页脚模式
 - `excessive_whitespace`: 多余空白
 - `format_markers`: HTML/Markdown标记
 - `clean_text`: 干净文本（验证不过度清理）
 - `code_blocks`: 代码块（验证保留内部格式）
 - `mixed_noise`: 真实混合场景
2. 创建 `TraceContext` 占位实现（uuid生成`trace_id`，`record_stage`存储阶段数据）
3. 实现 `BaseTransform` 抽象接口
4. 实现 `ChunkRefiner._rule_based_refine` 规则去噪逻辑（正则匹配+分段处理）
5. 编写规则模式单元测试（使用 `fixtures` 断言清洗效果）
6. 实现 `_llm_refine` 可选增强（读取 `prompt`、调用 LLM、错误处理）
7. 编写 LLM 模式单元测试（mock LLM 断言调用与输出）
8. 编写降级场景测试（LLM 失败时回退到规则结果，标记 `metadata`）
9. **编写真实LLM集成测试并执行验证**（必须执行，验证LLM配置）

- **验收标准：**

- **单元测试（快速反馈循环）：**

- 规则模式：对 `fixtures` 噪声样例能正确去噪（连续空白/页眉页脚/格式标记/分隔线）
- 保留能力：代码块内部格式不被破坏，Markdown结构完整保留
- LLM 模式：mock LLM 时能正确调用并返回重写结果，`metadata` 标记 `refined_by: "llm"`
- 降级行为：LLM 失败时回退到规则结果，`metadata` 标记 `refined_by: "rule"` 和 `fallback` 原因
- 配置开关：通过 `settings.yaml` 的 `ingestion.chunk_refiner.use_llm` 控制行为
- 异常处理：单个chunk处理异常不影响其他chunk，保留原文

- **集成测试（验收必须项）：**

-  **必须验证真实LLM调用成功**：使用前置条件中配置的LLM进行真实refinement
-  **必须验证输出质量**：LLM refined文本确实更干净（噪声减少、内容保留）
-  **必须验证降级机制**：无效模型名称时优雅降级到rule-based，不崩溃
- 说明：这是验证"前置条件中准备的LLM配置是否正确"的必要步骤

- **测试方法：**

- **阶段1-单元测试（开发中快速迭代）：**

```
pytest tests/unit/test_chunk_refiner.py -v
#  27个测试全部通过，使用Mock隔离，无需真实API
```

- 阶段2-集成测试（验收必须执行）：

```
# 1. 运行真实LLM集成测试（必须）
pytest tests/integration/test_chunk_refiner_llm.py -v -s
# ✅ 验证LLM配置正确，refinement效果符合预期
# ⚠️ 会产生真实API调用与费用

# 2. Review打印输出，确认精炼质量
# - 噪声是否被有效去除？
# - 有效内容是否完整保留？
# - 降级机制是否正常工作？
```

- 测试分层逻辑：

- 单元测试：验证代码逻辑正确
- 集成测试：验证系统可用性
- 两者互补，缺一不可

C6: MetadataEnricher（规则增强 + 可选 LLM 增强 + 降级）

- **目标：**实现元数据增强模块：提供规则增强的默认实现，并重点支持 LLM 增强（配置已就绪，LLM 开关打开）。利用 LLM 对 chunk 进行高质量的 title 生成、summary 摘要和 tags 提取。同时保留失败降级机制，确保不阻塞 ingestion。
- **修改文件：**
 - `src/ingestion/transform/metadata_enricher.py`
 - `tests/unit/test_metadata_enricher_contract.py`
- **验收标准：**
 - 规则模式：作为兜底逻辑，输出 metadata 必须包含 `title/summary/tags`（至少非空）。
 - **LLM 模式（核心）：**在 LLM 打开的情况下，确保真实调用 LLM（或高质量 Mock）并生成语义丰富的 metadata。需验证在有真实 LLM 配置下的连通性与效果。
 - 降级行为：LLM 调用失败时回退到规则模式结果（可在 metadata 标记降级原因，但不抛出致命异常）。
- **测试方法：**`pytest -q tests/unit/test_metadata_enricher_contract.py`，并确保包含开启 LLM 的集成测试用例。

C7: ImageCaptioner（可选生成 caption + 降级不阻塞）

- **目标：**实现 `image_captioner.py`：当启用 Vision LLM 且存在 `image_refs` 时生成 caption 并写回 chunk metadata；当禁用/不可用/异常时走降级路径，不阻塞 ingestion。
- **修改文件：**
 - `src/ingestion/transform/image_captioner.py`
 - `config/prompts/image_captioning.txt`（作为默认 prompt 来源；可在测试中注入替代文本）
 - `tests/unit/test_image_captioner_fallback.py`
- **验收标准：**
 - 启用模式：存在 `image_refs` 时会生成 caption 并写入 metadata（测试中用 mock Vision LLM 断言调

- 用与输出)。
- 降级模式：当配置禁用或异常时，chunk 保留 image_refs，但不生成 caption 且标记 `has_unprocessed_images`。
- 测试方法：`pytest -q tests/unit/test_image_captioner_fallback.py`。

C8: DenseEncoder (依赖 `libs.embedding`)

- 目标：实现 `dense_encoder.py`，把 `chunks.text` 批量送入 `BaseEmbedding`。
- 修改文件：
 - `src/ingestion/embedding/dense_encoder.py`
 - `tests/unit/test_dense_encoder.py`
- 验收标准：encoder 输出向量数量与 `chunks` 数量一致，维度一致。
- 测试方法：`pytest -q tests/unit/test_dense_encoder.py`。

C9: SparseEncoder (BM25 统计与输出契约)

- 目标：实现 `sparse_encoder.py`：对 `chunks` 建立 BM25 所需统计（可先仅输出 term weights 结构，索引落地下一步做）。
- 修改文件：
 - `src/ingestion/embedding/sparse_encoder.py`
 - `tests/unit/test_sparse_encoder.py`
- 验收标准：输出结构可用于 `bm25_indexer`；对空文本有明确行为。
- 测试方法：`pytest -q tests/unit/test_sparse_encoder.py`。

C10: BatchProcessor (批处理编排)

- 目标：实现 `batch_processor.py`：将 `chunks` 分 batch，驱动 dense/sparse 编码，记录批次耗时（为 trace 预留）。
- 修改文件：
 - `src/ingestion/embedding/batch_processor.py`
 - `tests/unit/test_batch_processor.py`
- 验收标准：`batch_size=2` 时对 5 `chunks` 分成 3 批，且顺序稳定。
- 测试方法：`pytest -q tests/unit/test_batch_processor.py`。

—— 存储阶段分界线：以下任务负责将编码结果持久化 ——

说明：C8-C10完成了Dense和Sparse的编码工作，C11-C13负责将编码结果存储到不同的后端。

- C11 (BM25Indexer)：处理Sparse编码结果 → 构建倒排索引 → 存储到文件系统
 - C12 (VectorUpserter)：处理Dense编码结果 → 生成稳定ID → 存储到向量数据库
 - C13 (ImageStorage)：处理图片数据 → 文件存储 + 索引映射
-

C11: BM25Indexer (倒排索引构建与持久化)

- **目标:** 实现 `bm25_indexer.py`: 接收 `SparseEncoder` 的 term statistics 输出, 计算 IDF, 构建倒排索引, 并持久化到 `data/db/bm25/`。
- **核心功能:**
 - 计算 IDF (Inverse Document Frequency): $IDF(term) = \log((N - df + 0.5) / (df + 0.5))$
 - 构建倒排索引结构: `{term: {idf, postings: [{chunk_id, tf, doc_length}]}}`
 - 索引序列化与加载 (支持增量更新与重建)
- **修改文件:**
 - `src/ingestion/storage/bm25_indexer.py`
 - `tests/unit/test_bm25_indexer_roundtrip.py`
- **验收标准:**
 - build 后能 load 并对同一语料查询返回稳定 top ids
 - IDF 计算准确 (可用已知语料对比验证)
 - 支持索引重建与增量更新
- **测试方法:** `pytest -q tests/unit/test_bm25_indexer_roundtrip.py`。
- **备注:** 本任务完成 Sparse 路径的最后一环, 为 D3 (`SparseRetriever`) 提供可查询的 BM25 索引。

C12: VectorUpserter (向量存储与幂等性保证)

- **目标:** 实现 `vector_upserter.py`: 接收 `DenseEncoder` 的向量输出, 生成稳定的 `chunk_id`, 并调用 `VectorStore` 进行幂等写入。
- **核心功能:**
 - 生成确定性 `chunk_id`: `hash(source_path + chunk_index + content_hash[:8])`
 - 调用 `BaseVectorStore.upsert()` 写入向量数据库
 - 保证幂等性: 同一内容重复写入不产生重复记录
- **修改文件:**
 - `src/ingestion/storage/vector_upserter.py`
 - `tests/unit/test_vector_upserter_idempotency.py`
- **验收标准:**
 - 同一 chunk 两次 upsert 产生相同 id
 - 内容变更时 id 变更
 - 支持批量 upsert 且保持顺序
- **测试方法:** `pytest -q tests/unit/test_vector_upserter_idempotency.py`。
- **备注:** 本任务完成 Dense 路径的最后一环, 为 D2 (`DenseRetriever`) 提供可查询的向量数据库。

C13: ImageStorage (图片文件存储与索引表契约)

- **目标:** 实现 `image_storage.py`: 保存图片到 `data/images/{collection}/`, 并使用 **SQLite** 记录 `image_id`→`path` 映射。
- **修改文件:**
 - `src/ingestion/storage/image_storage.py`
 - `tests/unit/test_image_storage.py`
- **验收标准:** 保存后文件存在; 查找 `image_id` 返回正确路径; 映射关系持久化在 `data/db/image_index.db`。
- **技术方案:**
 - 复用项目已有的 SQLite 架构模式 (参考 `file_integrity.py` 的 `SQLiteIntegrityChecker`)
 - 数据库表结构:

```
CREATE TABLE image_index (  
    image_id TEXT PRIMARY KEY,  
    file_path TEXT NOT NULL,  
    collection TEXT,  
    doc_hash TEXT,  
    page_num INTEGER,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
CREATE INDEX idx_collection ON image_index(collection);  
CREATE INDEX idx_doc_hash ON image_index(doc_hash);
```

- 提供并发安全访问 (WAL 模式)
 - 支持按 `collection` 批量查询
- **测试方法:** `pytest -q tests/unit/test_image_storage.py`。

C14: Pipeline 编排 (MVP 串起来)

- **目标:** 实现 `pipeline.py`: 串行执行 (`integrity`→`load`→`split`→`transform`→`encode`→`store`), 并对失败步骤做清晰异常。
- **修改文件:**
 - `src/ingestion/pipeline.py`
 - `tests/integration/test_ingestion_pipeline.py`
- **测试数据:**
 - **主测试文档:** `tests/fixtures/sample_documents/complex_technical_doc.pdf`
 - 8章节技术文档 (~21KB)
 - 包含3张嵌入图片 (需测试图片提取和描述)
 - 包含5个表格 (测试表格内容解析)
 - 多页多段落 (测试完整分块流程)
 - **辅助测试:** `tests/fixtures/sample_documents/simple.pdf` (简单场景回归)
- **验收标准:**

- 对 `complex_technical_doc.pdf` 跑完整 pipeline, 成功输出:
 - 向量索引文件到 ChromaDB
 - BM25 索引文件到 `data/db/bm25/`
 - 提取的图片到 `data/images/` (SHA256命名)
- Pipeline 日志清晰展示各阶段进度
- 失败步骤抛出明确异常信息
- 测试方法: `pytest -v tests/integration/test_ingestion_pipeline.py`。

C15: 脚本入口 `ingest.py` (离线可用)

- 目标: 实现 `scripts/ingest.py`, 支持 `--collection`、`--path`、`--force`, 并调用 pipeline。
- 修改文件:
 - `scripts/ingest.py`
 - `tests/e2e/test_data_ingestion.py`
- 验收标准: 命令行可运行并在 `data/db` 产生产物; 重复运行在未变更时跳过。
- 测试方法: `pytest -q tests/e2e/test_data_ingestion.py` (尽量用临时目录)。

阶段 D: Retrieval MVP (目标: 能 query 并返回 Top-K chunks)

D1: QueryProcessor (关键词提取 + filters 结构)

- 目标: 实现 `query_processor.py`: 关键词提取 (先规则/分词), 并解析通用 filters 结构 (可空实现)。
- 修改文件:
 - `src/core/query_engine/query_processor.py`
 - `tests/unit/test_query_processor.py`
- 验收标准: 对输入 query 输出 `keywords` 非空 (可根据停用词策略), filters 为 dict。
- 测试方法: `pytest -q tests/unit/test_query_processor.py`。

D2: DenseRetriever (调用 `VectorStore.query`)

- 目标: 实现 `dense_retriever.py`, 组合 `EmbeddingClient` (query 向量化) + `VectorStore` (向量检索), 完成语义召回。
- 前置任务:
 1. 需先在 `src/core/types.py` 中定义 `RetrievalResult` 类型 (包含 `chunk_id`, `score`, `text`, `metadata` 字段)
 2. 需确认 `ChromaStore.query()` 返回结果包含 `text` (当前存储在 `documents` 字段, 需补充返回)
- 修改文件:
 - `src/core/types.py` (新增 `RetrievalResult` 类型)
 - `src/libs/vector_store/chroma_store.py` (修复: query 返回结果需包含 `text` 字段)

- `src/core/query_engine/dense_retriever.py`
- `tests/unit/test_dense_retriever.py`
- 实现类/函数：
 - `RetrievalResult` dataclass: `chunk_id: str, score: float, text: str, metadata: Dict`
 - `DenseRetriever.__init__(settings, embedding_client?, vector_store?)`: 支持依赖注入用于测试
 - `DenseRetriever.retrieve(query: str, top_k: int, filters?: dict, trace?) -> List[RetrievalResult]`
 - 内部流程: `query` → `embedding_client.embed([query])` → `vector_store.query(vector, top_k, filters)` → 从返回结果提取 `text` → 规范化结果
- 验收标准：
 - `RetrievalResult` 类型已定义并可序列化
 - `ChromaStore.query()` 返回结果包含 `text` 字段
 - 对输入 `query` 能生成 `embedding` 并调用 `VectorStore` 检索
 - 返回结果包含 `chunk_id`、`score`、`text`、`metadata`
 - mock `EmbeddingClient` 和 `VectorStore` 时能正确编排调用
- 测试方法: `pytest -q tests/unit/test_dense_retriever.py` (mock embedding + vector store)。

D3: SparseRetriever (BM25 查询)

- 目标: 实现 `sparse_retriever.py`: 从 `data/db/bm25/` 载入索引并查询。
- 前置任务: 需在 `BaseVectorStore` 和 `ChromaStore` 中添加 `get_by_ids()` 方法, 用于根据 `chunk_id` 批量获取 `text` 和 `metadata`
- 修改文件：
 - `src/libs/vector_store/base_vector_store.py` (新增 `get_by_ids()` 抽象方法)
 - `src/libs/vector_store/chroma_store.py` (实现 `get_by_ids()` 方法)
 - `src/core/query_engine/sparse_retriever.py`
 - `tests/unit/test_sparse_retriever.py`
- 实现类/函数：
 - `BaseVectorStore.get_by_ids(ids: List[str]) -> List[Dict]`: 根据 ID 批量获取记录
 - `ChromaStore.get_by_ids(ids: List[str]) -> List[Dict]`: 调用 ChromaDB 的 `get` 方法
 - `SparseRetriever.__init__(settings, bm25_indexer?, vector_store?)`: 支持依赖注入用于测试
 - `SparseRetriever.retrieve(keywords: List[str], top_k: int, trace?) -> List[RetrievalResult]`
 - 内部流程：
 1. `keywords` → `bm25_indexer.query(keywords, top_k)` → `[{chunk_id, score}]`
 2. `chunk_ids` → `vector_store.get_by_ids(chunk_ids)` → `[{id, text, metadata}]`
 3. 合并 `score` 与 `text/metadata`, 组装为 `RetrievalResult` 列表

- 注意: keywords 来自 `QueryProcessor.process()` 的 `ProcessedQuery.keywords`
- 验收标准:
 - `BaseVectorStore.get_by_ids()` 和 `ChromaStore.get_by_ids()` 已实现
 - 对已构建索引的 fixtures 语料, 关键词检索命中预期 `chunk_id`
 - 返回结果包含完整的 text 和 metadata
- 测试方法: `pytest -q tests/unit/test_sparse_retriever.py`。

D4: Fusion (RRF 实现)

- 目标: 实现 `fusion.py`: RRF 融合 dense/sparse 排名并输出统一排序。
- 修改文件:
 - `src/core/query_engine/fusion.py`
 - `tests/unit/test_fusion_rrf.py`
- 验收标准: 对构造的排名输入输出 deterministic; k 参数可配置。
- 测试方法: `pytest -q tests/unit/test_fusion_rrf.py`。

D5: HybridSearch 编排

- 目标: 实现 `hybrid_search.py`: 编排 Dense + Sparse + Fusion 的完整混合检索流程, 并集成 Metadata 过滤逻辑。
- 前置依赖: D1 (QueryProcessor)、D2 (DenseRetriever)、D3 (SparseRetriever)、D4 (Fusion)
- 修改文件:
 - `src/core/query_engine/hybrid_search.py`
 - `tests/integration/test_hybrid_search.py`
- 实现类/函数:
 - `HybridSearch.__init__(settings, query_processor, dense_retriever, sparse_retriever, fusion)`
 - `HybridSearch.search(query: str, top_k: int, filters?: dict, trace?) -> List[RetrievalResult]`
 - `HybridSearch._apply_metadata_filters(candidates, filters) -> List[RetrievalResult]`: 后置过滤兜底
 - 内部流程: `query_processor.process()` → 并行(`dense.retrieve` + `sparse.retrieve`) → `fusion.fuse()` → `metadata_filter` → Top-K
- 验收标准:
 - 对 fixtures 数据, 能返回 Top-K (包含 chunk 文本与 metadata)
 - 支持 filters 参数 (如 `collection`、`doc_type`) 进行过滤
 - Dense/Sparse 任一路径失败时能降级到单路结果
- 测试方法: `pytest -q tests/integration/test_hybrid_search.py`。

D6: Reranker (Core 层编排 + fallback)

- **目标:** 实现 `core/query_engine/reranker.py`: 接入 `libs.reranker` 后端, 失败/超时回退 fusion 排名。
- **修改文件:**
 - `src/core/query_engine/reranker.py`
 - `config/prompts/rerank.txt` (仅当启用 LLM Rerank 后端时使用)
 - `tests/unit/test_reranker_fallback.py`
- **验收标准:** 模拟后端异常时不影响最终返回, 且标记 `fallback=true`。
- **测试方法:** `pytest -q tests/unit/test_reranker_fallback.py`。

D7: 脚本入口 query.py (查询可用)

- **目标:** 实现 `scripts/query.py`, 作为在线查询的命令行入口, 调用完整的 HybridSearch + Reranker 流程并输出检索结果。
- **前置依赖:** D5 (HybridSearch) 、 D6 (Reranker)
- **修改文件:**
 - `scripts/query.py`
- **实现功能:**
 - **参数支持:**
 - `--query "问题"`: 必填, 查询文本
 - `--top-k 10`: 可选, 返回结果数量 (默认 10)
 - `--collection xxx`: 可选, 限定检索集合
 - `--verbose`: 可选, 显示各阶段中间结果
 - `--no-rerank`: 可选, 跳过 Reranker 阶段
 - **输出内容:**
 - 默认模式: Top-K 结果 (序号、score、文本摘要、来源文件、页码)
 - Verbose 模式: 额外显示 Dense 召回结果、Sparse 召回结果、Fusion 结果、Rerank 结果
 - **内部流程:**
 1. 加载配置 `Settings`
 2. 初始化组件 (EmbeddingClient、VectorStore、BM25Indexer、Reranker)
 3. 创建 `QueryProcessor`、`DenseRetriever`、`SparseRetriever`、`HybridSearch` 实例
 4. 调用 `HybridSearch.search()` 获取候选结果
 5. 调用 `Reranker.rerank()` 进行精排 (除非 `--no-rerank`)
 6. 格式化输出结果
- **验收标准:**
 - 命令行可运行: `python scripts/query.py --query "如何配置 Azure? "`
 - 返回格式化的 Top-K 检索结果
 - `--verbose` 模式显示各阶段中间结果 (便于调试)

- 无数据时返回友好提示（如"未找到相关文档，请先运行 ingest.py 摄取数据"）
 - **测试方法：**手动运行 `python scripts/query.py --query "测试查询" --verbose`（依赖已摄取的数据）。
 - **与 MCP Tool 的关系：**
 - `scripts/query.py` 是开发调试用的命令行工具
 - `E3 query_knowledge_hub` 是生产环境的 MCP Tool
 - 两者共享 Core 层逻辑（HybridSearch + Reranker），但入口和输出格式不同
-

阶段 E：MCP Server 层与 Tools（目标：对外可用的 MCP tools）

E1：MCP Server 入口与 Stdio 约束

- **目标：**实现 `mcp_server/server.py`：遵循"stdout 只输出 MCP 消息，日志到 stderr"。
- **修改文件：**
 - `src/mcp_server/server.py`
 - `tests/integration/test_mcp_server.py`
- **验收标准：**启动 server 能完成 initialize；stderr 有日志但 stdout 不污染。
- **测试方法：**`pytest -q tests/integration/test_mcp_server.py`（子进程方式）。

E2：Protocol Handler 协议解析与能力协商

- **目标：**实现 `mcp_server/protocol_handler.py`：封装 JSON-RPC 2.0 协议解析，处理 `initialize`、`tools/list`、`tools/call` 三类核心方法，并实现规范的错误处理。
- **修改文件：**
 - `src/mcp_server/protocol_handler.py`
 - `tests/unit/test_protocol_handler.py`
- **实现要点：**
 - **ProtocolHandler 类：**
 - `handle_initialize(params)` → 返回 server capabilities（支持的 tools 列表、版本信息）
 - `handle_tools_list()` → 返回已注册的 tool schema（name, description, inputSchema）
 - `handle_tools_call(name, arguments)` → 路由到具体 tool 执行，捕获异常并转换为 JSON-RPC error
 - **错误码规范：**遵循 JSON-RPC 2.0（-32600 Invalid Request, -32601 Method not found, -32602 Invalid params, -32603 Internal error）
 - **能力协商：**在 `initialize` 响应中声明 `capabilities.tools`
- **验收标准：**
 - 发送 `initialize` 请求能返回正确的 `serverInfo` 和 `capabilities`
 - 发送 `tools/list` 能返回已注册 tools 的 schema
 - 发送 `tools/call` 能正确路由并返回结果或规范错误

- **错误处理**: 无效方法返回 -32601, 参数错误返回 -32602, 内部异常返回 -32603 且不泄露堆栈
- **测试方法**: `pytest -q tests/unit/test_protocol_handler.py`。

E3: 实现 tool: `query_knowledge_hub`

- **目标**: 实现 `tools/query_knowledge_hub.py`: 调用 HybridSearch + Reranker, 构建带引用的响应, 返回 Markdown + structured citations。
- **前置依赖**: D5 (HybridSearch)、D6 (Reranker)、E1 (Server)、E2 (Protocol Handler)
- **修改文件**:
 - `src/mcp_server/tools/query_knowledge_hub.py`
 - `src/core/response/response_builder.py` (新增: 构建 MCP 响应格式)
 - `src/core/response/citation_generator.py` (新增: 生成引用信息)
 - `tests/unit/test_response_builder.py` (新增)
 - `tests/integration/test_mcp_server.py` (补用例)
- **实现类/函数**:
 - `ResponseBuilder.build(retrieval_results, query) -> MCPResponse`: 构建 MCP 格式响应
 - `CitationGenerator.generate(retrieval_results) -> List[Citation]`: 生成引用列表
 - `query_knowledge_hub(query, top_k?, collection?) -> MCPToolResult`: Tool 入口函数
- **验收标准**:
 - tool 返回 `content[0]` 为可读 Markdown (含 [1]、[2] 等引用标注)
 - `structuredContent.citations` 包含 `source / page / chunk_id / score` 字段
 - 无结果时返回友好提示而非空数组
- **测试方法**: `pytest -q tests/integration/test_mcp_server.py -k query_knowledge_hub`。

E4: 实现 tool: `list_collections`

- **目标**: 实现 `tools/list_collections.py`: 列出 `data/documents/` 下集合并附带统计 (可延后到下一步)。
- **修改文件**:
 - `src/mcp_server/tools/list_collections.py`
 - `tests/unit/test_list_collections.py`
- **验收标准**: 对 fixtures 中的目录结构能返回集合名列表。
- **测试方法**: `pytest -q tests/unit/test_list_collections.py`。

E5: 实现 tool: `get_document_summary`

- **目标**: 实现 `tools/get_document_summary.py`: 按 doc_id 返回 title/summary/tags (可先从 metadata/缓存取)。
- **修改文件**:
 - `src/mcp_server/tools/get_document_summary.py`
 - `tests/unit/test_get_document_summary.py`

- **验收标准:** 对不存在 doc_id 返回规范错误; 存在时返回结构化信息。
- **测试方法:** `pytest -q tests/unit/test_get_document_summary.py`。

E6: 多模态返回组装 (Text + Image)

- **目标:** 实现 `multimodal_assembler.py`: 命中 chunk 含 image_refs 时读取图片并 base64 返回 ImageContent。
- **修改文件:**
 - `src/core/response/multimodal_assembler.py`
 - `tests/integration/test_mcp_server.py` (补图像返回用例)
- **验收标准:** 返回 content 中包含 image type, mimeType 正确, data 为 base64 字符串。
- **测试方法:** `pytest -q tests/integration/test_mcp_server.py -k image`。

阶段 F: Trace 基础设施与打点 (目标: Ingestion + Query 双链路可追踪)

F1: TraceContext 增强 (finish + 耗时统计 + trace_type)

- **目标:** 增强已有的 `TraceContext` (C5 已实现基础版), 添加 `finish()` 方法、耗时统计、`trace_type` 字段 (区分 query/ingestion)、`to_dict()` 序列化功能。
- **修改文件:**
 - `src/core/trace/trace_context.py` (增强: 添加 trace_type/finish/elapsed_ms/to_dict)
 - `src/core/trace/trace_collector.py` (新增: 收集并持久化 trace)
 - `tests/unit/test_trace_context.py` (补充 finish/to_dict 相关测试)
- **实现类/函数:**
 - `TraceContext.__init__(trace_type: str = "query")`: 支持 "query" 或 "ingestion" 类型
 - `TraceContext.finish() -> None`: 标记 trace 结束, 计算总耗时
 - `TraceContext.elapsed_ms(stage_name?) -> float`: 获取指定阶段或总耗时
 - `TraceContext.to_dict() -> dict`: 序列化为可 JSON 输出的字典 (含 trace_type)
 - `TraceCollector.collect(trace: TraceContext) -> None`: 收集 trace 并触发持久化
- **验收标准:**
 - `record_stage` 追加阶段数据 (已有)
 - `finish()` 后 `to_dict()` 输出包含 `trace_id`、`trace_type`、`started_at`、`finished_at`、`total_elapsed_ms`、`stages`
 - 输出 dict 可直接 `json.dumps()` 序列化
- **测试方法:** `pytest -q tests/unit/test_trace_context.py`。

F2: 结构化日志 logger (JSON Lines)

- **目标:** 增强 `observability/logger.py`, 支持 JSON Lines 格式输出, 并实现 trace 持久化到 `logs/traces.jsonl`。
- **修改文件:**
 - `src/observability/logger.py` (增强: 添加 JSONFormatter + FileHandler)
 - `tests/unit/test_jsonl_logger.py`
- **实现类/函数:**
 - `JSONFormatter`: 自定义 logging Formatter, 输出 JSON 格式
 - `get_trace_logger() -> logging.Logger`: 获取配置了 JSON Lines 输出的 logger
 - `write_trace(trace_dict: dict) -> None`: 将 trace 字典写入 `logs/traces.jsonl`
- **与 F1 的分工:**
 - F1 负责 TraceContext 的数据结构 (含 `trace_type`) 和 `finish()` 方法
 - F2 负责将 `trace.to_dict()` 的结果持久化到文件
- **验收标准:** 写入一条 trace 后文件新增一行合法 JSON, 包含 `trace_type` 字段。
- **测试方法:** `pytest -q tests/unit/test_jsonl_logger.py`。

F3: 在 Query 链路打点

- **目标:** 在 HybridSearch/Rerank 中注入 TraceContext (`trace_type="query"`), 利用 B 阶段抽象接口中预留的 `trace` 参数, 显式调用 `trace.record_stage()` 记录各阶段数据。
- **前置依赖:** D5 (HybridSearch)、D6 (Reranker)、F1 (TraceContext 增强)、F2 (结构化日志)
- **修改文件:**
 - `src/core/query_engine/hybrid_search.py` (增加 trace 记录: dense/sparse/fusion 阶段)
 - `src/core/query_engine/reranker.py` (增加 trace 记录: rerank 阶段)
 - `tests/integration/test_hybrid_search.py` (断言 trace 中存在各阶段)
- **说明:** B 阶段的接口已预留 `trace: TraceContext | None = None` 参数, 本任务负责在调用时传入实际的 TraceContext 实例, 并在各阶段记录 `method / provider / details` 字段。
- **验收标准:**
 - 一次查询生成 trace, 包含 `query_processing / dense_retrieval / sparse_retrieval / fusion / rerank` 阶段
 - 每个阶段记录 `elapsed_ms` 耗时字段和 `method` 字段
 - `trace.to_dict()` 中 `trace_type == "query"`
- **测试方法:** `pytest -q tests/integration/test_hybrid_search.py`。

F4: 在 Ingestion 链路打点

- **目标:** 在 IngestionPipeline 中注入 TraceContext (`trace_type="ingestion"`) , 记录各摄取阶段的处理数据。
- **前置依赖:** C5 (Pipeline) 、 F1 (TraceContext 增强) 、 F2 (结构化日志)
- **修改文件:**
 - `src/ingestion/pipeline.py` (增加 trace 传递: load/split/transform/embed/upsert 阶段)
 - `tests/integration/test_ingestion_pipeline.py` (断言 trace 中存在各阶段)
- **验收标准:**
 - 一次摄取生成 trace, 包含 `load/split/transform/embed/upsert` 阶段
 - 每个阶段记录 `elapsed_ms`、`method` (如 `markdown/recursive/chroma`) 和处理详情
 - `trace.to_dict()` 中 `trace_type == "ingestion"`
- **测试方法:** `pytest -q tests/integration/test_ingestion_pipeline.py`。

F5: Pipeline 进度回调 (on_progress)

- **目标:** 在 `IngestionPipeline.run()` 方法中新增可选 `on_progress` 回调参数, 支持外部实时获取处理进度。
- **前置依赖:** F4 (Ingestion 打点)
- **修改文件:**
 - `src/ingestion/pipeline.py` (在各阶段调用 `on_progress(stage_name, current, total)`)
 - `tests/unit/test_pipeline_progress.py` (新增: 验证回调被正确调用)
- **实现要点:**
 - 回调签名: `on_progress(stage_name: str, current: int, total: int)`
 - `on_progress` 为 `None` 时完全不影响现有行为
 - 各阶段在处理每个 batch 或完成时触发回调
- **验收标准:** Pipeline 运行时传入 mock 回调, 断言各阶段均被调用且参数正确。
- **测试方法:** `pytest -q tests/unit/test_pipeline_progress.py`。

阶段 G: 可视化管理平台 Dashboard (目标: 六页面完整可视化 管理)

G1: Dashboard 基础架构与系统总览页

- **目标:** 搭建 Streamlit 多页面应用框架, 实现系统总览页面 (展示组件配置与数据统计) 。
- **前置依赖:** F1-F2 (Trace 基础设施)
- **修改文件:**
 - `src/observability/dashboard/app.py` (重写: 多页面导航架构)
 - `src/observability/dashboard/pages/overview.py` (新增: 系统总览页面)

- `src/observability/dashboard/services/config_service.py` (新增: 配置读取服务)
- `scripts/start_dashboard.py` (新增: Dashboard 启动脚本)
- 实现要点:
 - `app.py` 使用 `st.navigation()` 注册六个页面 (未完成的页面显示占位提示)
 - Overview 页面: 读取 `Settings` 展示组件卡片, 调用 `ChromaStore.get_collection_stats()` 展示数据统计
 - `ConfigService`: 封装 `Settings` 读取, 格式化组件配置信息
- 验收标准: `streamlit run src/observability/dashboard/app.py` 可启动, 总览页展示当前配置信息。
- 测试方法: 手动运行 `python scripts/start_dashboard.py` 并验证页面渲染。

G2: DocumentManager 实现

- 目标: 实现 `src/ingestion/document_manager.py`: 跨存储的文档生命周期管理 (list/delete/stats) 。
- 前置依赖: C5 (Pipeline + 各存储模块已就绪)
- 修改文件:
 - `src/ingestion/document_manager.py` (新增)
 - `src/libs/vector_store/chroma_store.py` (增强: 添加 `delete_by_metadata`)
 - `src/ingestion/storage/bm25_indexer.py` (增强: 添加 `remove_document`)
 - `src/libs/loader/file_integrity.py` (增强: 添加 `remove_record` + `list_processed`)
 - `tests/unit/test_document_manager.py` (新增)
- 实现类/函数:
 - `DocumentManager.__init__(chroma_store, bm25_indexer, image_storage, file_integrity)`
 - `DocumentManager.list_documents(collection?) -> List[DocumentInfo]`
 - `DocumentManager.get_document_detail(doc_id) -> DocumentDetail`
 - `DocumentManager.delete_document(source_path, collection) -> DeleteResult`
 - `DocumentManager.get_collection_stats(collection?) -> CollectionStats`
- 验收标准:
 - `list_documents` 返回已摄入文档列表 (source、chunk 数、图片数)
 - `delete_document` 协调删除 Chroma + BM25 + ImageStorage + FileIntegrity 四个存储
 - 删除后再次 list 不包含已删除文档
- 测试方法: `pytest -q tests/unit/test_document_manager.py`。

G3: 数据浏览器页面

- 目标: 实现 Dashboard 数据浏览器页面 (查看文档列表、Chunk 详情、图片预览) 。
- 前置依赖: G1 (Dashboard 架构) 、G2 (DocumentManager)
- 修改文件:
 - `src/observability/dashboard/pages/data_browser.py` (新增)

- `src/observability/dashboard/services/data_service.py` (新增: 封装 ChromaStore/ImageStorage 读取)
- **实现要点:**
 - 文档列表视图: 展示 `source_path`、集合、chunk 数、摄入时间; 支持集合筛选
 - Chunk 详情视图: 点击文档展开所有 chunk, 显示内容 (可折叠)、`metadata` 字段、关联图片
 - `DataService`: 封装 `ChromaStore.get_by_metadata()` 和 `ImageStorage.list_images()` 调用
- **验收标准:** 可在 Dashboard 中浏览已摄入的文档和 chunk 详情。
- **测试方法:** 手动验证 (先 ingest 样例数据, 再在 Dashboard 浏览)。

G4: Ingestion 管理页面

- **目标:** 实现 Dashboard Ingestion 管理页面 (文件上传触发摄取、进度展示、文档删除)。
- **前置依赖:** G2 (DocumentManager)、G3 (DataService)、F5 (on_progress 回调)
- **修改文件:**
 - `src/observability/dashboard/pages/ingestion_manager.py` (新增)
- **实现要点:**
 - 文件上传: `st.file_uploader` 选择文件 + 集合选择
 - 摄取触发: 调用 `IngestionPipeline.run(on_progress=...)` + `st.progress()` 实时进度
 - 文档删除: 在文档列表中提供删除按钮, 调用 `DocumentManager.delete_document()`
- **验收标准:** 可在 Dashboard 中上传文件触发摄取、看到实时进度条、删除已有文档。
- **测试方法:** 手动验证 (上传 PDF → 观察进度 → 删除 → 确认已移除)。

G5: Ingestion 追踪页面

- **目标:** 实现 Dashboard Ingestion 追踪页面 (摄取历史列表、阶段耗时瀑布图)。
- **前置依赖:** F4 (Ingestion 打点)、G1 (Dashboard 架构)
- **修改文件:**
 - `src/observability/dashboard/pages/ingestion_traces.py` (新增)
 - `src/observability/dashboard/services/trace_service.py` (新增: 解析 `traces.jsonl`)
- **实现要点:**
 - 历史列表: 按时间倒序展示 `trace_type == "ingestion"` 记录
 - 详情页: 横向条形图展示 `load/split/transform/embed/upsert` 耗时分布
 - `TraceService`: 读取 `logs/traces.jsonl`, 解析为 Trace 对象列表
- **验收标准:** 执行 ingest 后, Dashboard 显示对应的追踪记录与耗时瀑布图。
- **测试方法:** 手动验证 (先 ingest → 打开 Dashboard → 查看追踪)。

G6: Query 追踪页面

- **目标:** 实现 Dashboard Query 追踪页面 (查询历史、Dense/Sparse 对比、Rerank 变化)。
- **前置依赖:** F3 (Query 打点)、G1 (Dashboard 架构)、G5 (TraceService 已实现)
- **修改文件:**
 - `src/observability/dashboard/pages/query_traces.py` (新增)
- **实现要点:**
 - 历史列表: 按时间倒序展示 `trace_type == "query"` 记录, 支持按 Query 关键词搜索
 - 详情页: 耗时瀑布图 + Dense vs Sparse 并列对比 + Rerank 前后排名变化
- **验收标准:** 执行 query 后, Dashboard 显示查询追踪详情与各阶段对比。
- **测试方法:** 手动验证 (先 query → 打开 Dashboard → 查看追踪)。

阶段 H: 评估体系 (目标: 可插拔评估 + 可量化回归)

H1: RagasEvaluator 实现

- **目标:** 实现 `ragas_evaluator.py`: 封装 Ragas 框架, 实现 `BaseEvaluator` 接口。
- **修改文件:**
 - `src/observability/evaluation/ragas_evaluator.py` (新增)
 - `src/libs/evaluator/evaluator_factory.py` (注册 ragas provider)
 - `tests/unit/test_ragas_evaluator.py` (新增)
- **实现类/函数:**
 - `RagasEvaluator(BaseEvaluator)`: 实现 `evaluate()` 方法
 - 支持指标: Faithfulness, Answer Relevancy, Context Precision
 - 优雅降级: Ragas 未安装时抛出明确的 `ImportError` 提示
- **验收标准:** mock LLM 环境下, `evaluate()` 返回包含 faithfulness/answer_relevancy 的 metrics 字典。
- **测试方法:** `pytest -q tests/unit/test_ragas_evaluator.py`。

H2: CompositeEvaluator 实现

- **目标:** 实现 `composite_evaluator.py`: 组合多个 Evaluator 并行执行, 汇总结果。
- **修改文件:**
 - `src/observability/evaluation/composite_evaluator.py` (新增)
 - `tests/unit/test_composite_evaluator.py` (新增)
- **实现类/函数:**
 - `CompositeEvaluator.__init__(evaluators: List[BaseEvaluator])`
 - `CompositeEvaluator.evaluate() -> dict`: 并行执行所有 evaluator, 合并 metrics
 - 配置驱动: `evaluation.backends: [ragas, custom]` → 工厂自动组合
- **验收标准:** 配置两个 evaluator 时, 返回的 metrics 包含两者的指标。

- **测试方法:** `pytest -q tests/unit/test_composite_evaluator.py`。

H3: EvalRunner + Golden Test Set

- **目标:** 实现 `eval_runner.py`: 读取 `tests/fixtures/golden_test_set.json`, 跑 retrieval 并产出 metrics。
- **前置依赖:** D5 (HybridSearch) 、 H1-H2 (评估器)
- **修改文件:**
 - `src/observability/evaluation/eval_runner.py` (新增)
 - `tests/fixtures/golden_test_set.json` (新增: 黄金测试集)
 - `scripts/evaluate.py` (新增: 评估运行脚本)
- **实现类/函数:**
 - `EvalRunner.__init__(settings, hybrid_search, evaluator)`
 - `EvalRunner.run(test_set_path) -> EvalReport`: 运行评估并返回报告
 - `EvalReport`: 包含 hit_rate, mrr, 各 query 结果详情
- **golden_test_set.json 格式:**

```
{
  "test_cases": [
    {
      "query": "如何配置 Azure OpenAI?",
      "expected_chunk_ids": ["chunk_abc_001", "chunk_abc_002"],
      "expected_sources": ["config_guide.pdf"]
    }
  ]
}
```

- **验收标准:** `python scripts/evaluate.py` 可运行, 输出 metrics。
- **测试方法:** `pytest -q tests/integration/test_hybrid_search.py` 或 `python scripts/evaluate.py`。

H4: 评估面板页面

- **目标:** 实现 Dashboard 评估面板页面 (运行评估、查看指标、历史对比)。
- **前置依赖:** H3 (EvalRunner) 、 G1 (Dashboard 架构)
- **修改文件:**
 - `src/observability/dashboard/pages/evaluation_panel.py` (实现: 替换占位提示)
- **实现要点:**
 - 选择评估后端与 golden test set
 - 点击运行, 展示评估结果 (hit_rate、mrr、各 query 明细)
 - 可选: 历史评估结果对比图
- **验收标准:** 可在 Dashboard 中运行评估并查看指标。
- **测试方法:** 手动验证。

H5: Recall 回归测试 (E2E)

- **目标:** 实现 `tests/e2e/test_recall.py`: 基于 golden set 做最小召回阈值 (例如 hit@k)。
 - **前置依赖:** H3 (EvalRunner + golden_test_set)
 - **修改文件:**
 - `tests/e2e/test_recall.py` (新增)
 - `tests/fixtures/golden_test_set.json` (补齐若干条)
 - **验收标准:** hit@k 达到阈值 (阈值写死在测试里, 便于回归)。
 - **测试方法:** `pytest -q tests/e2e/test_recall.py`。
-

阶段 I: 端到端验收与文档收口 (目标: 开箱即用的"可复现"工程)

I1: E2E: MCP Client 侧调用模拟

- **目标:** 实现 `tests/e2e/test_mcp_client.py`: 以子进程启动 server, 模拟 tools/list + tools/call。
- **修改文件:**
 - `tests/e2e/test_mcp_client.py`
- **验收标准:** 完整走通 query_knowledge_hub 并返回 citations。
- **测试方法:** `pytest -q tests/e2e/test_mcp_client.py`。

I2: E2E: Dashboard 冒烟测试

- **目标:** 验证 Dashboard 各页面在有数据时可正常渲染、无 Python 异常。
- **修改文件:**
 - `tests/e2e/test_dashboard_smoke.py` (新增)
- **实现要点:**
 - 使用 Streamlit 的 `AppTest` 框架进行自动化冒烟测试
 - 验证 6 个页面均可加载、不抛异常
- **验收标准:** 所有页面冒烟测试通过。
- **测试方法:** `pytest -q tests/e2e/test_dashboard_smoke.py`。

I3: 完善 README (运行说明 + 测试说明 + MCP 配置 + Dashboard 使用)

- **目标:** 让新用户能在 10 分钟内跑通 ingest + query + dashboard + tests, 并能在 Copilot/Claude 中使用。
- **修改文件:**
 - `README.md`
- **验收标准:** README 包含以下章节:
 - **快速开始:** 安装依赖、配置 API Key、运行首次摄取
 - **配置说明:** `settings.yaml` 各字段含义

- **MCP 配置示例**: GitHub Copilot `mcp.json` 与 Claude Desktop `claude_desktop_config.json`
- **Dashboard 使用指南**: 启动命令、各页面功能说明、截图示例
- **运行测试**: 单元测试、集成测试、E2E 测试命令
- **常见问题**: API Key 配置、依赖安装、连接问题排查
- **测试方法**: 按 README 手动走一遍。

I4: 清理接口一致性 (契约测试补齐)

- **目标**: 为关键抽象 (VectorStore / Reranker / Evaluator / DocumentManager) 补齐契约测试。
- **修改文件**:
 - `tests/unit/test_vector_store_contract.py` (补齐 `delete_by_metadata` 边界)
 - `tests/unit/test_reranker_factory.py` (补齐边界)
 - `tests/unit/test_custom_evaluator.py` (补齐边界)
- **验收标准**: `pytest -q` 全绿, 且 contract tests 覆盖主要输入输出形状。
- **测试方法**: `pytest -q`。

I5: 全链路 E2E 验收

- **目标**: 执行完整的端到端验收流程: ingest → query via MCP → Dashboard 可视化 → evaluate。
- **修改文件**: 无新文件, 验收已有功能
- **验收标准**:
 - `python scripts/ingest.py --path tests/fixtures/sample_documents/ --collection test` 成功
 - `python scripts/query.py --query "测试查询" --verbose` 返回结果
 - Dashboard 可展示摄取与查询追踪
 - `python scripts/evaluate.py` 输出评估指标
- **测试方法**: 手动全链路走通 + `pytest -q` 全量测试。

交付里程碑 (建议)

- **M1 (完成阶段 A+B)**: 工程可测 + 可插拔抽象层就绪, 后续实现可并行推进。
- **M2 (完成阶段 C)**: 离线摄取链路可用, 能构建本地索引。
- **M3 (完成阶段 D+E)**: 在线查询 + MCP tools 可用, 可在 Copilot/Claude 中调用。
- **M4 (完成阶段 F)**: Ingestion + Query 双链路可追踪, JSON Lines 持久化。
- **M5 (完成阶段 G)**: 六页面可视化管理平台就绪 (评估面板为占位), 数据可浏览、可管理、链路可追踪。
- **M6 (完成阶段 H+I)**: 评估体系完整 + E2E 验收通过 + 文档完善, 形成"面试/教学/演示"可复现项目。

7. 可扩展性与未来展望

7.1 云端部署与后端架构学习

虽然当前阶段我们主要采用“本地运行”模式，但本项目的架构设计完全支持向云端迁移。这也是一个极佳的学习后端工程化的切入点。

- **Server 容器化**：计划编写 Dockerfile，将 MCP Server 打包为容器。这让我们有机会深入理解 Python 环境隔离、依赖管理以及 Docker 的最佳实践。
- **云端接入**：未来可以将 Server 部署至 Azure Container Apps 或 AWS Lambda。
 - **挑战与学习点**：处理网络延时、配置 API Gateway、增加 AuthN/AuthZ 鉴权机制（保护私有数据不被公开访问）。
- **多租户与并发**：从单用户本地服务转变为支持团队共享的服务。
 - **学习点**：在 Chroma 中实现 Namespace 隔离、处理并发请求锁、优化 embedding 缓存策略。

7.2 业务深耕：从“通用”到“垂直” (Vertical Domain Adaptation)

RAG 系统的上限取决于其对特定业务数据的理解深度。未来的核心扩展方向是将通用的技术框架与具体的业务场景深度结合。在将本项目应用到实际生产环境时，识别并解决以下“最后一公里”的难题，将是提升系统价值的关键：

- **多源异构数据的复杂适配**：
 - 现实业务中不仅有 PDF，还大量存在 PPTX, DOCX, XLSX 甚至 HTML 数据。
 - **挑战**：如何处理不同格式的特有语义？例如 PPT 中的演讲者备注往往比正文更关键，Excel 中的公式逻辑与跨行关联如何保留？目前的通用处理方式容易丢失这些“隐性知识”，未来需要针对每种格式探索更深度的解析能力。
- **复杂结构化数据的精确理解**：
 - 简单的文本切分（Chunking）在处理表格、层级列表时往往会破坏语义。
 - **挑战**：
 - **表格理解**：如何处理跨页长表格、合并单元格以及含有复杂表头的财务报表？如果切分不当，检索时只能找到数字却不知道对应的列名（指标含义）。
 - **上下文断裂**：当一个完整的逻辑段落（如合同条款）被切分到两个 chunk 时，如何保证检索其中一段时能感知到整体的上下文约束？
- **业务逻辑驱动的生成控制**：
 - 仅仅根据“相似度”召回文档在企业级场景中往往不够。
 - **挑战**：
 - **时效性与版本管理**：当知识库中同时存在“2023版”和“2024版”规章时，如何确保系统不会混淆历史数据与最新标准？
 - **权限与受众适配**：面对内部员工与外部客户，如何控制生成答案的详略程度与敏感信息披露？
 - **拒答机制**：当召回内容的置信度不足时，如何让系统诚实地回答“不知道”而不是基于相关性较低的片段强行拼凑答案（幻觉问题）？

7.3 迈向自主智能：Agentic RAG 的演进路径

当前的 RAG 架构主要遵循“一次检索-一次生成”的固有范式，但在面对极其复杂的问题（如跨文档对比、多步推理）时，单一的线性流程往往力不从心。本项目作为标准的 MCP Server，天然具备向 **Agentic RAG（代理式 RAG）** 演进的潜力。这不需要重写现有代码，而是通过在 Server 端提供更细粒度的工具，赋能 Client 端的 Agent 具备更强的自主性：

- **从“单步检索”到“多步决策”：**
 - 目前 Agent 可能只调用一个通用的 `search` 工具。
 - **未来演进：** Server 可以暴露如 `list_directory`（查看目录结构）、`preview_document`（预览摘要）、`verify_fact`（事实核查）等更原子化的工具。Agent 可以像人类研究员一样，先看目录圈定范围，再针对性阅读，最后交叉验证信息，从而解决复杂问题。
- **让 Agent 具备“反思”能力：**
 - **未来演进：** 利用现有的评估模块，Server 可以提供一个 `self_check` 接口。Agent 在生成答案后，可以自主调用该接口检测是否存在幻觉，或者检索结果是否真正支撑了论点。如果发现不足，Agent 可以自主决定进行第二轮更深度的搜索。
- **动态策略选择：**
 - **未来演进：** 不再硬编码使用混合检索。Server 可以将 `keyword_search` 和 `semantic_search` 作为独立工具暴露。Agent 可以根据用户意图自主判断：如果是搜人名，只用关键词搜；如果是搜概念，通过语义搜。这种工具使用的灵活性正是 Agentic RAG 的核心魅力。

这种演进方向将把本项目从一个“智能搜索引擎”升级为一个“智能研究助理”的基础设施底座。